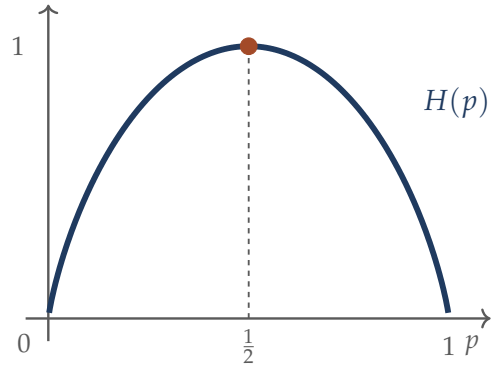
Informatik von Grund auf — Band 7

Das Maß der Nachricht

Eine vollständige Einführung in die Informationstheorie



Von der Entropie bis zur Kanalkapazität

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 7
Das Maß der Nachricht · Informationstheorie

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 7: Das Maß der Nachricht*. Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 158 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Das Maß 7

1 Was Information ist 9

2 Entropie 17

3 Was zwei Größen voneinander wissen 27

II Verdichten 33

4 Codes 35

5 Der Quellencodierungssatz 43

6 Verfahren zur Verdichtung 51

7 Wörterbücher und Modelle 61

8 Wenn man etwas verlieren darf 69

III Der Kanal 79

9 Kanäle und Kapazität 81

10 Der Kanalcodierungssatz 89

11 Fehler erkennen und beheben 95

12 Codes, die die Grenze erreichen 105

IV Weiterungen 115

13 Die Information einer einzelnen Folge 117

14 Dieselbe Größe in anderen Fächern 125

15 Der Stand im Jahr 2026 **133**

V Anhang **139**

A Formeln und Zahlen **141**

B Landkarte des Bandes **145**

C Glossar **147**

D Zeittafel **149**

E Kommentierte Quellen **151**

Vorwort: Was in einer Nachricht steckt

Dieser Band handelt von einer Größe, die man messen kann, ohne zu verstehen, wovon die Rede ist. *Wie viel Information eine Nachricht enthält, hängt nicht davon ab, was sie bedeutet.*

Zwei Nachrichten von gleicher Länge

Beide bestehen aus zwanzig Zeichen.

	Text	lässt sich kürzen auf
A	`aaaaaaaaaaaaaaaaaaaa`	≈ 3 **Byte**
B	`qzt7vk3mxb9wnf2rjhs4`	≈ 13 Byte

Die erste Zeile lässt sich beschreiben als „zwanzigmal a“. Bei der zweiten hilft keine Beschreibung, die kürzer wäre als die Zeichenfolge selbst. Der Unterschied ist messbar, und die Größe, die ihn misst, heißt Entropie.

Die Kernfrage dieses Bandes.

Verdichten	Wie kurz kann ich eine Nachricht machen, ohne etwas zu verlieren?
Übertragen	Wie viel kann ich über eine gestörte Leitung schicken, ohne Fehler?

Auf beide Fragen gibt es eine exakte Antwort, und beide Antworten stammen aus derselben Arbeit von 1948. Das ist bemerkenswert, denn die Fragen sehen einander nicht ähnlich.

Der Zufallszahlengenerator, der keiner war

Ein einfaches Beispiel dafür, dass Information nichts mit Bedeutung zu tun hat.

Quelle	sieht aus wie	Entropie je Zeichen
Münzwurf	zufällig	1,00 bit
Gezinkte Münze ($p = 0{,}9$)	fast immer Kopf	0,47 bit
Deutscher Text	sinnvoll	$\approx$ 1,3 bit
Deutscher Text, Buchstaben-häufigkeit allein	sinnvoll	$\approx$ 4,1 bit

Die beiden letzten Zeilen beschreiben denselben Text. Der Unterschied liegt allein darin, wie viel der Empfänger über die Sprache weiß. Wer nur die Buchstabenhäufigkeiten kennt, ist häufiger überrascht als jemand, der Wörter und Grammatik kennt.

Daraus folgt etwas Unbequemes.
Die Entropie ist keine Eigenschaft der Nachricht allein, sondern des Verhältnisses zwischen Nachricht und Empfänger.

Naiv	„Diese Datei enthält n bit Information“
Genauer	„Bezüglich dieses Modells enthält sie n bit“

Ein besseres Modell senkt die Entropie und damit die erreichbare Dateigröße. Genau darum dreht sich der ganze Teil II dieses Bandes: Verdichten heißt vorhersagen.

Was dieses Buch will

Erstens: Die Größe herleiten, statt sie hinzuschreiben.
Die Formel für die Entropie fällt in vielen Darstellungen vom Himmel. Sie lässt sich aber aus drei ganz schlichten Forderungen ableiten, und dann ist sie die einzig mögliche. Kapitel 2 führt das vollständig durch.

Zweitens: Beide Sätze von Shannon beweisen.

> *Der Quellencodierungssatz und der Kanalcodierungssatz sind die beiden Ergebnisse, um die es geht. Ich zeige beide vollständig, den zweiten in der Fassung mit zufälliger Codewahl, die einfacher ist als ihr Ruf.*

Drittens: Zeigen, wo dieselbe Größe wieder auftaucht.
Entropie kommt in der Thermodynamik vor, in der Statistik, in der Neurowissenschaft und in der Frage, was ein Programm über eine Zeichenfolge aussagen kann. Kapitel 13 und 14 gehen diesen Verbindungen nach.

Drei Warnungen

Erstens: „Information" heißt hier nicht, was es umgangssprachlich heißt.
Eine Nachricht mit viel Information kann völlig sinnlos sein. Zufälliges Rauschen hat die höchste Entropie überhaupt. Wer das verwechselt, kommt bei jeder zweiten Aussage dieses Bandes ins Straucheln.
Zweitens: Die Sätze sagen, was möglich ist, nicht wie.

> *Shannon bewies 1948, dass fehlerfreie Übertragung nahe an der Kapazität möglich ist. Codes, die das praktisch erreichen, gab es erst ab 1993. Dazwischen liegen fünfundvierzig Jahre.*

Drittens: Die Logarithmen sind zur Basis zwei.
Wenn nicht ausdrücklich etwas anderes steht, ist log in diesem Band immer $\log_2$, und die Einheit ist das bit. Andere Basen ergeben andere Einheiten und ändern sonst nichts.

Zum Aufbau

- **Teil I** (Kap. 1–3): Das Maß. Was Information ist, was Entropie ist, und wie sich zwei Größen gegenseitig etwas verraten.
- **Teil II** (Kap. 4–8): Verdichten. Codes, der Quellencodierungssatz, die Verfahren, und was geschieht, wenn man etwas verlieren darf.
- **Teil III** (Kap. 9–12): Übertragen. Kanäle, Kapazität, der Kanalcodierungssatz und die Codes, die ihn einlösen.
- **Teil IV** (Kap. 13–15): Weiterungen. Algorithmische Information, dieselbe Größe in anderen Fächern, und der Stand im Jahr 2026.

Eine Arbeit, die ein Fach begründete

Im Juli und Oktober 1948 erschien im *Bell System Technical Journal* ein zweiteiliger Aufsatz von CLAUDE SHANNON mit dem Titel „A Mathematical Theory of Communication".

Er war siebenundfünfzig Seiten lang und enthielt ein vollständiges Fach: die Definition der Entropie, den Quellencodierungssatz, die Kanalkapazität, den Kanalcodierungssatz und die Erweiterung auf zeitkontinuierliche Signale.

> *Das ist ungewöhnlich. Die meisten Fächer entstehen über Jahrzehnte aus vielen Arbeiten. Hier stand fast alles Wesentliche auf einmal da, und die folgenden Jahrzehnte bestanden zu einem guten Teil darin, es zu verstehen und praktisch einzulösen.*

Was vorher da war.

Jahr	Wer	Was
1924	NYQUIST	Wie viele Signalstufen je Sekunde
1928	HARTLEY	Information als Logarithmus der Anzahl
1946	WIENER	Statistische Nachrichtentheorie

Hartleys Maß zählte die Zahl der möglichen Nachrichten und nahm ihren Logarithmus. Das war richtig und behandelte alle Nachrichten als gleich wahrscheinlich.

> *Shannons Schritt bestand darin, die Wahrscheinlichkeiten einzubeziehen. Damit wurde aus einer Zählung eine Erwartung, und aus einer technischen Faustregel eine Theorie.*

Woher der Name kommt.
Shannon war unschlüssig, wie er seine Größe nennen sollte. Nach einer von ihm selbst erzählten Anekdote riet ihm JOHN VON NEUMANN, sie Entropie zu nennen: erstens habe die Funktion in der statistischen Mechanik denselben Bau, und zweitens wisse ohnehin niemand genau, was Entropie sei, was ihm in Diskussionen einen Vorteil verschaffe.

Die Geschichte ist möglicherweise ausgeschmückt. Der formale Zusammenhang zur Thermodynamik ist es nicht; Kapitel 14 geht ihm nach.

Was aus dem Wort „bit“ wurde.
Der Ausdruck stammt von JOHN TUKEY und erscheint in Shannons Arbeit als Kurzform für *binary digit*. Er bezeichnet dort eine Menge an Information, nicht eine Speicherstelle.

Speicherstelle	Ein Behälter, der 0 oder 1 aufnimmt
Informationsmenge	Was man erfährt, wenn eine faire Münze fällt

Beide heißen bit, und sie stimmen nur überein, wenn die Speicherstelle beide Werte gleich häufig annimmt. Eine Speicherstelle, die zu neunzig Prozent Null enthält, trägt weniger als ein halbes bit.

Shannon arbeitete bei Bell Labs, an denselben Laboratorien, aus denen zwei Jahrzehnte später Unix hervorging (Band 5). Er war außerdem für seine Basteleien bekannt: eine jonglierende Maschine, ein Einrad, und ein Kästchen, dessen einzige Funktion darin bestand, sich nach dem Einschalten selbst wieder auszuschalten.

Achtzig Jahre Informationstheorie

1924	Nyquist	Grenze der Signalisierungsrate
1928	Hartley	Information als Logarithmus
1948	**Shannon**	**A Mathematical Theory of Communication**
1949	Shannon	Theorie der Geheimhaltung
1950	Hamming	fehlerkorrigierende Codes
1951	Shannon	Entropie der englischen Sprache
1952	**Huffman**	**optimaler Präfixcode**
1955	Elias	Faltungscodes
1960	Reed, Solomon	Codes über endlichen Körpern
1961	Landauer	Löschen kostet Energie
1961	Barlow	sparsame Codierung im Nervensystem
1962	**Gallager**	**LDPC-Codes, dann vergessen**
1964	Solomonoff	induktive Inferenz
1965	**Kolmogorow**	**algorithmische Information**
1966	Chaitin	dasselbe, unabhängig
1968	Berlekamp, Massey	Decodierung von RS-Codes
1976	Rissanen	arithmetische Codierung

1977	**Ziv, Lempel**	**LZ77: Verdichten ohne Vorwissen**
1978	Rissanen	kürzeste Beschreibung als Modellwahl
1978	Ziv, Lempel	LZ78
1984	Welch	LZW, in `compress` und GIF
1987	Cleary, Witten	PPM: Vorhersage aus dem Kontext
1991	Deutsch	`deflate`, in `zip` und `gzip`
1993	**Berrou u. a.**	**Turbo-Codes: nahe an der Grenze**
1996	MacKay, Neal	LDPC wiederentdeckt
1996	Burrows, Wheeler	Blocksortierung, in `bzip2`
2003	DVB-S2	LDPC im Rundfunk
2009	**Arıkan**	**Polare Codes: beweisbar optimal**
2009	Duda	asymmetrische Zahlensysteme
2015	Zstandard	ANS im Massengebrauch
2016	5G	polare Codes im Mobilfunk
2020	Neuronale Verdichtung	Modelle als Vorhersager
2026	—	Verdichtung und Vorhersage als dasselbe

Zwei Zeilen verdienen eine Anmerkung.
1962, Gallager. Er beschrieb in seiner Doktorarbeit Codes, die fast an die Kanalkapazität heranreichen. Sie waren für die Rechner der Zeit zu aufwendig und gerieten dreißig Jahre lang in Vergessenheit. Erst 1996 fand man sie wieder, und heute stecken sie in jeder Satellitenverbindung.
1977, Ziv und Lempel. Ihr Verfahren braucht keine Kenntnis der Quelle. Es lernt beim Lesen und erreicht trotzdem asymptotisch die Entropie. Fast jedes Verdichtungsprogramm, das heute läuft, geht darauf zurück.

Die grundlegende Aufgabe der Nachrichtenübertragung besteht darin, an einem Punkt eine Nachricht wiederzugeben, die an einem anderen Punkt ausgewählt wurde.

— sinngemäß der erste Satz von CLAUDE SHANNON, 1948.
Der entscheidende Ausdruck ist ausgewählt*: Es geht nicht darum, was die Nachricht bedeutet, sondern darum, aus wie vielen sie ausgewählt wurde.*

Part I

Das Maß

1 Was Information ist

Überraschung als Maß

Der Ausgangspunkt ist eine Beobachtung, die jeder teilt: Eine Nachricht, die man ohnehin erwartet hat, sagt einem wenig.

Nachricht	erwartet?	Informationsgehalt
„Morgen geht die Sonne auf“	sicher	**null**
„Morgen regnet es“	oft	wenig
„Morgen schneit es im Juli“	selten	viel

Der Informationsgehalt eines Ereignisses soll also mit sinkender Wahrscheinlichkeit steigen. Damit ist er noch nicht festgelegt; es gibt viele Funktionen mit dieser Eigenschaft.
Die zweite Forderung entscheidet.
Zwei unabhängige Ereignisse zusammen sollen so viel Information tragen wie beide einzeln zusammengenommen.

$$I(x \text{ und } y) = I(x) + I(y) \qquad \text{bei Unabhängigkeit.}$$

Zugleich gilt für unabhängige Ereignisse $P(x, y) = P(x) \cdot P(y)$. Gesucht ist also eine Funktion, die ein Produkt in eine Summe verwandelt.

> *Das leistet genau eine Funktionenfamilie, nämlich der Logarithmus. Damit ist das Maß bis auf die Basis festgelegt, und die Basis bestimmt nur die Einheit.*

Definition 1.1. Der **Informationsgehalt** eines Ereignisses x mit der Wahrscheinlichkeit $P(x)$ ist

$$I(x) = -\log_2 P(x) = \log_2 \frac{1}{P(x)} \quad \text{bit.}$$

$P(x)$	$I(x)$	**Beispiel**
1	0 bit	sicheres Ereignis
$\frac{1}{2}$	1 bit	Münzwurf
$\frac{1}{8}$	3 bit	drei Münzwürfe
$\frac{1}{1000}$	9,97 bit	
$\to 0$	$\to \infty$	das Unmögliche

Die letzte Zeile ist kein Mangel, sondern angemessen: Wenn etwas eintritt, das man für unmöglich hielt, erfährt man beliebig viel, nämlich dass das eigene Modell falsch war.

Die Wahl der Basis

Basis	**Einheit**	**benannt nach**	**Umrechnung**
2	**bit**	—	1
e	nat	—	1,443 bit
10	dit, hartley	Hartley	3,322 bit
3	trit	—	1,585 bit

Die Wahl ist reine Konvention, wie die Wahl zwischen Metern und Fuß. In der Technik nimmt man bit, in Beweisen oft nat, weil die Ableitung von ln *handlicher ist.*

Warum „bit“ zwei Dinge bedeutet

	Speicherstelle	Informationsmenge
Ist	ein Behälter	eine Zahl
Nimmt an	0 oder 1	jeden Wert ≥ 0
Beispiel	ein Flipflop	0,47 bit
Zusammenhang	Eine Speicherstelle trägt **höchstens** ein bit Information	

Die letzte Zeile ist der ganze Grund für die Verdichtung. Wenn eine Speicherstelle im Mittel weniger als ein bit trägt, verschwendet man Platz, und man kann ihn zurückholen.

Umgekehrt lässt sich ein bit Information nicht in weniger als einer Speicherstelle unterbringen. Auch das ist ein Satz und keine Faustregel; Kapitel 5 beweist ihn.

Hartleys Vorläufer

1928 schlug RALPH HARTLEY vor, den Informationsgehalt einer Auswahl aus n gleich wahrscheinlichen Möglichkeiten als $\log n$ zu messen.

Auswahl aus	Hartley	Bemerkung
2	1 bit	
256	8 bit	ein Byte
26	4,70 bit	ein Buchstabe
10^6	19,93 bit	

Warum log **und nicht** n **selbst?**
Hartleys Begründung ist praktisch. Wer zwei Zeichen aus je n Möglichkeiten überträgt, hat n^2 mögliche Nachrichten. Das Maß soll sich dabei verdoppeln und nicht quadrieren.

$$\log(n^2) = 2\log n. \qquad \checkmark$$

Es ist dieselbe Überlegung wie oben, nur ohne Wahrscheinlichkeiten. Hartleys Maß ist der Sonderfall von Shannons Maß, in dem alle Möglichkeiten gleich wahrscheinlich sind.

	Hartley 1928	**Shannon 1948**
Betrachtet	Anzahl	**Wahrscheinlichkeiten**
Maß	$\log n$	$-\sum p_i \log p_i$
Gleichverteilung	$\log n$	$\log n$ *identisch*
Sonst	—	kleiner

Die letzte Zeile enthält den ganzen Fortschritt. Sobald die Möglichkeiten unterschiedlich häufig sind, liegt der wahre Informationsgehalt unter Hartleys Wert, und die Differenz ist der Spielraum für die Verdichtung.

Aufgabe 1.1.

(a) Berechnen Sie Informationsgehalte für mehrere Ereignisse.
(b) Bestimmen Sie, wie viele Fragen man braucht, um eine Zahl zu erraten.
(c) Untersuchen Sie das Wägeproblem mit einer Balkenwaage.

Lösung Schritt für Schritt.

(a) Informationsgehalte.
Ein Würfelwurf.

$$P(x) = \tfrac{1}{6}, \qquad I(x) = \log_2 6 = 2{,}585 \text{ bit}.$$

Der Wert ist nicht ganzzahlig, und das ist in Ordnung. Er sagt: Um sehr viele Würfelwürfe zu übertragen, braucht man im Mittel 2,585 Speicherstellen je Wurf. Ein einzelner Wurf braucht drei.

Die Probe mit mehreren Würfen.

Würfe	**Möglichkeiten**	**nötige bit**	**je Wurf**
1	6	3	3,000
3	216	8	2,667
10	$6{,}05 \cdot 10^7$	26	2,600
100	$6{,}5 \cdot 10^{77}$	259	**2,590**

Die letzte Spalte nähert sich 2,585*. Das ist der Kern des Quellencodierungssatzes: Die Grenze wird erreicht, wenn man viele Zeichen zusammen behandelt, nicht einzeln.*

Zwei Ereignisse im Vergleich.

Ereignis	P	I
Sechser im Lotto	$7{,}15 \cdot 10^{-8}$	23,7 bit
Ein Buchstabe im Deutschen	—	$\approx$ 4,1 bit
Ein zufälliges Byte	2^{-8}	8 bit
Blitzschlag pro Person und Jahr	$\approx 10^{-6}$	19,9 bit

Ein Lottogewinn trägt also etwa so viel Information wie drei zufällige Bytes. Das klingt entwertend und ist genau der Punkt: Das Maß misst Unwahrscheinlichkeit, nicht Bedeutung.

(b) Zahlenraten.
Die Aufgabe. Jemand denkt sich eine Zahl zwischen 1 und 1000. Wie viele Ja-Nein-Fragen braucht man?
Die untere Schranke.

$$I = \log_2 1000 = 9{,}966 \text{ bit}.$$

Jede Ja-Nein-Frage liefert höchstens ein bit, also braucht man mindestens $\lceil 9{,}966 \rceil = 10$ Fragen.
Warum höchstens ein bit.
Eine Frage mit den Antworten „ja“ mit Wahrscheinlichkeit p und „nein“ mit $1 - p$ liefert im Mittel

$$H(p) = -p \log p - (1-p) \log(1-p) \leq 1,$$

mit Gleichheit genau bei $p = \frac{1}{2}$.

Das ist die Funktion vom Titelbild dieses Bandes. Sie sagt: Eine Frage ist genau dann am ergiebigsten, wenn man ihre Antwort am wenigsten vorhersehen kann.

Die Strategie, die das erreicht.

Frage	Bereich	Größe	Rest-bit
—	1–1000	1000	9,97
1	1–500	500	8,97
2	251–500	250	7,97
5	—	32	5,00
9	—	2	1,00
10	—	1	**0**

Eine schlechte Strategie zum Vergleich.
Wer der Reihe nach fragt („Ist es 1?“, „Ist es 2?“), braucht im Mittel 500 Fragen.

Strategie	Fragen im Mittel	bit je Frage
Halbieren	10	**1,00**
Drittelung	$\approx 12{,}6$	0,79
Der Reihe nach	500	0,02

Die letzte Zeile zeigt, warum die lineare Suche so schlecht ist. Die Frage „Ist es 1?“ wird in 999 von 1000 Fällen mit Nein beantwortet und trägt deshalb fast keine Information.

Damit ist die untere Schranke für vergleichsbasiertes Sortieren aus Band 1 nichts anderes als eine Aussage der Informationstheorie: $n!$ mögliche Anordnungen brauchen
$$\log_2(n!) \approx n \log_2 n \text{ Vergleiche.}$$

(c) Das Wägeproblem.
Die Aufgabe. Unter zwölf Kugeln ist eine, die ein anderes Gewicht hat. Man weiß nicht, ob sie schwerer oder leichter ist. Wie viele Wägungen mit einer Balkenwaage braucht man?
Die Informationsrechnung.
Zahl der möglichen Antworten:

$$12 \text{ Kugeln} \times 2 \text{ Richtungen} = 24.$$

$$I = \log_2 24 = 4{,}585 \text{ bit.}$$

Eine Wägung hat drei mögliche Ausgänge: links schwerer, rechts schwerer, Gleichgewicht.

$$\text{höchstens } \log_2 3 = 1{,}585 \text{ bit je Wägung.}$$

$$\frac{4{,}585}{1{,}585} = 2{,}89 \quad \Rightarrow \quad \text{mindestens } \mathbf{3} \text{ Wägungen.}$$

Die Rechnung sagt, dass drei Wägungen nicht ausgeschlossen sind. Sie sagt nicht, dass es geht. Dass es tatsächlich geht, muss man durch Angabe eines Verfahrens zeigen.

Das Verfahren. Kugeln 1 bis 12; wir wägen $\{1,2,3,4\}$ gegen $\{5,6,7,8\}$.

Ausgang	**Möglichkeiten**	**Rest-bit**
Gleichgewicht	8 (9–12, beide Richtungen)	3,00
Links schwerer	8 (1–4 schwer, 5–8 leicht)	3,00
Rechts schwerer	8	3,00

Alle drei Ausgänge sind gleich wahrscheinlich, also liefert die Wägung volle 1,585 *bit. Genau darauf muss man beim Entwurf achten: Die Gruppen so wählen, dass die drei Fälle gleich groß sind.*

Der Fall „Gleichgewicht", zweite Wägung. Wäge $\{9,10,11\}$ gegen $\{1,2,3\}$ (bekannt gut).

Ausgang	**Möglichkeiten**	**dritte Wägung**
Gleichgewicht	12 schwer oder leicht (2)	12 gegen 1
Links schwerer	9, 10, 11 schwer (3)	9 gegen 10
Rechts schwerer	9, 10, 11 leicht (3)	9 gegen 10

Bei drei Möglichkeiten und drei Ausgängen genügt eine Wägung. Damit ist das Verfahren vollständig.

Wo die Rechnung an ihre Grenze kommt

Bei dreizehn Kugeln ergibt dieselbe Rechnung

$$\log_2 26 = 4{,}70 \text{ bit}, \qquad \frac{4{,}70}{1{,}585} = 2{,}97 < 3.$$

Die Schranke erlaubt also drei Wägungen, und trotzdem geht es nicht. Der Grund ist, dass sich die erste Wägung nicht in drei gleich große Fälle aufteilen lässt: 26 *ist nicht durch drei teilbar.*

Kugeln	**Möglichkeiten**	**3 Wägungen?**
12	$24 \le 27$	ja
13	$26 \le 27$	nein
13 mit Vergleichskugel	26	ja

Die Informationsschranke ist notwendig und nicht hinreichend. Sie sagt, was ausgeschlossen ist, und lässt offen, ob der Rest erreichbar ist. Diese Unterscheidung zieht sich durch den ganzen Band: Auch Shannons Sätze sagen, was möglich ist, ohne ein Verfahren anzugeben.

Hartleys Aufsatz von 1928[12] enthält den Vorläufer des Maßes; Shannons Arbeit[24] beginnt mit genau dieser Motivation; Cover und Thomas[6] entwickeln das Maß streng; Pierce[20] erklärt es ohne Formeln und bemerkenswert klar.

2 Entropie

Vom Einzelfall zum Mittelwert

Kapitel 1 maß den Gehalt eines einzelnen Ereignisses. Für eine Quelle, die immer wieder sendet, interessiert der Mittelwert.

Definition 2.1. Die **Entropie** einer Quelle mit den Zeichen $x_1, \ldots, x_n$ und Wahrscheinlichkeiten $p_1, \ldots, p_n$ ist

$$H(X) = \sum_{i=1}^{n} p_i \cdot I(x_i) = -\sum_{i=1}^{n} p_i \log_2 p_i \quad \text{bit je Zeichen.}$$

Dabei wird $0 \log 0 = 0$ gesetzt.

> *Die Entropie ist also nichts anderes als der Erwartungswert der Überraschung. Zeichen, die selten kommen, überraschen stark, tragen aber selten bei; Zeichen, die häufig kommen, überraschen wenig und tragen oft bei.*

Die Festsetzung $0 \log 0 = 0$.
Sie ist keine Willkür, sondern der Grenzwert:

$$\lim_{p \to 0^+} p \log p = 0.$$

Ein Zeichen, das nie vorkommt, trägt zum Mittelwert nichts bei, obwohl sein Einzelgehalt unendlich wäre.

Quelle	H	**Bemerkung**
Faire Münze	1,000 bit	Höchstwert bei $n = 2$
Münze mit $p = 0{,}9$	0,469 bit	
Münze mit $p = 0{,}99$	0,081 bit	
Fester Wert	0 bit	keine Nachricht
Fairer Würfel	2,585 bit	$= \log_2 6$
26 Buchstaben gleichverteilt	4,700 bit	

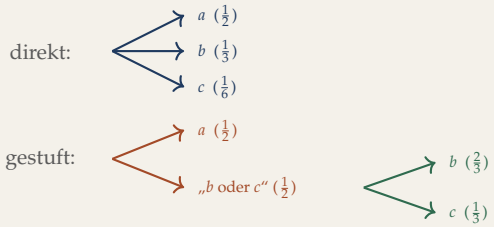
Die Formel ist die einzig mögliche

Man kann die Entropie aus drei Forderungen herleiten, ohne sie vorauszusetzen. SHANNON tat das 1948 im Anhang seiner Arbeit.

Stetigkeit	H soll sich bei kleiner Änderung der p_i nur wenig ändern
Monotonie	Bei n gleich wahrscheinlichen Zeichen soll H mit n wachsen
Zerlegbarkeit	Fasst man eine Wahl in zwei Stufen, so sollen sich die Beiträge gewichtet addieren

Die dritte Forderung ist die entscheidende und die am wenigsten selbstverständliche. Sie besagt: Es darf nicht darauf ankommen, in welcher Reihenfolge man sich entscheidet.

Die Zerlegbarkeit an einem Beispiel.
Eine Quelle mit drei Zeichen und den Wahrscheinlichkeiten $\frac{1}{2}, \frac{1}{3}, \frac{1}{6}$.



Die Forderung lautet

$$H\left(\tfrac{1}{2}, \tfrac{1}{3}, \tfrac{1}{6}\right) = H\left(\tfrac{1}{2}, \tfrac{1}{2}\right) + \tfrac{1}{2} \cdot H\left(\tfrac{2}{3}, \tfrac{1}{3}\right).$$

Die Probe.

$$\begin{aligned} \text{links} &= -\tfrac{1}{2}\log\tfrac{1}{2} - \tfrac{1}{3}\log\tfrac{1}{3} - \tfrac{1}{6}\log\tfrac{1}{6} \\ &= 0{,}5000 + 0{,}5283 + 0{,}4308 = 1{,}4591. \end{aligned}$$

$$\begin{aligned} \text{rechts} &= 1 + \tfrac{1}{2}\left(-\tfrac{2}{3}\log\tfrac{2}{3} - \tfrac{1}{3}\log\tfrac{1}{3}\right) \\ &= 1 + \tfrac{1}{2} \cdot 0{,}9183 = 1{,}4591. \qquad \checkmark \end{aligned}$$

Der Faktor $\frac{1}{2}$ vor der zweiten Entropie ist der Schlüssel. Die zweite Entscheidung fällt nur in der Hälfte der Fälle an, also zählt sie auch nur zur Hälfte.

Shannon zeigte, dass aus diesen drei Forderungen zwingend $H = -K\sum p_i \log p_i$ folgt, mit einer positiven Konstanten K, die nur die Einheit festlegt. Es gibt kein anderes Maß, das alle drei erfüllt.

Vier Eigenschaften

Nichtnegativ	$H(X) \geq 0$, mit Gleichheit genau bei einem sicheren Ereignis
Obere Schranke	$H(X) \leq \log_2 n$, mit Gleichheit genau bei Gleichverteilung
Symmetrie	Die Reihenfolge der p_i ist gleichgültig
Konkav	Mischt man zwei Verteilungen, so steigt die Entropie

Beweis der oberen Schranke.
Wir benutzen die Ungleichung $\ln t \leq t - 1$ für $t > 0$, mit Gleichheit genau bei $t = 1$.

$$H(X) - \log n = \sum_i p_i \log \frac{1}{p_i} - \sum_i p_i \log n = \sum_i p_i \log \frac{1}{n\, p_i}.$$

Mit $\log t = \ln t / \ln 2$ und der Ungleichung:

$$\sum_i p_i \log \frac{1}{n p_i} \leq \frac{1}{\ln 2} \sum_i p_i \left(\frac{1}{n p_i} - 1 \right) = \frac{1}{\ln 2} \left(\sum_i \frac{1}{n} - \sum_i p_i \right) = 0.$$

Also $H(X) \leq \log n$. Gleichheit tritt genau dann ein, wenn $\frac{1}{n p_i} = 1$ für alle i, das heißt $p_i = \frac{1}{n}$.

> *Das ist die formale Fassung einer vertrauten Erfahrung: Am meisten erfährt man, wenn man vorher am wenigsten wusste. Und es verbindet Shannons Maß mit Hartleys:* $\log n$ *ist genau Hartleys Wert.*

Entropie ist keine Eigenschaft der Daten

Man liest oft, eine Datei „habe“ eine bestimmte Entropie. Das ist bequem und streng genommen falsch.

Definiert für	eine *Quelle*, also eine Wahrscheinlichkeitsverteilung
Nicht definiert für	eine einzelne, feste Zeichenfolge

Wenn man von der Entropie einer Datei spricht, meint man in Wahrheit: die Entropie desjenigen Modells, das man aus dieser Datei geschätzt hat. Ein anderes Modell liefert einen anderen Wert für dieselben Bytes.

Modell für deutschen Text	H je Zeichen	**Datei von 1 MB**
Gleichverteilte Bytes	8,00 bit	1000 KB
Buchstabenhäufigkeit	4,10 bit	513 KB
Paare	3,50 bit	438 KB
Dreiergruppen	2,80 bit	350 KB
Guter Vorhersager	$\approx$ 1,30 bit	**163 KB**

Es gibt einen Begriff, der die Entropie einer einzelnen Zeichenfolge ohne Bezug auf ein Modell fasst. Er heißt Kolmogorow-Komplexität, und Kapitel 13 zeigt, warum man ihn nicht berechnen kann.

Aufgabe 2.1.

(a) Berechnen Sie Entropien für mehrere Verteilungen.
(b) Untersuchen Sie die binäre Entropiefunktion.
(c) Schätzen Sie die Entropie deutschen Textes.

Lösung Schritt für Schritt.

(a) Entropien im Vergleich.
Vier Quellen mit je vier Zeichen.

Quelle	Verteilung	H	Anteil
Q_1	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$	2,000	100 %
Q_2	$\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$	1,750	87,5 %
Q_3	0,7 ; 0,1 ; 0,1 ; 0,1	1,357	67,9 %
Q_4	0,97 ; 0,01 ; 0,01 ; 0,01	0,242	12,1 %

Die Rechnung für Q_2.

$$H = \tfrac{1}{2}\cdot 1 + \tfrac{1}{4}\cdot 2 + \tfrac{1}{8}\cdot 3 + \tfrac{1}{8}\cdot 3 = 0{,}5 + 0{,}5 + 0{,}375 + 0{,}375 = 1{,}750.$$

Bei Q_2 sind alle Wahrscheinlichkeiten Zweierpotenzen. Deshalb sind alle Einzelgehalte ganzzahlig, und es gibt einen Code, der die Entropie exakt erreicht. Kapitel 6 kommt darauf zurück.

Die Rechnung für Q_3.

$$\begin{aligned} H &= -0{,}7\log 0{,}7 - 3\cdot 0{,}1\log 0{,}1 \\ &= 0{,}7\cdot 0{,}5146 + 0{,}3\cdot 3{,}3219 = 0{,}3602 + 0{,}9966 = 1{,}357. \end{aligned}$$

Was die letzte Spalte bedeutet.
Der Anteil sagt, wie viel Platz eine ideale Verdichtung noch braucht, verglichen mit der naiven Speicherung mit zwei bit je Zeichen.

Quelle	naiv	ideal	Ersparnis
Q_1	2000 bit	2000 bit	0 %
Q_2	2000 bit	1750 bit	12,5 %
Q_3	2000 bit	1357 bit	32,1 %
Q_4	2000 bit	242 bit	**87,9** %

(b) Die binäre Entropiefunktion.

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p).$$

p	$H(p)$	p	$H(p)$
0,00	0,000	0,50	**1,000**
0,05	0,286	0,60	0,971
0,10	0,469	0,70	0,881
0,20	0,722	0,80	0,722
0,30	0,881	0,90	0,469
0,40	0,971	1,00	0,000

Das Maximum.
Ableiten und null setzen:

$$\frac{dH}{dp} = \log_2 \frac{1-p}{p} = 0 \quad \Longleftrightarrow \quad \frac{1-p}{p} = 1 \quad \Longleftrightarrow \quad p = \tfrac{1}{2}.$$

Das Verhalten an den Rändern.

$$\frac{dH}{dp}\Big|_{p \to 0^+} = +\infty.$$

> *Die Kurve läuft an beiden Rändern senkrecht ein. Das hat eine praktische Bedeutung: Eine Quelle, die zu 99,9 Prozent dasselbe Zeichen sendet, trägt noch* 0,0114 *bit, und eine Verzehnfachung der Seltenheit senkt diesen Wert nur um etwa Faktor acht, nicht um zehn.*

Die Symmetrie.

$$H(p) = H(1-p).$$

Es macht keinen Unterschied, welches der beiden Zeichen das seltene ist. Auch das ist einleuchtend: Man könnte die Namen vertauschen.

(c) Die Entropie deutschen Textes.
Stufe 0 – Gleichverteilung.
Mit 30 Zeichen (26 Buchstaben, Umlaute, Leerzeichen):

$$H_0 = \log_2 30 = 4{,}907 \text{ bit je Zeichen.}$$

Stufe 1 – Buchstabenhäufigkeiten.

Zeichen	Anteil	Zeichen	Anteil
Leerz.	0,151	`a`	0,054
`e`	0,147	`d`	0,043
`n`	0,088	`h`	0,039
`i`	0,064	`u`	0,035
`s`	0,062	`l`	0,030
`r`	0,059	`c`	0,028
`t`	0,055	Rest	0,145

$H_1 \approx 4{,}1$ bit je Zeichen.

Allein die ungleiche Häufigkeit senkt den Wert um 0,8 bit, also um etwa 16 Prozent.

Stufe 2 und höher – der Zusammenhang.

Modell	H	was es nutzt
H_0	4,91	nichts
H_1	4,10	Häufigkeit
H_2	$\approx 3{,}50$	Vorgängerzeichen
H_3	$\approx 2{,}80$	zwei Vorgänger
H_∞	$\approx 1{,}30$	**die ganze Sprache**

Wie Shannon das gemessen hat.
Er ließ 1951 Versuchspersonen Buchstaben raten. Man zeigt einen Text bis zu einer Stelle und lässt raten, wie es weitergeht, und zählt die Fehlversuche.

Versuche bis Treffer	Anteil
1 (sofort)	$\approx 79\,\%$
2	$\approx 8\,\%$
3	$\approx 3\,\%$
mehr	$\approx 10\,\%$

Aus dieser Verteilung lassen sich obere und untere Schranken für die Entropie ableiten. Shannon kam für Englisch auf einen Bereich zwischen 0,6 und 1,3 bit je Buchstabe. Der Versuch ist mit einfachsten Mitteln durchführbar und methodisch bemerkenswert elegant.

Die Gegenprobe von heute.

Verfahren	bit je Zeichen	Jahr
Shannons Rateversuch	0,6–1,3	1951
`gzip`	$\approx 2{,}6$	1991
`bzip2`	$\approx 2{,}1$	1996
Kontextmischung	$\approx 1{,}2$	2005
Große Sprachmodelle	$\approx 0{,}7$	2020er

Shannons Schätzung von 1951, mit Bleistift und Versuchspersonen gewonnen, liegt siebzig Jahre später noch im richtigen Bereich. Und die letzte Zeile deutet an, worauf Kapitel 15 hinausläuft: Ein guter Vorhersager und ein guter Verdichter sind dasselbe.

Shannons Arbeit[24] enthält die Herleitung im Anhang; sein Aufsatz über gedruckten Text[26] beschreibt den Rateversuch; Cover und Thomas[6] behandeln die Eigenschaften der Entropie vollständig.

3 Was zwei Größen voneinander wissen

Drei abgeleitete Größen

Sobald zwei Zufallsgrößen im Spiel sind, lassen sich mehrere Fragen stellen.

Größe	Zeichen	Frage
Verbundentropie	$H(X,Y)$	Wie viel steckt in beiden zusammen?
Bedingte Entropie	$H(Y\|X)$	Was bleibt an Y offen, wenn X bekannt ist?
Transinformation	$I(X;Y)$	Wie viel verrät X über Y?

Definition 3.1.

$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y),$$

$$H(Y|X) = -\sum_{x,y} p(x,y)\log p(y|x),$$

$$I(X;Y) = \sum_{x,y} p(x,y)\log \frac{p(x,y)}{p(x)\,p(y)}.$$

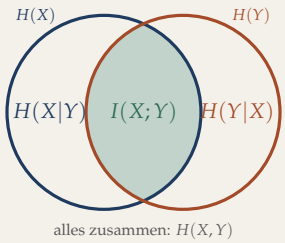
Die Kettenregel.

$$H(X,Y) = H(X) + H(Y|X).$$

In Worten: Um beides zu erfahren, erfährt man erst X und dann, was von Y noch übrig ist. Das ist genau die Zerlegbarkeitsforderung aus Kapitel 2, hier auf zwei Größen angewandt.

Die Transinformation aus drei Blickwinkeln.

$$\begin{aligned} I(X;Y) &= H(X) - H(X|Y) \\ &= H(Y) - H(Y|X) \\ &= H(X) + H(Y) - H(X,Y). \end{aligned}$$



Das Bild ist hilfreich und hat eine Tücke: Bei drei Größen kann der entsprechende Mittelteil negativ werden. Die Mengenanalogie trägt also nur so weit.

Vier Eigenschaften der Transinformation

Symmetrisch	$I(X;Y) = I(Y;X)$: X verrät über Y genau so viel wie umgekehrt
Nichtnegativ	$I(X;Y) \geq 0$, mit Gleichheit genau bei Unabhängigkeit
Obere Schranke	$I(X;Y) \leq \min\{H(X), H(Y)\}$
Datenverarbeitung	Nachbearbeitung kann die Transinformation nur senken

Die erste Eigenschaft ist überraschend.
Dass ein Rauchmelder etwas über ein Feuer verrät, leuchtet ein. Dass ein Feuer genau ebenso viel über den Rauchmelder verrät, wirkt zunächst seltsam und folgt unmittelbar aus der Formel, die in x und y symmetrisch ist.

Der Grund liegt darin, dass die Transinformation keine Ursache misst, sondern einen statistischen Zusammenhang. Sie sagt nichts darüber, was wovon abhängt.

Die Datenverarbeitungsungleichung.
Bildet $X \to Y \to Z$ eine Kette, in der Z nur über Y von X abhängt, so gilt

$$I(X;Z) \leq I(X;Y).$$

In Worten	Nachträgliches Rechnen schafft keine neue Information über die Quelle
Praktisch	Ein Filter, eine Glättung, eine Verdichtung kann nur verlieren
Nützlich	Es kann trotzdem sinnvoll sein: Was übrig bleibt, ist leichter zugänglich

Die dritte Zeile ist wichtig, damit der Satz nicht als Verbot missverstanden wird. Eine Bildverbesserung schafft keine Information hinzu und macht die vorhandene sichtbar. Beides ist wahr und widerspricht sich nicht.

Aufgabe 3.1.

(a) Rechnen Sie alle Größen für eine Verbundverteilung aus.
(b) Untersuchen Sie ein medizinisches Testbeispiel.
(c) Zeigen Sie, dass Bedingen die Entropie nicht erhöht.

Lösung Schritt für Schritt.

(a) Eine Verbundverteilung vollständig.
Die Tabelle. Wetter X und Regenschirm Y.

$p(x,y)$	$Y=$ ja	$Y=$ nein	$p(x)$
$X=$ Regen	0,25	0,05	0,30
$X=$ trocken	0,10	0,60	0,70
$p(y)$	0,35	0,65	1,00

Schritt 1 – Die Randentropien.

$$H(X) = H(0{,}3) = 0{,}8813 \text{ bit.}$$

$$H(Y) = H(0{,}35) = 0{,}9341 \text{ bit.}$$

Schritt 2 – Die Verbundentropie.

$$\begin{aligned} H(X,Y) &= -0{,}25\log 0{,}25 - 0{,}05\log 0{,}05 - 0{,}10\log 0{,}10 - 0{,}60\log 0{,}60 \\ &= 0{,}5000 + 0{,}2161 + 0{,}3322 + 0{,}4422 \\ &= 1{,}4905 \text{ bit.} \end{aligned}$$

Schritt 3 – Die Transinformation.

$$I(X;Y) = H(X) + H(Y) - H(X,Y) = 0{,}8813 + 0{,}9341 - 1{,}4905 = \mathbf{0{,}3249} \text{ bit.}$$

Schritt 4 – Die bedingten Entropien.

$$H(Y|X) = H(X,Y) - H(X) = 1{,}4905 - 0{,}8813 = 0{,}6092 \text{ bit.}$$

$$H(X|Y) = H(X,Y) - H(Y) = 1{,}4905 - 0{,}9341 = 0{,}5564 \text{ bit.}$$

Die Probe.

$$I(X;Y) = H(Y) - H(Y|X) = 0{,}9341 - 0{,}6092 = 0{,}3249. \qquad \checkmark$$

Die Deutung.

Größe	Wert	Bedeutung
$H(Y)$	0,9341	Unsicherheit über den Schirm
$H(Y\|X)$	0,6092	Rest, wenn das Wetter bekannt ist
$I(X;Y)$	0,3249	**was das Wetter verrät**
Anteil	34,8 %	der Unsicherheit beseitigt

Ein Drittel der Unsicherheit über den Schirm verschwindet, wenn man das Wetter kennt. Zwei Drittel bleiben, weil manche Leute bei Regen keinen mitnehmen und andere bei Sonne einen dabeihaben.

(b) Ein diagnostischer Test.
Die Angaben. Ein Test auf ein Merkmal, das bei einem Prozent der Untersuchten vorliegt.

Größe	Wert	
Prävalenz $p(K)$	0,01	
Sensitivität $p(+\|K)$	0,95	richtig positiv
Spezifität $p(-\|\bar{K})$	0,90	richtig negativ

Schritt 1 – Die Verbundverteilung.

	$T=+$	$T=-$	p
K	0,0095	0,0005	0,01
$\bar{K}$	0,0990	0,8910	0,99
$p(T)$	0,1085	0,8915	1,00

Schritt 2 – Die Entropien.

$$H(K) = H(0{,}01) = 0{,}0808 \text{ bit}.$$

$$H(T) = H(0{,}1085) = 0{,}4954 \text{ bit}.$$

$$\begin{aligned} H(K,T) &= -0{,}0095\log 0{,}0095 - 0{,}0005\log 0{,}0005 \\ &\quad - 0{,}0990\log 0{,}0990 - 0{,}8910\log 0{,}8910 \\ &= 0{,}0637 + 0{,}0055 + 0{,}3305 + 0{,}1483 = 0{,}5480 \text{ bit.} \end{aligned}$$

Schritt 3 – Die Transinformation.

$$I(K;T) = 0{,}0808 + 0{,}4954 - 0{,}5480 = \mathbf{0{,}0282} \text{ bit.}$$

Größe	**Wert**	
$H(K)$ vorher	0,0808 bit	
$I(K;T)$	0,0282 bit	
Anteil	**34,9** %	der Unsicherheit beseitigt
$H(K\|T)$	0,0526 bit	bleibt offen

Die Probe über den positiven Vorhersagewert.

$$p(K|+) = \frac{0{,}0095}{0{,}1085} = 0{,}0876.$$

Von hundert positiven Ergebnissen liegt das Merkmal in knapp neun Fällen wirklich vor. Der Test ist nicht schlecht; die Seltenheit des Merkmals sorgt dafür, dass die falsch Positiven die richtig Positiven weit übertreffen.

Prävalenz	$I(K;T)$	**Anteil**	$p(K\|+)$
0,001	0,0035	30,7 %	0,0094
0,01	0,0282	34,9 %	0,0876
0,10	0,1743	37,2 %	0,5135
0,50	0,4200	42,0 %	0,9048

Der Anteil der beseitigten Unsicherheit bleibt erstaunlich stabil, während der Vorhersagewert um zwei Größenordnungen schwankt. Die beiden Größen beantworten verschiedene Fragen, und beim Reihenscreening ist der Unterschied folgenreich.

(c) Bedingen erhöht die Entropie nicht.
Die Behauptung.

$$H(Y|X) \le H(Y), \quad \text{mit Gleichheit genau bei Unabhängigkeit.}$$

Der Beweis. Es genügt, $I(X;Y) \geq 0$ zu zeigen, denn $I(X;Y) = H(Y) - H(Y|X)$.

$$-I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x)p(y)}{p(x,y)}.$$

Mit $\log t \leq \frac{t-1}{\ln 2}$:

$$-I(X;Y) \leq \frac{1}{\ln 2} \sum_{x,y} p(x,y) \left(\frac{p(x)p(y)}{p(x,y)} - 1 \right) = \frac{1}{\ln 2} \left(\sum_{x,y} p(x)p(y) - 1 \right) = 0.$$

Also $I(X;Y) \geq 0$. Gleichheit gilt genau dann, wenn $p(x,y) = p(x)p(y)$ überall, also bei Unabhängigkeit.

Der Satz gilt im Mittel, nicht im Einzelfall

$$H(Y|X) \leq H(Y), \qquad \text{aber} \qquad H(Y|X = x) \leq H(Y) \text{ ist falsch.}$$

Ein Gegenbeispiel.

$p(x,y)$	$Y=0$	$Y=1$	$p(x)$
$X=0$	0,45	0,45	0,90
$X=1$	0,10	0,00	0,10
$p(y)$	0,55	0,45	1,00

$$H(Y) = H(0{,}45) = 0{,}9928.$$

$$H(Y|X{=}0) = H(0{,}5) = \mathbf{1{,}0000} > H(Y).$$

$$H(Y|X{=}1) = H(0) = 0.$$

$$H(Y|X) = 0{,}9 \cdot 1{,}0 + 0{,}1 \cdot 0 = 0{,}9000 \leq 0{,}9928. \qquad \checkmark$$

Eine einzelne Beobachtung kann die Unsicherheit also durchaus erhöhen. Im Mittel über alle Beobachtungen kann sie das nicht. Wer je nach einer Messung ratloser war als vorher, hat das erlebt.

Cover und Thomas[6] behandeln alle diese Größen mit vollständigen Beweisen; MacKay[18] verbindet sie mit der Statistik; Csiszár und Körner[7] sind die strengste Darstellung.

Part II

Verdichten

4 Codes

Was ein Code leisten muss

Ein Code ordnet jedem Quellzeichen ein Codewort aus Nullen und Einsen zu. Damit das brauchbar ist, muss man die Folge wieder zerlegen können.

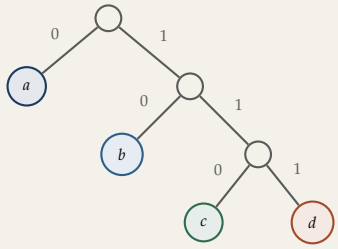
Eindeutig	Verschiedene Zeichen bekommen verschiedene Codewörter
Eindeutig zerlegbar	Jede Folge von Codewörtern lässt sich nur auf eine Weise auftrennen
Präfixfrei	Kein Codewort ist Anfang eines anderen

Die drei Forderungen werden strenger von oben nach unten. Die zweite ist das, was man braucht; die dritte ist stärker und dafür bequem.

Zeichen	**Code A**	**Code B**	**Code C**	**Code D**
a	`0`	`0`	`0`	`0`
b	`0`	`01`	`10`	`10`
c	`1`	`011`	`110`	`110`
d	`10`	`0111`	`1110`	`111`
	nicht eindeutig	zerlegbar	zerlegbar	präfixfrei

Code B ist eindeutig zerlegbar, aber unbequem: Um `0111` von `011 1` zu unterscheiden, muss man weiterlesen. Bei Code D weiß man sofort, wann ein Codewort zu Ende ist.

Präfixfrei heißt: ein Baum.
Man kann jeden präfixfreien Code als Binärbaum zeichnen, in dem die Codewörter an den Blättern hängen.



Präfixfreiheit bedeutet genau, dass die Codewörter an Blättern hängen und nicht an inneren Knoten. Das Decodieren wird damit zu einem Weg durch den Baum: bei jeder gelesenen Ziffer einen Schritt, bei jedem Blatt ein Zeichen ausgeben und zur Wurzel zurück.

Die Ungleichung von Kraft

Sie sagt genau, welche Codewortlängen möglich sind.

> **Satz (Kraft, 1949).** Zu Längen $\ell_1, \ldots, \ell_n$ gibt es genau dann einen präfixfreien Binärcode, wenn
>
> $$\sum_{i=1}^{n} 2^{-\ell_i} \leq 1.$$

Warum das gilt: die Anschauung.
Denke an einen vollständigen Baum der Tiefe $L = \max \ell_i$. Er hat 2^L Blätter. Ein Codewort der Länge ℓ sperrt alle Blätter unterhalb seines Knotens, das sind $2^{L-\ell}$ Stück.

$$\sum_i 2^{L-\ell_i} \leq 2^L \quad \Longleftrightarrow \quad \sum_i 2^{-\ell_i} \leq 1.$$

Jedes Codewort belegt also einen Anteil des Baumes, und die Anteile dürfen sich zu höchstens eins summieren. Kurze Codewörter sind teuer: Eines der Länge 1 verbraucht schon die Hälfte.

Länge	Anteil	davon passen
1	0,5	2
2	0,25	4
3	0,125	8
8	0,0039	256

Ein Beispiel und ein Gegenbeispiel.

Längen	$\sum 2^{-\ell_i}$	möglich?
$1, 2, 3, 3$	$0{,}5 + 0{,}25 + 0{,}125 + 0{,}125 = 1$	ja, genau voll
$2, 2, 2, 2$	$4 \cdot 0{,}25 = 1$	ja
$1, 2, 2, 3$	$0{,}5 + 0{,}25 + 0{,}25 + 0{,}125 = 1{,}125$	nein
$1, 3, 3, 3$	$0{,}5 + 3 \cdot 0{,}125 = 0{,}875$	ja, mit Rest

Die letzte Zeile ist möglich und verschwenderisch. Ein Rest bedeutet, dass ein Blatt ungenutzt bleibt und man ein Codewort hätte kürzen können.

Die Ungleichung gilt auch ohne Präfixfreiheit

BROCKWAY MCMILLAN zeigte 1956 etwas Stärkeres: Die Kraft-Ungleichung gilt für *jeden* eindeutig zerlegbaren Code, nicht nur für präfixfreie.

> *Daraus folgt eine wichtige Konsequenz: Zu jedem eindeutig zerlegbaren Code gibt es einen präfixfreien mit denselben Wortlängen. Man verliert also nichts, wenn man sich von vornherein auf präfixfreie Codes beschränkt.*

Deshalb behandelt der Rest dieses Bandes nur noch präfixfreie Codes. Sie sind bequemer, und sie sind ohne Einbuße.

Aufgabe 4.1.

(a) Prüfen Sie Codes auf eindeutige Zerlegbarkeit.
(b) Bestimmen Sie die kürzestmöglichen Codewortlängen.
(c) Untersuchen Sie Codes für unbekannt große Zahlen.

Lösung Schritt für Schritt.

(a) Codes prüfen.
Code 1. $\{\texttt{0}, \texttt{01}, \texttt{11}\}$.
Präfixfrei? Nein, `0` ist Anfang von `01`. Eindeutig zerlegbar?

Folge	Zerlegung 1	Zerlegung 2
`011`	`0` `11`	`01` ? Rest `1`

Da `1` allein kein Codewort ist, scheitert die zweite Zerlegung. Systematisch prüft man das mit dem Verfahren von Sardinas und Patterson.

1. Bilde alle Reste: Ist u Anfang von v, nimm den Rest von v auf.
2. Wiederhole mit den Resten gegen die Codewörter.
3. Erscheint ein Codewort als Rest, ist der Code **nicht** zerlegbar.
4. Wiederholt sich die Restmenge ohne Treffer, ist er zerlegbar.

Runde	Reste	Codewort dabei?
1	`1`	nein
2	`1`	nein, wiederholt sich

$\Rightarrow$ Code 1 ist eindeutig zerlegbar. ✓

Code 2. $\{$`0`, `01`, `10`$\}$.

Runde	Reste	Codewort dabei?
1	`1`	nein
2	`0`	**ja** Abbruch

Die Probe: `010` lässt sich als `0` `10` oder als `01` `0` lesen. Nicht zerlegbar.

(b) Kürzeste Codewortlängen.
Die Aufgabe. Fünf Zeichen mit $p = (0{,}4;\ 0{,}25;\ 0{,}15;\ 0{,}12;\ 0{,}08)$.
Die ideale Länge.

$$\ell_i^* = -\log_2 p_i.$$

Zeichen	p_i	$-\log_2 p_i$	$\lceil\cdot\rceil$
a	0,40	1,322	2
b	0,25	2,000	2
c	0,15	2,737	3
d	0,12	3,059	4
e	0,08	3,644	4

Die Entropie.

$$\begin{aligned} H &= 0{,}40 \cdot 1{,}322 + 0{,}25 \cdot 2{,}000 + 0{,}15 \cdot 2{,}737 + 0{,}12 \cdot 3{,}059 + 0{,}08 \cdot 3{,}644 \\ &= 0{,}5288 + 0{,}5000 + 0{,}4106 + 0{,}3671 + 0{,}2915 = \mathbf{2{,}098}\ \text{bit}. \end{aligned}$$

Der Code mit aufgerundeten Längen.

$$\sum 2^{-\ell_i} = 0{,}25 + 0{,}25 + 0{,}125 + 0{,}0625 + 0{,}0625 = 0{,}75 \le 1. \qquad \checkmark$$

$$\begin{aligned} \bar{\ell} &= 0{,}40 \cdot 2 + 0{,}25 \cdot 2 + 0{,}15 \cdot 3 + 0{,}12 \cdot 4 + 0{,}08 \cdot 4 \\ &= 0{,}8 + 0{,}5 + 0{,}45 + 0{,}48 + 0{,}32 = 2{,}55\ \text{bit}. \end{aligned}$$

Der Rest von 0,25 zeigt, dass wir verschwenden. Tatsächlich lassen sich die Längen auf $(1, 2, 3, 4, 4)$ verbessern, was $\bar{\ell} = 2{,}20$ ergibt. Kapitel 6 zeigt das Verfahren, das die besten Längen findet.

Verfahren	$\bar{\ell}$	Überschuss
Entropie	2,098	—
Aufgerundet	2,550	+0,45
Huffman	2,200	+0,10
Feste Länge	3,000	+0,90

(c) Codes für unbekannt große Zahlen.
Das Problem. Man will eine natürliche Zahl übertragen und weiß vorher nicht, wie groß sie ist. Eine feste Länge scheidet aus.
Der Einer-Code. Die Zahl n wird als n Einsen, gefolgt von einer Null, codiert.

n	**Code**	**Länge**	**gut bei**
1	`10`	2	
3	`1110`	4	
8	`111111110`	9	$p(n) = 2^{-n}$

Der Einer-Code ist optimal, wenn die Wahrscheinlichkeiten mit 2^{-n} fallen. Bei größeren Zahlen ist er unbrauchbar.

Der Elias-Gamma-Code. Erst die Länge in Einer-Darstellung, dann die Zahl binär ohne führende Eins.

n	**binär**	**Gamma-Code**	**Länge**
1	`1`	`1`	1
2	`10`	`010`	3
5	`101`	`00101`	5
9	`1001`	`0001001`	7
100	`1100100`	`0000001100100`	13

$$\text{Länge} = 2\lfloor \log_2 n \rfloor + 1.$$

n	**Einer**	**Gamma**	**Delta**
1	2	1	1
10	11	7	8
100	101	13	**12**
10^6	$10^6 + 1$	39	**28**

Der Delta-Code wendet den Gamma-Code auf die Längenangabe selbst an. Er ist bei kleinen Zahlen etwas schlechter und bei großen deutlich besser. Man kann das beliebig oft schachteln; der Gewinn wird jedes Mal kleiner.

Wo diese Codes gebraucht werden

Sie sehen nach einer Spielerei aus und stecken in fast jeder Suchmaschine.

Invertierte Listen	Zu jedem Wort die Nummern der Dokumente, in denen es steht
Differenzen	Man speichert nicht die Nummern, sondern die Abstände
Verteilung	Die Abstände sind meist klein und gelegentlich groß

Darstellung	**bit je Eintrag**	
32-bit-Zahlen	32	
Differenzen, 32 bit	32	
Differenzen, Gamma	≈ 9	
Differenzen, Golomb	≈ 6	angepasst

Faktor fünf bei einem Index, der viele Gigabyte umfasst. Band 10 kommt darauf zurück; hier zeigt sich schon, dass die Wahl des Codes an die Verteilung angepasst sein muss, und dass man diese Verteilung erst einmal kennen muss.

Cover und Thomas[6] beweisen Kraft und McMillan vollständig; Sayood[23] behandelt die Zahlencodes praktisch; MacKay[18] zeigt den Zusammenhang zur Entropie besonders klar.

5 Der Quellencodierungssatz

Die Aussage

Der Satz beantwortet die erste der beiden Fragen aus dem Vorwort: Wie kurz kann man eine Nachricht machen, ohne etwas zu verlieren?

> **Satz (Shannon, 1948).** Für jeden eindeutig zerlegbaren Code einer Quelle X gilt für die mittlere Codewortlänge
>
> $$\bar{\ell} \;\geq\; H(X).$$
>
> Umgekehrt gibt es zu jedem $\varepsilon > 0$ einen Code mit
>
> $$\bar{\ell} \;<\; H(X) + \varepsilon,$$
>
> wenn man hinreichend lange Blöcke gemeinsam codiert.

Die Entropie ist also nicht irgendein Maß, sondern genau die Grenze. Darunter geht es nicht, und beliebig nah heran geht es. Das ist der Grund, warum diese eine Größe das ganze Fach trägt.

Beweis der unteren Schranke.
Sei ein präfixfreier Code mit Längen ℓ_i gegeben. Wir betrachten

$$\bar{\ell} - H(X) = \sum_i p_i \ell_i + \sum_i p_i \log p_i = \sum_i p_i \log \frac{p_i}{2^{-\ell_i}}.$$

Setze $q_i = 2^{-\ell_i}/c$ mit $c = \sum_j 2^{-\ell_j} \leq 1$ nach Kraft. Dann ist q eine Wahrscheinlichkeitsverteilung, und

$$\bar{\ell} - H(X) = \sum_i p_i \log \frac{p_i}{q_i} - \log c = D(p \,\|\, q) - \log c.$$

Nun ist $D(p \,\|\, q) \geq 0$ und $-\log c \geq 0$ wegen $c \leq 1$. Also

$$\bar{\ell} - H(X) \;\geq\; 0. \qquad \square$$

Der Beweis sagt zugleich, wann Gleichheit gilt: wenn $c = 1$ und $p = q$, also wenn $\ell_i = -\log_2 p_i$ für alle i. Das geht nur, wenn alle Wahrscheinlichkeiten Zweierpotenzen sind.

Der Verlust kommt aus	$D(p \,\|\, q)$: die Codelängen passen nicht genau zu den Wahrscheinlichkeiten
Und aus	$-\log c$: der Baum ist nicht ausgeschöpft

Die obere Schranke und warum Blöcke helfen

Ein Code, der immer funktioniert.
Wähle $\ell_i = \lceil -\log_2 p_i \rceil$. Dann ist die Kraft-Ungleichung erfüllt:

$$\sum_i 2^{-\lceil -\log p_i \rceil} \le \sum_i 2^{\log p_i} = \sum_i p_i = 1. \qquad \checkmark$$

Und wegen $\lceil t \rceil < t + 1$ gilt

$$\bar{\ell} = \sum_i p_i \lceil -\log p_i \rceil < \sum_i p_i(-\log p_i + 1) = H(X) + 1.$$

$H(X) \le \bar{\ell} < H(X) + 1$	**immer erreichbar**

Das zusätzliche bit stammt allein vom Aufrunden. Bei einer Quelle mit $H = 0{,}1$ bit ist das ein Aufschlag von tausend Prozent, bei $H = 8$ bit einer von zwölf.

Der Ausweg: Blöcke.
Codiert man n Zeichen gemeinsam, so gilt für die Entropie der Blockquelle $H(X^n) = nH(X)$ bei unabhängigen Zeichen, und damit

$$nH(X) \le \bar{\ell}_n < nH(X) + 1.$$

Geteilt durch n:

$$H(X) \;\le\; \frac{\bar{\ell}_n}{n} \;<\; H(X) + \frac{1}{n}.$$

Das ist der ganze Kunstgriff. Das eine verschwendete bit verteilt sich auf n Zeichen und wird beliebig klein. Der Preis ist ein Codebuch mit $|X|^n$ Einträgen, das rasch unhandlich wird.

Blocklänge	Überschuss	Codebuch	Bewertung
1	$< 1{,}000$ bit	2	
4	$< 0{,}250$ bit	16	
8	$< 0{,}125$ bit	256	gut
16	$< 0{,}063$ bit	65 536	
32	$< 0{,}031$ bit	$4 \cdot 10^9$	unbrauchbar

Kapitel 6 zeigt ein Verfahren, das die Grenze ohne Codebuch erreicht, indem es gar keine ganzzahligen Codewortlängen mehr benutzt.

Typische Folgen

Der Beweis der oberen Schranke geht auch ganz anders, und dieser zweite Weg trägt später den Kanalcodierungssatz.

Definition 5.1. Eine Folge $x_1 \dots x_n$ heißt **typisch** zum Grad ε, wenn

$$\left| -\tfrac{1}{n} \log_2 p(x_1 \dots x_n) - H(X) \right| < \varepsilon.$$

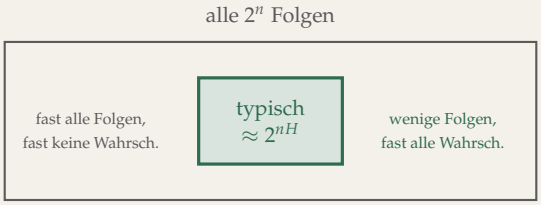
Fast alle Folgen sind typisch	Ihre Gesamtwahrscheinlichkeit geht gegen 1
Es gibt etwa 2^{nH} davon	Statt $\lvert X \rvert^n$ insgesamt
Alle etwa gleich wahrscheinlich	Jede hat etwa 2^{-nH}

Daraus folgt der Satz fast unmittelbar: Man nummeriert nur die typischen Folgen durch, wofür nH bit reichen, und gibt für die übrigen ein Ausweichcodewort aus. Da sie fast nie vorkommen, fällt ihr Aufwand nicht ins Gewicht.

Ein Beispiel. Eine Münze mit $p(1) = 0{,}2$, Folgen der Länge $n = 100$.

Größe	Wert	
Alle Folgen	$2^{100} \approx 10^{30}$	
Entropie	$H = 0{,}722$ bit	
Typische Folgen	$\approx 2^{72{,}2} \approx 5 \cdot 10^{21}$	
Anteil	$\approx 4 \cdot 10^{-9}$	winzig
Ihre Wahrscheinlichkeit	$\approx 0{,}99$	fast alles

Ein verschwindend kleiner Anteil der Folgen trägt fast die ganze Wahrscheinlichkeit. Das ist die Kernaussage, und sie ist die informationstheoretische Fassung des Gesetzes der großen Zahlen.



Aufgabe 5.1.

(a) Prüfen Sie die Schranke an einer konkreten Quelle nach.
(b) Rechnen Sie die typischen Folgen vollständig aus.
(c) Untersuchen Sie, was bei einem falschen Modell geschieht.

Lösung Schritt für Schritt.

(a) Die Schranke nachrechnen.
Die Quelle. Zwei Zeichen mit $p(a) = 0{,}9$, $p(b) = 0{,}1$.

$$H = H(0{,}1) = 0{,}469 \text{ bit}.$$

Einzelcodierung. Es gibt nur eine Möglichkeit: ein bit je Zeichen.

$$\bar{\ell} = 1{,}000, \qquad \text{Überschuss } 0{,}531 \text{ bit } (113\,\%).$$

Blöcke der Länge 2.

Block	p	Huffman	Länge
aa	0,81	`0`	1
ab	0,09	`11`	2
ba	0,09	`100`	3
bb	0,01	`101`	3

$$\begin{aligned}\bar{\ell}_2 &= 0{,}81\cdot 1 + 0{,}09\cdot 2 + 0{,}09\cdot 3 + 0{,}01\cdot 3\\ &= 0{,}81 + 0{,}18 + 0{,}27 + 0{,}03 = 1{,}29 \text{ bit je Block.}\end{aligned}$$

$$\frac{\bar{\ell}_2}{2} = 0{,}645 \text{ bit je Zeichen.}$$

Die Entwicklung.

Blocklänge	bit je Zeichen	Überschuss	in %
1	1,000	0,531	113
2	0,645	0,176	38
3	0,533	0,064	14
4	0,493	0,024	5
8	0,474	0,005	1
∞	0,469	0	0

Der Gewinn ist bei schiefen Verteilungen erheblich. Von einem bit auf 0,47 bit ist eine Halbierung, und sie entsteht allein daraus, dass man mehrere Zeichen zusammenfasst.

(b) Typische Folgen.
Die Quelle. $p(1) = 0{,}25$, Folgen der Länge $n = 20$.

$$H = H(0{,}25) = 0{,}811 \text{ bit.}$$

Die erwartete Zahl von Einsen.

$$\mathbb{E}[k] = 20\cdot 0{,}25 = 5.$$

Die Wahrscheinlichkeit einer Folge mit k Einsen.

$$p = 0{,}25^{k}\cdot 0{,}75^{20-k}.$$

$$-\tfrac{1}{n}\log_2 p = -\tfrac{1}{20}\big(k\log 0{,}25 + (20-k)\log 0{,}75\big) = \tfrac{1}{20}\big(2k + 0{,}415(20-k)\big).$$

k	$-\frac{1}{n}\log p$	$\lvert \cdot - H\rvert$	**typisch ($\varepsilon = 0{,}1$)**
2	0,574	0,237	nein
4	0,732	0,079	ja
5	0,811	0,000	ja
6	0,890	0,079	ja
8	1,049	0,238	nein

Die Zahl der typischen Folgen.

$$\binom{20}{4} + \binom{20}{5} + \binom{20}{6} = 4845 + 15\,504 + 38\,760 = 59\,109.$$

$\log_2 59\,109 = 15{,}85$ bit gegenüber $nH = 16{,}22$ bit. ✓

Ihre Gesamtwahrscheinlichkeit.

k	$\binom{20}{k}$	p je Folge	**Summe**
4	4845	$3{,}17 \cdot 10^{-5}$	0,1897
5	15 504	$1{,}06 \cdot 10^{-5}$	0,2023
6	38 760	$3{,}52 \cdot 10^{-6}$	0,1686
			0,561

Bei $n = 20$ ist die Menge noch nicht besonders typisch: 56 Prozent statt fast 100. Der Satz ist eine asymptotische Aussage, und man sieht hier, wie langsam sie greift. Bei $n = 1000$ läge der Anteil über 99 Prozent.

(c) Ein falsches Modell.
Die Lage. Die Quelle ist p, der Code wurde für q gebaut.
Der Mehraufwand.

$$\bar{\ell} = \sum_i p_i \lceil -\log q_i \rceil \approx H(p) + D(p \,\|\, q).$$

Ein Zahlenbeispiel. Vier Zeichen.

	p (wahr)	q (Modell)	$p \log(p/q)$
a	0,50	0,25	0,5000
b	0,25	0,25	0,0000
c	0,15	0,25	$-0{,}1106$
d	0,10	0,25	$-0{,}1322$
			$D = 0{,}2572$

$$
\begin{aligned}
H(p) &= 1{,}743 \text{ bit},\\
\bar{\ell} \text{ mit dem Modell } q &= 2{,}000 \text{ bit},\\
\text{Differenz} &= 0{,}257 = D(p \,\|\, q). \qquad \checkmark
\end{aligned}
$$

Die relative Entropie aus Kapitel 3 bekommt hier ihre handfeste Bedeutung: Sie ist genau der Preis in bit je Zeichen, den ein falsches Modell kostet.

Der Fall, in dem es ganz schiefgeht

Wenn das Modell einem Zeichen die Wahrscheinlichkeit null zuweist, das dann doch auftritt, ist $D = \infty$.

Der Code hat für dieses Zeichen kein Codewort. Es lässt sich nicht übertragen, und zwar unabhängig davon, wie viel Platz man aufwendet.

Die Regel	Ein Modell darf keinem möglichen Zeichen die Wahrscheinlichkeit null geben
Die Abhilfe	Man reserviert für jedes Zeichen einen kleinen Rest, meist über eine Glättung

Praktisch bewährt hat sich, jedem Zeichen einen Zähler von eins mitzugeben, bevor man beobachtet. Das Verfahren ist unter dem Namen Laplace-Glättung bekannt und stammt aus dem achtzehnten Jahrhundert.

Diese Vorsichtsmaßnahme ist der Grund, warum kein praktischer Verdichter die Entropie ganz erreicht: Er muss immer etwas Platz für das Unerwartete freihalten.

Shannon[24] beweist den Satz im Original; Cover und Thomas[6] führen ihn über typische Folgen; MacKay[18] macht das Bild der typischen Menge besonders anschaulich.

6 Verfahren zur Verdichtung

Der Huffman-Code

1951 stellte ROBERT FANO seinen Studenten am MIT die Aufgabe, den besten präfixfreien Code zu finden, wahlweise statt einer Abschlussprüfung. DAVID HUFFMAN, damals fünfundzwanzig, fand die Lösung.

1. Jedes Zeichen wird ein Knoten mit seiner Wahrscheinlichkeit als Gewicht.
2. Nimm die beiden Knoten mit dem **kleinsten** Gewicht.
3. Fasse sie zu einem neuen Knoten zusammen, Gewicht ist die Summe.
4. Wiederhole, bis ein Knoten übrig ist.
5. Beschrifte die Kanten mit 0 und 1; die Wege sind die Codewörter.

Der Einfall steckt in Schritt 2. Die beiden seltensten Zeichen bekommen die längsten Codewörter und unterscheiden sich nur in der letzten Ziffer. Das muss in jedem optimalen Code so sein, und daraus lässt sich der ganze Baum von unten aufbauen.

Warum es optimal ist.
Der Beweis läuft über zwei Beobachtungen, die man induktiv verbindet.

Erstens	In jedem optimalen Code haben die beiden seltensten Zeichen maximale und gleiche Länge und sind Geschwister
Zweitens	Fasst man sie zusammen, so ist ein optimaler Code für die kleinere Quelle auch für die größere optimal

Die erste Aussage zeigt man durch Vertauschen: Wäre ein häufigeres Zeichen länger codiert, so ließe sich durch Tausch verkürzen. Die zweite ist eine Rechnung, bei der sich die Differenz der mittleren Längen als die Summe der beiden Gewichte herausstellt.

Optimal heißt nicht: erreicht die Entropie

Huffman ist der beste Code mit ganzzahligen Codewortlängen. Das ist weniger, als es klingt.

Quelle	H	**Huffman**	**Verlust**
$(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$	1,500	1,500	0
$(0{,}4; 0{,}35; 0{,}25)$	1,559	1,600	0,041
$(0{,}9; 0{,}1)$	0,469	1,000	0,531

Die letzte Zeile ist der Ernstfall. Bei zwei Zeichen kann Huffman nur ein bit vergeben, auch wenn die Quelle fast nichts sagt. Der Verlust beträgt hier 113 Prozent.

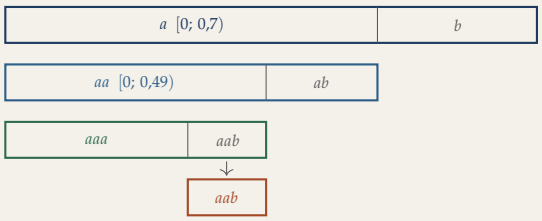
Das ist kein Fehler des Verfahrens, sondern eine Grenze der ganzen Bauart: Wer Codewörter aus ganzen bit zusammensetzt, kann die Länge 0,152 bit nicht darstellen.

Genau an dieser Stelle setzt die arithmetische Codierung an.

Arithmetische Codierung

Der Gedanke: Man codiert nicht Zeichen für Zeichen, sondern die ganze Nachricht als eine einzige Zahl im Einheitsintervall.

1. Beginne mit dem Intervall $[0, 1)$.
2. Teile es nach den Wahrscheinlichkeiten der Zeichen auf.
3. Wähle das Teilintervall des ersten Zeichens.
4. Wiederhole mit diesem Teilintervall für das nächste Zeichen.
5. Gib am Ende eine beliebige Zahl aus dem Endintervall aus, und zwar die mit der kürzesten Binärdarstellung.



Warum das die Entropie erreicht.
Nach n Zeichen hat das Intervall die Breite $p(x_1 \dots x_n)$. Um eine Zahl darin eindeutig anzugeben, braucht man etwa

$$-\log_2 p(x_1 \dots x_n) \text{ bit},$$

und im Mittel ist das genau $nH(X)$.

> *Jedes Zeichen trägt also seinen echten Informationsgehalt bei, auch wenn der 0,152 bit beträgt. Die Aufrundung geschieht nur einmal am Ende der ganzen Nachricht, nicht bei jedem Zeichen.*

	Huffman	**Arithmetisch**
Aufrundung	**je Zeichen**	einmal insgesamt
Verlust	< 1 bit je Zeichen	< 2 bit **insgesamt**
Modellwechsel	Baum neu bauen	Wahrscheinlichkeiten ändern
Geschwindigkeit	sehr hoch	mittel

Die dritte Zeile ist in der Praxis fast wichtiger als die zweite. Bei der arithmetischen Codierung darf sich das Modell nach jedem Zeichen ändern, ohne dass etwas neu aufgebaut werden muss. Damit lassen sich Modelle verwenden, die aus dem bisher Gelesenen lernen.

Die praktischen Schwierigkeiten

Die Beschreibung oben verlangt beliebig genaue Zahlen. Das geht nicht.

Skalierung	Sobald die obersten bit von Unter- und Obergrenze übereinstimmen, gibt man sie aus und verschiebt
Der Klemmfall	Wenn die Grenzen sich um $\frac{1}{2}$ herum annähern, ohne dass ein bit feststeht, merkt man sich einen Zähler
Ganzzahlrechnung	Alles läuft mit Ganzzahlen fester Breite, meist 32 bit

Die zweite Zeile ist der Grund, warum eine korrekte Umsetzung schwerer ist, als der Grundgedanke vermuten lässt. Sie war außerdem bis 2003 durch Patente belastet, was die Verbreitung erheblich verzögert hat.

Der Nachfolger: asymmetrische Zahlensysteme.
JAROSŁAW DUDA beschrieb ab 2009 ein Verfahren, das die Genauigkeit der arithmetischen Codierung mit der Geschwindigkeit von Huffman verbindet. Der Zustand ist eine einzige ganze Zahl, und das Codieren besteht aus wenigen Tabellenzugriffen.

Verfahren	MB/s	Verlust	Jahr
Huffman	≈ 400	merklich	1952
Arithmetisch	≈ 30	minimal	1976
ANS	≈ 350	minimal	2009

Duda veröffentlichte das Verfahren ausdrücklich patentfrei. Es steckt heute in Zstandard, in LZFSE und im Bildformat JPEG XL. Ein seltener Fall, in dem ein Verfahren in wenigen Jahren vom Aufsatz in den Massengebrauch gelangt.

Aufgabe 6.1.

(a) Bauen Sie einen Huffman-Code vollständig auf.
(b) Führen Sie eine arithmetische Codierung von Hand durch.
(c) Vergleichen Sie beide Verfahren an einer schiefen Quelle.

Lösung Schritt für Schritt.

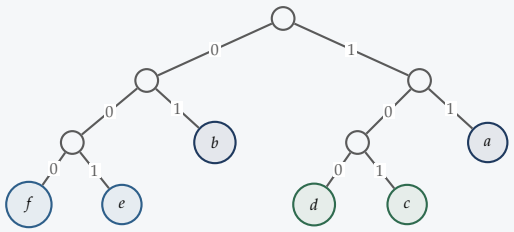
(a) Ein Huffman-Code.
Die Quelle. Sechs Zeichen.

a	0,30	c	0,15	e	0,10
b	0,25	d	0,12	f	0,08

Die Zusammenfassungen.

Schritt	kleinste zwei	neu	Restmenge
1	f 0,08, e 0,10	X 0,18	a, b, c, d, X
2	d 0,12, c 0,15	Y 0,27	a, b, X, Y
3	X 0,18, b 0,25	Z 0,43	a, Y, Z
4	Y 0,27, a 0,30	W 0,57	Z, W
5	Z 0,43, W 0,57	1,00	Wurzel

Der Baum.



Der Code.

Z.	p	Codewort	ℓ	$p \cdot \ell$
a	0,30	`11`	2	0,60
b	0,25	`01`	2	0,50
c	0,15	`101`	3	0,45
d	0,12	`100`	3	0,36
e	0,10	`001`	3	0,30
f	0,08	`000`	3	0,24
				$\bar{\ell} = 2{,}45$

Die Entropie.

$$\begin{aligned} H &= 0{,}30 \cdot 1{,}737 + 0{,}25 \cdot 2{,}000 + 0{,}15 \cdot 2{,}737 \\ &\quad + 0{,}12 \cdot 3{,}059 + 0{,}10 \cdot 3{,}322 + 0{,}08 \cdot 3{,}644 \\ &= 0{,}521 + 0{,}500 + 0{,}411 + 0{,}367 + 0{,}332 + 0{,}292 = \mathbf{2{,}423}\ \text{bit}. \end{aligned}$$

$$\text{Verlust} = 2{,}45 - 2{,}423 = 0{,}027\ \text{bit}\ (1{,}1\,\%).$$

Ein Prozent Verlust. Bei ausgewogenen Verteilungen ist Huffman fast optimal, und das ist der Grund, warum er trotz seiner bekannten Schwäche seit siebzig Jahren im Einsatz ist.

Die Kraft-Probe.

$$2 \cdot 2^{-2} + 4 \cdot 2^{-3} = 0{,}5 + 0{,}5 = 1. \qquad \checkmark$$

Der Baum ist voll ausgeschöpft, wie es bei einem Huffman-Code immer der Fall ist.

(b) Arithmetische Codierung von Hand.
Die Quelle. $p(a) = 0{,}7$, $p(b) = 0{,}2$, $p(c) = 0{,}1$. Zu codieren: `abc`.
Die Intervallaufteilung.

Z.	p	kumuliert	Intervall
a	0,7	0,0	$[0{,}0;\ 0{,}7)$
b	0,2	0,7	$[0{,}7;\ 0{,}9)$
c	0,1	0,9	$[0{,}9;\ 1{,}0)$

Schritt 1 – Zeichen a.

$$[0;\ 1) \longrightarrow [0;\ 0{,}7), \qquad \text{Breite } 0{,}7.$$

Schritt 2 – Zeichen b.

$$\begin{aligned}\text{unten} &= 0 + 0{,}7 \cdot 0{,}7 = 0{,}49,\\ \text{oben} &= 0 + 0{,}7 \cdot 0{,}9 = 0{,}63.\end{aligned}$$

$$[0{,}49;\ 0{,}63), \qquad \text{Breite } 0{,}14.$$

Schritt 3 – Zeichen c.

$$\begin{aligned}\text{unten} &= 0{,}49 + 0{,}14 \cdot 0{,}9 = 0{,}49 + 0{,}126 = 0{,}616,\\ \text{oben} &= 0{,}49 + 0{,}14 \cdot 1{,}0 = 0{,}630.\end{aligned}$$

$$[0{,}616;\ 0{,}630), \qquad \text{Breite } 0{,}014 = 0{,}7 \cdot 0{,}2 \cdot 0{,}1. \qquad \checkmark$$

Die Ausgabe. Gesucht ist die kürzeste Binärzahl im Intervall.

bit	Folge	Wert	drin?
1	`1`	0,5	nein
2	`10`	0,5	nein
3	`100`	0,5	nein
4	`1001`	0,5625	nein
5	`10011`	0,59375	nein
6	`100111`	0,609375	nein
7	`1001111`	0,6171875	**ja**

Die Probe.

$$-\log_2 0{,}014 = 6{,}16 \text{ bit}.$$

Wir brauchten sieben, also weniger als ein bit Überschuss für die ganze Nachricht.

> *Zum Vergleich: Ein Huffman-Code für diese Quelle vergibt die Längen 1, 2, 2 und bräuchte für `abc` fünf bit. Bei drei Zeichen ist Huffman also besser, weil die arithmetische Codierung ihren festen Aufschlag noch nicht amortisiert hat. Bei tausend Zeichen kehrt sich das um.*

(c) Eine schiefe Quelle.
Die Quelle. $p(a) = 0{,}95$, $p(b) = 0{,}05$.

$$H = H(0{,}05) = 0{,}286 \text{ bit}.$$

Verfahren	bit je Zeichen	Verhältnis zu H
Huffman, einzeln	1,000	3,50
Huffman, Blöcke zu 2	0,597	2,09
Huffman, Blöcke zu 4	0,396	1,38
Huffman, Blöcke zu 8	0,316	1,10
Arithmetisch	**0,286**	**1,00**

Um mit Huffman auf zehn Prozent an die Grenze zu kommen, braucht man Blöcke von acht Zeichen und ein Codebuch mit 256 Einträgen. Die arithmetische Codierung erreicht die Grenze ohne Codebuch.

Wo dieser Fall auftritt.

Faxbilder	Weiß ist weit häufiger als Schwarz
Vorhersagefehler	Bei Bild- und Tonverdichtung sind kleine Fehler weit häufiger als große
Sprachmodelle	Das nächste Wort ist oft mit über 90 Prozent vorhersagbar

Warum Huffman trotzdem überlebt

Die Tabelle spricht deutlich gegen ihn, und er steckt trotzdem in `gzip`, in JPEG und in vielen Netzwerkprotokollen.

Geschwindigkeit	Decodieren ist ein Tabellenzugriff je Codewort
Einfachheit	Wenige Dutzend Zeilen, leicht zu prüfen
Einstiegspunkte	Man kann mitten in einem Datenstrom aufsetzen
Vorgeschaltet	Nach LZ77 sind die Verteilungen nicht mehr so schief

Die letzte Zeile ist der praktische Punkt. In `gzip` arbeitet Huffman nicht auf dem Rohtext, sondern auf dem Ergebnis der Wörterbuchstufe, und dort sind die Wahrscheinlichkeiten ausgewogener. Der Verlust fällt dann kaum ins Gewicht.

Seit ANS verfügbar ist, verschiebt sich das Bild. Zstandard ersetzt Huffman durch ANS und erreicht damit bessere Verdichtung bei vergleichbarer Geschwindigkeit.

Huffmans Aufsatz[13] ist drei Seiten lang; Sayood[23] behandelt beide Verfahren mit vollständigen Beispielen; Duda[8] beschreibt die asymmetrischen Zahlensysteme.

7 Wörterbücher und Modelle

Das Problem mit den Wahrscheinlichkeiten

Alle Verfahren aus Kapitel 6 setzen voraus, dass man die Verteilung kennt. Woher?

Weg	Vorgehen	Nachteil
Fest	Tabelle mitliefern	passt nur zu einer Textart
Zwei Durchgänge	erst zählen, dann codieren	Tabelle mitschicken
Anpassend	beim Lesen lernen	keiner, aber Anlaufzeit
Wörterbuch	Wiederholungen als Verweis	lernt implizit

Die dritte Zeile ist der bequemste Weg: Codierer und Decodierer bauen dasselbe Modell auf, jeder aus dem, was er bisher gesehen hat. Es muss nichts übertragen werden, weil beide dieselben Schritte gehen.

Ziv und Lempel

1977 veröffentlichten JACOB ZIV und ABRAHAM LEMPEL ein Verfahren, das ohne jede Kenntnis der Quelle auskommt und trotzdem asymptotisch die Entropie erreicht.

Der Gedanke. Was schon einmal dastand, wird nicht wiederholt, sondern als Verweis geschrieben.

Ausgabe	Bedeutung
Zeichen	neu, wörtlich übernehmen
(Abstand, Länge)	**kopiere von weiter vorn**

Ein Beispiel. Der Text `abcabcabcx`.

Position	Ausgabe	erzeugt	Kosten
0	`a`	`a`	8 bit
1	`b`	`b`	8 bit
2	`c`	`c`	8 bit
3	$(3; 6)$	`abcabc`	$\approx$ 16 bit
9	`x`	`x`	8 bit

Bemerkenswert ist die vierte Zeile: Die Länge 6 überschreitet den Abstand 3. Das ist erlaubt und sogar erwünscht, weil beim Kopieren Zeichen für Zeichen gearbeitet wird und die frisch geschriebenen sofort wieder als Quelle dienen. So lassen sich Wiederholungen beliebiger Länge mit einem einzigen Verweis ausdrücken.

Warum es die Entropie erreicht.
Ziv und Lempel bewiesen, dass die Ausgabelänge je Zeichen für jede stationäre ergodische Quelle gegen deren Entropie strebt, ohne dass das Verfahren etwas über die Quelle wüsste.

Universell	Dasselbe Verfahren ist für alle Quellen asymptotisch optimal
Ohne Vorwissen	Es braucht keine Statistik und keine Trainingsphase
Langsam	Die Annäherung geht mit $\mathcal{O}(1/\log n)$, also zäh

Die dritte Zeile ist der Haken. Bei kurzen Dateien ist LZ deutlich schlechter als ein Verfahren mit gutem Modell. Bei einer Datei von hundert Byte lernt es fast nichts.

Dateigröße	`gzip`	Rohgröße	Verhältnis
100 B	122 B	100 B	1,22
10 KB	3,4 KB	10 KB	0,34
1 MB	290 KB	1 MB	0,29
100 MB	27 MB	100 MB	0,27

Die erste Zeile ist kein Fehler. Bei sehr kleinen Dateien ist das Ergebnis größer als die Eingabe, weil Kopfdaten und Prüfsumme hinzukommen und noch nichts gelernt wurde.

Die LZ-Familie

Name	Jahr	Verweise auf	steckt in
LZ77	1977	ein Fenster im Text	`deflate`, `zip`
LZ78	1978	ein Wörterbuch	
LZW	1984	dasselbe, ohne Zeichen	GIF, `compress`, TIFF
LZSS	1982	Fenster, mit Kennbit	viele Spiele
LZMA	1998	Fenster + Kontextmodell	`7z`, `xz`
LZ4	2011	Fenster, sehr schnell	Dateisysteme, Kern
Zstandard	2015	Fenster + ANS	fast überall
Brotli	2015	Fenster + festes Wörterbuch	Webverkehr

Der Aufbau von `deflate`.
Das verbreitetste Verfahren überhaupt kombiniert zwei Stufen.



Die erste Stufe entfernt Wiederholungen, die zweite die ungleichen Häufigkeiten. Beide zusammen decken die beiden wichtigsten Arten von Redundanz ab, und das Ergebnis ist seit 1991 der Standard für fast alles.

Stufe	aus 1 MB	Anteil
Roh	1000 KB	100 %
nach LZ77	420 KB	42 %
nach Huffman	290 KB	29 %

Vorhersagen statt nachschlagen

Der andere Weg besteht darin, aus dem Zusammenhang das nächste Zeichen vorherzusagen und nur die Überraschung zu codieren.

PPM	Schätzt die Verteilung des nächsten Zeichens aus den letzten k Zeichen; fällt auf kürzere Zusammenhänge zurück, wenn nötig
CM	Mehrere Modelle sagen gleichzeitig voraus; ihre Aussagen werden gewichtet gemischt
Neuronal	Ein gelerntes Netz liefert die Verteilung

Alle drei liefern eine Wahrscheinlichkeitsverteilung, die dann arithmetisch codiert wird. Die Verdichtung ist damit vollständig auf die Frage zurückgeführt, wie gut man vorhersagen kann.

Verfahren	bit/Zeichen	MB/s	Speicher
`gzip`	2,45	60	1 MB
`bzip2`	2,10	12	8 MB
`xz`	1,85	3	700 MB
`zstd -19`	1,90	6	300 MB
PPM	1,75	2	500 MB
CM (`cmix`)	**1,05**	0,0002	30 GB
Sprachmodell	$\approx 0{,}9$	sehr klein	sehr groß

Die untersten Zeilen sind praktisch unbenutzbar und theoretisch interessant. Sie zeigen, wie weit die Entropie deutscher und englischer Texte tatsächlich hinuntergeht, wenn man beliebig viel Aufwand treibt.

Aufgabe 7.1.

(a) Führen Sie LZ77 und LZW von Hand durch.
(b) Untersuchen Sie, warum manche Dateien nicht kleiner werden.
(c) Zeigen Sie, dass kein Verfahren alle Dateien verkleinern kann.

Lösung Schritt für Schritt.

(a) Die Verfahren von Hand.

LZ77 auf `abracadabra`.

Pos.	Vorher	Ausgabe	erzeugt
0	—	`a`	`a`
1	`a`	`b`	`b`
2	`ab`	`r`	`r`
3	`abr`	$(3;1)$	`a`
4	`abra`	`c`	`c`
5	`abrac`	$(2;1)$	`a`
6	`abraca`	`d`	`d`
7	`abracad`	$(7;4)$	`abra`

Größe	Wert	
Eingabe	11 Zeichen	88 bit
Ausgabe	5 Zeichen + 3 Paare	$\approx$ 88 bit

Kein Gewinn. Elf Zeichen sind für ein Wörterbuchverfahren viel zu wenig. Erst ab einigen Kilobyte trägt es.

LZW auf `abababababa`.

Das Wörterbuch enthält anfangs alle Einzelzeichen; hier $\{\texttt{a}\to 0, \texttt{b}\to 1\}$.

gelesen	Ausgabe	neuer Eintrag	Nummer
`a`	—	—	—
`ab`	0	`ab`	2
`ba`	1	`ba`	3
`aba`	2	`aba`	4
`abab`	—	—	—
`ababa`	4	`abab`	5
Ende	0		

Ausgabe: $0, 1, 2, 4, 0$ — fünf Nummern für neun Zeichen.

> *LZW baut sein Wörterbuch mit jedem Schritt um genau einen Eintrag aus. Der Decodierer tut dasselbe und braucht das Wörterbuch deshalb nicht mitgeschickt zu bekommen. Es gibt einen Sonderfall, in dem der Decodierer einen Eintrag benutzen muss, den er gerade erst anlegt; er lässt sich aus dem vorigen Eintrag rekonstruieren.*

(b) Dateien, die nicht kleiner werden.

Dateiart	`gzip`-Faktor	**Grund**
Textdatei	0,29	viel Redundanz
Quelltext	0,22	sehr viel
Rohbild (BMP)	0,55	Nachbarpunkte ähneln sich
JPEG	0,99	schon verdichtet
MP3	0,99	ebenso
Verschlüsselt	1,00	**sieht aus wie Zufall**
Zufallszahlen	1,00	ist Zufall

> *Die vorletzte Zeile ist ein brauchbarer Prüfstein für Verschlüsselung. Wenn sich ein Geheimtext verdichten lässt, enthält er noch Struktur, und das ist ein schlechtes Zeichen. Band 8 kommt darauf zurück.*

Warum Verdichten zweimal nichts bringt.
Nach der ersten Verdichtung ist die Entropie je Symbol nahe am Maximum. Ein zweiter Durchgang findet keine Struktur mehr und fügt nur Kopfdaten hinzu.

Durchgang	**Größe**	**Änderung**	
Original	1000 KB	—	
1× `gzip`	290 KB	−71 %	
2×	290 KB	+20 B	größer
3×	290 KB	+20 B	

(c) Kein Verfahren verkleinert alles.
Die Behauptung. Es gibt kein verlustfreies Verfahren, das jede Datei verkleinert.

Der Beweis durch Abzählen.
Betrachte alle Dateien mit genau n bit. Davon gibt es 2^n.

1.	Ein verlustfreies Verfahren muss verschiedene Eingaben auf verschiedene Ausgaben abbilden, sonst könnte man nicht zurück.
2.	Sollen alle 2^n Eingaben kürzer werden, müssen sie auf Dateien mit höchstens $n-1$ bit abgebildet werden.
3.	Davon gibt es $2^0 + 2^1 + \cdots + 2^{n-1} = 2^n - 1$ Stück.
4.	2^n Dinge lassen sich nicht injektiv auf $2^n - 1$ abbilden.

$\Rightarrow$ Mindestens eine Datei wird nicht kürzer. $\square$

Das ist das Schubfachprinzip, und es ist der ganze Beweis. Er braucht keine Informationstheorie, nur Abzählen.

Die schärfere Fassung.
Man kann mehr sagen: Fast alle Dateien werden nicht kürzer.

Verkürzung um	**Anteil, der das kann**
mindestens 1 bit	$< 50\,\%$
mindestens 10 bit	$< 0{,}1\,\%$
mindestens 20 bit	$< 0{,}0001\,\%$
mindestens 100 bit	$< 10^{-30}$

Verdichtung funktioniert also nur deshalb, weil die Dateien, mit denen wir tatsächlich zu tun haben, ein verschwindend kleiner und sehr besonderer Teil aller möglichen Dateien sind. Texte, Bilder und Programme sind hochgradig strukturiert.

Das immer wiederkehrende Versprechen

Etwa alle fünf Jahre kündigt jemand ein Verfahren an, das beliebige Daten um einen festen Faktor verkleinert, gern mit dem Zusatz, es funktioniere auch mehrfach angewandt.

> *Wendet man ein solches Verfahren wiederholt an, so schrumpft jede Datei auf ein einziges bit. Aus diesem bit müsste sich dann jede beliebige Datei zurückgewinnen lassen. Das genügt als Widerlegung, ohne dass man das Verfahren kennen muss.*

Meist steckt dahinter	Ein Teil der Daten wird anderswo abgelegt, etwa im Dateinamen oder in Nebendaten
Oder	Es funktioniert nur für eine bestimmte Datenart, was in Ordnung und nichts Neues ist
Oder	Es ist nicht verlustfrei

Der Abzählbeweis ist eines der wenigen Ergebnisse dieses Bandes, das man einem Skeptiker in zwei Minuten vollständig vorführen kann. Er lohnt sich, weil er zeigt, dass Unmöglichkeit manchmal einfacher zu beweisen ist als Möglichkeit.

Ziv und Lempel[28] führten das universelle Verfahren ein; Sayood[23] behandelt die ganze Familie mit Beispielen; MacKay[18] bringt den Abzählbeweis in seiner schärfsten Form.

8 Wenn man etwas verlieren darf

Die andere Frage

Bisher galt: Die Nachricht muss exakt wiederherstellbar sein. Die Entropie war die Grenze. Lässt man Abweichungen zu, verschiebt sich alles.

	verlustfrei	**verlustbehaftet**
Bedingung	Bit für Bit gleich	**ähnlich genug**
Grenze	Entropie H	**Rate-Distortion-Funktion**
Erreichbar	2–5-fach	10–100-fach
Umkehrbar	ja	nein

Der Gewinn ist erheblich, und der Preis ist endgültig. Was einmal weggeworfen wurde, kommt nicht zurück, auch nicht durch spätere Bearbeitung. Die Datenverarbeitungsungleichung aus Kapitel 3 sagt es genau.

Man braucht ein Maß für den Fehler.

Quadratischer Fehler	$d(x, \hat{x}) = (x - \hat{x})^2$; rechnerisch bequem, wahrnehmungsfern
Hamming-Abstand	Zahl der abweichenden Stellen; für diskrete Quellen
Wahrnehmungsmaße	Was ein Mensch tatsächlich bemerkt; schwer zu fassen

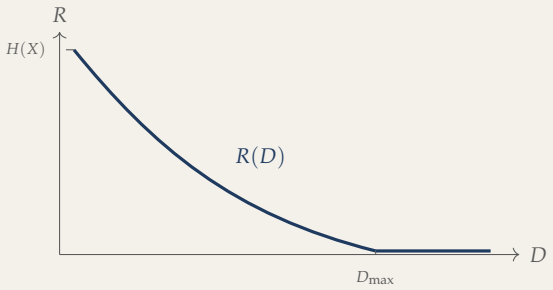
Die dritte Zeile ist der eigentliche Maßstab und der unhandlichste. Zwei Bilder mit demselben quadratischen Fehler können völlig verschieden aussehen, und ein Verfahren, das den quadratischen Fehler minimiert, sieht oft schlechter aus als eines, das ihn nicht minimiert.

Definition 8.1. Die **Rate-Distortion-Funktion** $R(D)$ gibt die kleinste Rate in bit je Zeichen an, mit der sich eine Quelle so darstellen lässt, dass die mittlere Verzerrung höchstens D beträgt:

$$R(D) = \min_{p(\hat{x}|x)\,:\,\mathbb{E}[d(X,\hat{X})]\le D} I(X;\hat{X}).$$

Bemerkenswert daran ist, dass wieder die Transinformation auftaucht. Die kleinste nötige Rate ist genau die Information, die die Näherung über das Original noch enthalten muss.

Der Verlauf von $R(D)$



$R(0) = H(X)$	Ohne Verzerrung ist man wieder beim Quellencodierungssatz
Fallend	Mehr erlaubter Fehler heißt weniger nötige bit
Konvex	Die ersten zugelassenen Fehler bringen den größten Gewinn
$R(D_{\max}) = 0$	Ab einer gewissen Verzerrung genügt es, gar nichts zu senden

Die dritte Zeile hat eine praktische Bedeutung, die jeder kennt: Von unkomprimiert auf leichte Kompression verliert man kaum etwas Sichtbares und spart sehr viel Platz. Von mittel auf stark spart man wenig und verliert viel.

Zwei Fälle mit geschlossener Formel.

Binäre Quelle	Bei $p = \frac{1}{2}$ und Hamming-Abstand: $R(D) = 1 - H(D)$ für $D \leq \frac{1}{2}$
Gaußsche Quelle	Bei Varianz σ^2 und quadratischem Fehler: $R(D) = \frac{1}{2} \log_2 \frac{\sigma^2}{D}$ für $D \leq \sigma^2$

D/σ^2	$R(D)$	**Faktor**	
1,00	0,00 bit	—	alles weg
0,25	1,00 bit	16,0	
0,0625	2,00 bit	8,0	
0,0039	4,00 bit	4,0	
$1{,}5 \cdot 10^{-5}$	8,00 bit	2,0	

Jedes zusätzliche bit je Abtastwert viertelt den Fehler. Das ist die informationstheoretische Fassung der bekannten Faustregel, dass ein bit etwa sechs Dezibel bringt.

Wie Verfahren das ausnutzen

Kein praktisches Verfahren rechnet $R(D)$ aus. Alle beruhen auf derselben Abfolge.

1. **Transformieren**: Die Daten so umrechnen, dass wenige Werte fast alles tragen.
2. **Quantisieren**: Diese Werte grob darstellen, die übrigen sehr grob oder gar nicht.
3. **Verlustfrei verdichten**: Das Ergebnis mit den Mitteln aus Teil II packen.

Nur Schritt 2 wirft etwas weg. Die Schritte 1 und 3 sind umkehrbar. Das ist eine saubere Trennung, die es erlaubt, den Verlust an genau einer Stelle zu steuern.

Format	Transformation	Quantisierung	Entropie
JPEG	DCT, 8×8	nach Tabelle	Huffman
JPEG 2000	Wavelets	bitebenenweise	arithmetisch
MP3	Filterbank	**nach Gehörmodell**	Huffman
AAC	MDCT	Gehörmodell	arithmetisch
H.265	DCT + Vorhersage	nach Tabelle	arithmetisch

Warum die Transformation so viel bringt.
Benachbarte Bildpunkte sind ähnlich. Nach einer Kosinustransformation stecken fast alle Energie in wenigen Koeffizienten.

Darstellung	Werte $\neq 0$	von 64
Bildpunkte	64	100 %
Nach DCT	64	100 %
Nach DCT und Quantisierung	≈ 6	**9** %

Die Transformation allein spart nichts. Sie sortiert die Information so um, dass sich das Wegwerfen lohnt: Die kleinen Koeffizienten stehen für feine Details, die kaum jemand vermisst.

Warum mehrfaches Umwandeln schadet

Jeder Durchgang durch ein verlustbehaftetes Verfahren wirft erneut etwas weg, und zwar nicht dieselben Anteile wie beim ersten Mal.

Durchgänge	Verlust	Eindruck
1	Bezugsgröße	gut
5	merklich mehr	Ränder werden unruhig
20	stark	sichtbare Blöcke

Ein Bild, das mehrfach geöffnet, verändert und wieder gespeichert wird, verliert bei jedem Mal. Deshalb bewahrt man Ausgangsmaterial verlustfrei auf und wandelt erst am Ende um.

Bei gleichbleibenden Einstellungen und ohne Bearbeitung stabilisiert sich JPEG allerdings nach wenigen Durchgängen, weil die Quantisierung dann fast nichts mehr ändert. Der Verfall entsteht durch das Verschieben, Zuschneiden und Nachbearbeiten dazwischen.

Aufgabe 8.1.

(a) Berechnen Sie die Rate-Distortion-Funktion einer binären Quelle.
(b) Verfolgen Sie einen JPEG-Block vollständig.
(c) Untersuchen Sie, was ein Gehörmodell leistet.

Lösung Schritt für Schritt.

(a) Die binäre Quelle.
Die Behauptung. Für eine Quelle mit $p = \frac{1}{2}$ und Hamming-Verzerrung gilt

$$R(D) = 1 - H(D) \quad \text{für } 0 \le D \le \tfrac{1}{2}, \qquad R(D) = 0 \text{ sonst.}$$

Die untere Schranke.

$$\begin{aligned} I(X;\hat{X}) &= H(X) - H(X|\hat{X}) \\ &= 1 - H(X \oplus \hat{X} \mid \hat{X}) \\ &\ge 1 - H(X \oplus \hat{X}) \\ &\ge 1 - H(D). \end{aligned}$$

Zeile 2	X ist aus $\hat{X}$ und $X \oplus \hat{X}$ eindeutig bestimmt
Zeile 3	Bedingen erhöht die Entropie nicht (Kap. 3)
Zeile 4	$\mathbb{P}(X \neq \hat{X}) \leq D \leq \frac{1}{2}$ und H ist dort wachsend

Die Erreichbarkeit. Man wählt einen Kanal, der mit Wahrscheinlichkeit D umkippt. Dann ist die Verzerrung genau D und die Transinformation genau $1 - H(D)$.
Zahlen.

D	$H(D)$	$R(D)$	**Ersparnis**
0,00	0,000	1,000	0 %
0,01	0,081	0,919	8 %
0,05	0,286	0,714	29 %
0,11	0,500	0,500	50 %
0,25	0,811	0,189	81 %
0,50	1,000	0,000	100 %

Um die Hälfte der bit zu sparen, muss man elf Prozent Fehler zulassen. Um neunzig Prozent zu sparen, schon etwa dreißig Prozent. Die Kurve wird schnell ungünstig, und das ist der Grund, warum verlustbehaftete Verfahren bei zufälligen Daten nichts bringen.

(b) Ein JPEG-Block.
Der Ausgangsblock. 8×8 Helligkeitswerte, hier ein sanfter Verlauf.

139	144	149	153	155	155	155	155
144	151	153	156	159	156	156	156
150	155	160	163	158	156	156	156
159	161	162	160	160	159	159	159
159	160	161	162	162	155	155	155
161	161	161	161	160	157	157	157
162	162	161	163	162	157	157	157
162	162	161	161	163	158	158	158

Schritt 1 – Verschieben. Von jedem Wert 128 abziehen, damit der Bereich symmetrisch wird.
Schritt 2 – Kosinustransformation. Die wichtigsten Koeffizienten:

235,6	−1,0	−12,1	−5,2	2,1	⋯
−22,6	−17,5	−6,2	−3,2	−2,9	⋯
−10,9	−9,3	−1,6	1,5	0,2	⋯
−7,1	−1,9	0,2	1,5	0,9	⋯
−0,6	−0,8	1,5	1,6	−0,1	⋯

Der Wert links oben, der Gleichanteil, ist mit Abstand der größte. Nach rechts und unten werden die Werte schnell klein. Das ist die Umsortierung, von der oben die Rede war.

Schritt 3 – Quantisieren. Jeder Koeffizient wird durch einen Tabellenwert geteilt und gerundet.

Position	**Wert**	**Teiler**	**Ergebnis**
$(0, 0)$	235,6	16	**15**
$(0, 1)$	−1,0	11	0
$(1, 0)$	−22,6	12	**−2**
$(1, 1)$	−17,5	12	**−1**
$(2, 0)$	−10,9	14	−1
$(4, 4)$	−0,1	68	0

Die Teiler wachsen nach rechts unten. Feine Details werden also stärker gerundet als grobe, und das ist die ganze Idee: Das Auge bemerkt grobe Helligkeitsunterschiede und feine Muster sehr ungleich.

Schritt 4 – Zickzack und Codierung.

Zickzack	Die Koeffizienten werden diagonal abgelesen, damit die Nullen ans Ende rücken
Lauflängen	Nullfolgen werden als Anzahl codiert
Huffman	Der Rest verlustfrei nach Kapitel 6

Die Bilanz.

Stufe	bit	
Roh	512	64×8
Nach Quantisierung	≈ 90	6 Werte $\neq 0$
Nach Huffman	≈ 40	
Faktor	**12,8**	

Nur Schritt 3 hat etwas weggeworfen. Die Schritte 2 und 4 sind umkehrbar, und Schritt 2 allein hätte nichts gebracht.

(c) Das Gehörmodell.
Der Gedanke. Statt den mathematischen Fehler klein zu halten, hält man den *hörbaren* Fehler klein.

Ruhehörschwelle	Was zu leise ist, wird gar nicht übertragen
Verdeckung in der Frequenz	Ein lauter Ton macht benachbarte Frequenzen unhörbar
Verdeckung in der Zeit	Nach einem lauten Ereignis ist das Gehör kurz unempfindlich, und sogar davor

Die dritte Zeile enthält eine Merkwürdigkeit: Die Verdeckung wirkt auch rückwärts, etwa fünf Millisekunden vor dem lauten Ereignis. Das liegt daran, dass laute Reize im Nervensystem schneller verarbeitet werden als leise und diese gewissermaßen überholen.

Wie das Verfahren es nutzt.

1. Das Signal wird in Frequenzbänder zerlegt.
2. Für jedes Band wird berechnet, wie viel Rauschen dort unhörbar bliebe.
3. Die Quantisierung wird so gewählt, dass ihr Fehler gerade unter dieser Schwelle liegt.
4. Bänder, die vollständig verdeckt sind, entfallen.

Verfahren	Rate	Beurteilung
CD, unkomprimiert	1411 kbit/s	Bezugsgröße
FLAC (verlustfrei)	$\approx$ 800 kbit/s	identisch
MP3	192 kbit/s	meist ununterscheidbar
AAC	128 kbit/s	meist ununterscheidbar
Opus	96 kbit/s	meist ununterscheidbar

Faktor 15 gegenüber der CD, ohne dass die meisten Hörer einen Unterschied bemerken. Die verlustfreie Verdichtung schafft im selben Material nur Faktor 1,8. Der Unterschied ist genau das, was das Gehörmodell an Verzichtbarem findet.

Das Modell gilt für einen Durchschnittshörer

Die Schwellen stammen aus Versuchen mit vielen Personen. Einzelne hören mehr.

Kritisches Material	Applaus, Cembalo, Triangel; scharfe Einsätze verraten die Zeitauflösung
Kritische Hörer	Geübte Ohren und gute Anlagen finden Artefakte, die andere nicht hören
Kritische Nutzung	Wer nachbearbeiten will, braucht das Original

Deshalb gibt es verlustfreie Formate weiterhin, obwohl sie viermal so groß sind. Sie sind nicht für das Hören gedacht, sondern für das Aufbewahren.

Berger[3] ist die maßgebliche Darstellung der Rate-Distortion-Theorie; Cover und Thomas[6] beweisen den Satz; Sayood[23] behandelt die praktischen Verfahren ausführlich.

Part III

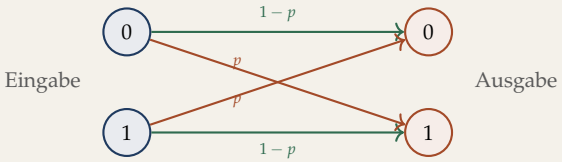
Der Kanal

9 Kanäle und Kapazität

Was ein Kanal ist

Ein Kanal nimmt ein Zeichen entgegen und gibt eines aus, das nicht dasselbe sein muss.

Definition 9.1. Ein **diskreter gedächtnisloser Kanal** besteht aus einem Eingabealphabet $\mathcal{X}$, einem Ausgabealphabet $\mathcal{Y}$ und den Übergangswahrscheinlichkeiten $p(y|x)$. Gedächtnislos heißt: Jede Übertragung ist von den vorigen unabhängig.



Das ist der **binäre symmetrische Kanal**, das Standardbeispiel des Fachs. Mit Wahrscheinlichkeit p kippt das Bit um.

$p = 0$	Fehlerfrei; man kann ein bit je Übertragung senden
$p = \frac{1}{2}$	**Nutzlos**; die Ausgabe ist von der Eingabe unabhängig
$p = 1$	Ebenfalls fehlerfrei; man dreht am Ende jedes Bit um

Die dritte Zeile überrascht viele. Ein Kanal, der immer das Falsche liefert, ist genauso gut wie einer, der immer das Richtige liefert. Schlimm ist nicht der Fehler, sondern die Unvorhersehbarkeit.

Die Kapazität

Definition 9.2. Die **Kapazität** eines Kanals ist

$$C = \max_{p(x)} I(X;Y)$$

in bit je Kanalbenutzung. Das Maximum läuft über alle Eingangsverteilungen.

Der Kanal gibt die Übergangswahrscheinlichkeiten vor; die Eingangsverteilung darf man wählen. Die Kapazität ist also das Beste, was sich aus dem Kanal herausholen lässt, wenn man die Quelle passend einrichtet.

Die Kapazität des binären symmetrischen Kanals.

$$C = 1 - H(p).$$

p	$H(p)$	C	**Wirkungsgrad**
0,000	0,000	1,000	100 %
0,010	0,081	0,919	92 %
0,050	0,286	0,714	71 %
0,110	0,500	0,500	50 %
0,250	0,811	0,189	19 %
0,500	1,000	**0,000**	0 %
0,900	0,469	0,531	53 %

Die letzte Zeile bestätigt die Bemerkung von oben: Bei neunzig Prozent Fehlerwahrscheinlichkeit ist die Kapazität wieder hoch. Die Kurve ist symmetrisch um $p = \frac{1}{2}$, wo sie ihren einzigen Nullpunkt hat.

Der Löschkanal.
Manchmal geht ein Zeichen nicht kaputt, sondern verloren, und der Empfänger merkt es.

	Kippkanal	**Löschkanal**
Ausgabe	0 oder 1	0, 1 oder „?“
Empfänger weiß	nichts	wo der Fehler ist
Kapazität	$1 - H(p)$	$\mathbf{1 - p}$
bei $p = 0{,}1$	0,531	**0,900**

Bei gleicher Fehlerrate ist der Löschkanal erheblich besser. Zu wissen, wo ein Fehler steckt, ist fast so viel wert wie ihn zu vermeiden. Genau darauf beruhen die Verfahren, die bei Datenträgern und Netzwerkpaketen benutzt werden.

Der Kanal mit Rauschen und Bandbreite

Für zeitkontinuierliche Kanäle mit weißem gaußschem Rauschen gilt die berühmteste Formel des Fachs:

$$C = B \log_2\left(1 + \frac{S}{N}\right) \quad \text{bit je Sekunde.}$$

B	Bandbreite in Hertz
S/N	Verhältnis von Signal- zu Rauschleistung

Anwendung	B	S/N	C
Telefonleitung	3,1 kHz	30 dB	$\approx$ 31 kbit/s
WLAN-Kanal	20 MHz	25 dB	$\approx$ 166 Mbit/s
Tiefraumsonde	1 MHz	-3 dB	$\approx$ 0,6 Mbit/s

Die erste Zeile erklärt, warum analoge Modems bei etwa 33 kbit/s stehen blieben. Es lag nicht an der Technik, sondern an einer Grenze, die 1948 berechnet worden war.

Die dritte Zeile zeigt etwas Bemerkenswertes: Auch bei mehr Rauschen als Signal ist die Kapazität positiv. Man muss nur länger senden und mehr Redundanz aufwenden.

Deshalb konnte die Sonde Voyager aus Milliarden Kilometern Entfernung Bilder liefern. Die Empfangsleistung lag bei etwa einem Milliardstel Watt, und die Reed-Solomon-Codes aus Kapitel 11 holten die Daten trotzdem heraus.

Aufgabe 9.1.

(a) Leiten Sie die Kapazität des binären symmetrischen Kanals her.
(b) Berechnen Sie die Kapazität des Löschkanals.
(c) Untersuchen Sie, was Wiederholung bringt.

Lösung Schritt für Schritt.

(a) Der binäre symmetrische Kanal.

Schritt 1 – Die Transinformation ansetzen.

$$I(X;Y) = H(Y) - H(Y|X).$$

Schritt 2 – Der zweite Term.
Ist $X = x$ bekannt, so ist Y mit Wahrscheinlichkeit p verschieden von x:

$$H(Y|X = x) = H(p) \quad \text{für beide } x.$$

$$H(Y|X) = H(p), \quad \text{unabhängig von } p(x).$$

Das ist der Kunstgriff. Der zweite Term hängt nicht von der Eingangsverteilung ab, also muss man nur den ersten maximieren.

Schritt 3 – Den ersten Term maximieren.
Y ist binär, also $H(Y) \leq 1$ mit Gleichheit bei $p(Y{=}0) = \frac{1}{2}$.
Mit $p(X{=}0) = q$:

$$p(Y{=}0) = q(1-p) + (1-q)p.$$

Setzt man $q = \frac{1}{2}$:

$$p(Y{=}0) = \tfrac{1}{2}(1-p) + \tfrac{1}{2}p = \tfrac{1}{2}. \qquad \checkmark$$

Schritt 4 – Zusammensetzen.

$$C = 1 - H(p).$$

Die gleichverteilte Eingabe ist optimal, und zwar für jedes p. Das ist bei symmetrischen Kanälen immer so und bei unsymmetrischen nicht.

Ein unsymmetrisches Gegenbeispiel.
Ein Kanal, bei dem nur die Eins gestört wird: $p(1|1) = 0{,}5$, $p(0|0) = 1$.

$q = p(X{=}0)$	$I(X;Y)$
0,30	0,3966
0,40	0,3958
0,3333	**0,3962**
0,50	0,3113

Das Maximum liegt hier nicht bei $\frac{1}{2}$. Bei allgemeinen Kanälen muss man das Maximum numerisch suchen; das übliche Verfahren stammt von Blahut und Arimoto.

(b) Der Löschkanal.
Der Kanal. Mit Wahrscheinlichkeit $1 - p$ kommt das Zeichen durch, mit Wahrscheinlichkeit p kommt „?“.
Schritt 1 – Die bedingte Entropie.

$$H(X|Y) = p \cdot H(X) + (1 - p) \cdot 0 = p\, H(X).$$

Kam ein Zeichen durch, weiß man alles. Kam ein Fragezeichen, weiß man gar nichts. Der Erwartungswert ist einfach p mal die volle Unsicherheit.

Schritt 2 – Die Transinformation.

$$I(X;Y) = H(X) - p\, H(X) = (1 - p)\, H(X).$$

Schritt 3 – Maximieren.

$$C = (1 - p) \cdot \max H(X) = (1 - p) \cdot 1 = \mathbf{1} - \mathbf{p}.$$

Der Vergleich mit dem Kippkanal.

p	**Kippen**	**Löschen**	**Verhältnis**
0,01	0,919	0,990	1,08
0,05	0,714	0,950	1,33
0,10	0,531	0,900	1,69
0,25	0,189	0,750	3,97
0,40	0,029	0,600	**20,7**

Bei hoher Fehlerrate ist der Unterschied dramatisch. Deshalb baut man Systeme so, dass Fehler erkennbar werden: eine Prüfsumme über jeden Block verwandelt einen Kippkanal in einen Löschkanal.

Genau das tut jedes Netzwerkprotokoll. Ein Paket mit falscher Prüfsumme wird verworfen, nicht repariert. Aus einem Bitfehler wird ein Paketverlust, und der ist billiger zu beheben.

(c) Was Wiederholung bringt.
Der naive Ansatz. Jedes Bit dreimal senden, der Empfänger entscheidet nach Mehrheit.
Die Fehlerrate. Bei $p = 0{,}1$ geht es schief, wenn zwei oder drei Bits kippen:

$$P_{\text{Fehler}} = \binom{3}{2} p^2(1 - p) + p^3 = 3 \cdot 0{,}01 \cdot 0{,}9 + 0{,}001 = 0{,}028.$$

Wiederh.	Rate	Fehlerrate
1	1,000	0,1
3	0,333	$2{,}8 \cdot 10^{-2}$
5	0,200	$8{,}6 \cdot 10^{-3}$
11	0,091	$3{,}3 \cdot 10^{-4}$
21	0,048	$1{,}3 \cdot 10^{-5}$
101	0,010	$2{,}9 \cdot 10^{-21}$

Die Beobachtung.
Die Fehlerrate geht gegen null, und die Rate geht mit.

Das ist die Vorstellung, die vor 1948 herrschte: Fehlerfreiheit kostet Rate, und wer beliebig zuverlässig sein will, muss die Rate gegen null schicken. Es klingt zwingend und ist falsch.

Der Vergleich mit der Kapazität.

$$C = 1 - H(0{,}1) = 1 - 0{,}469 = 0{,}531.$$

Verfahren	Rate	Fehlerrate
Wiederholung $\times 101$	0,010	≈ 0
Was möglich ist	**0,531**	≈ 0
Verhältnis	**53**-fach	

Man kann also mehr als das Fünfzigfache übertragen und dabei ebenso zuverlässig sein. Die Wiederholung ist ein sehr schlechter Code, und dass es sehr viel bessere gibt, ist der Inhalt des nächsten Kapitels.

Warum die Wiederholung so schlecht ist

Sie verteilt die Redundanz gleichmäßig auf jedes einzelne Bit, statt sie über viele Bits zu bündeln.

Wiederholung	Schützt jedes Bit einzeln; drei Fehler an einer Stelle sind tödlich, drei verstreute harmlos
Guter Code	Schützt einen ganzen Block gemeinsam; die Fehler dürfen liegen, wo sie wollen

Bei einem langen Block gleicht sich die Zahl der Fehler statistisch aus. Bei einer Blocklänge von 10 000 und $p = 0{,}1$ liegt die Zahl der Fehler mit hoher Sicherheit zwischen 940 und 1060. Ein Code, der 1100 Fehler verkraftet, ist damit praktisch immer erfolgreich.

Das ist der Kerngedanke von Shannons Beweis: Bei langen Blöcken wird das Verhalten des Kanals berechenbar, obwohl der einzelne Fehler es nicht ist. Es ist dasselbe Argument wie bei den typischen Folgen aus Kapitel 5, nur auf der anderen Seite angewandt.

Nyquist[19] bestimmte 1928 die Grenze der Signalisierungsrate; Shannons Arbeit[24] führt die Kapazität ein; Gallager[10] behandelt Kanalmodelle gründlich; Cover und Thomas[6] rechnen viele Beispiele durch.

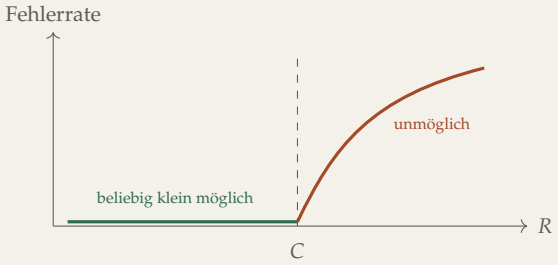
10 Der Kanalcodierungssatz

Die Aussage

Definition 10.1 (Kanalcodierungssatz, Shannon 1948)**.** Für jeden diskreten gedächtnislosen Kanal mit Kapazität C gilt:

- Zu jeder Rate $R < C$ und jedem $\varepsilon > 0$ gibt es einen Code hinreichender Länge mit Fehlerwahrscheinlichkeit kleiner als ε.
- Zu keiner Rate $R > C$ gibt es solche Codes; die Fehlerwahrscheinlichkeit bleibt von null weg beschränkt.

Das ist die überraschendste Aussage der Informationstheorie. Vor 1948 nahm man an, dass Zuverlässigkeit nur durch Rateverzicht zu haben ist. Shannon zeigte, dass es eine scharfe Grenze gibt, unterhalb derer man beides bekommt.



Was der Satz nicht sagt.

Kein Code	Er gibt kein Verfahren an, sondern zeigt nur die Existenz
Keine Länge	Wie lang die Blöcke sein müssen, sagt er nicht genau
Keine Decodierung	Der Beweis benutzt eine Suche, die für lange Codes undurchführbar ist

Genau darin liegt die fünfundvierzigjährige Lücke zwischen 1948 und 1993. Die Grenze war bekannt, und niemand wusste, wie man sie mit vertretbarem Aufwand erreicht.

Der Beweisgedanke: zufällige Codes

Shannons Beweis ist so kühn wie einfach: Er konstruiert keinen guten Code, sondern zeigt, dass die *meisten* Codes gut sind.

1.	Man wählt 2^{nR} Codewörter der Länge n rein zufällig aus.
2.	Man sendet ein Codewort; der Empfänger sucht dasjenige, das am besten zur Empfangsfolge passt.
3.	Man berechnet die **mittlere** Fehlerwahrscheinlichkeit über alle möglichen Codes.
4.	Ist der Mittelwert klein, so muss mindestens ein Code mindestens so gut sein.

Schritt 4 ist der Trick. Man muss den guten Code nicht angeben; es genügt zu wissen, dass er existiert, weil ein Mittelwert nicht kleiner sein kann als alle Einzelwerte.

Warum die Fehlerwahrscheinlichkeit klein wird.
Der Empfänger sucht das Codewort, das mit der Empfangsfolge **gemeinsam typisch** ist. Zwei Dinge können schiefgehen.

Fehler 1	Das gesendete Codewort ist nicht typisch. Wird für großes n beliebig unwahrscheinlich
Fehler 2	Ein *anderes* Codewort passt zufällig auch. Genau hier kommt die Rate ins Spiel

Die Abzählung für Fehler 2.
Die Wahrscheinlichkeit, dass ein zufällig gewähltes Codewort zufällig zur Empfangsfolge passt, ist etwa $2^{-nI(X;Y)}$. Es gibt $2^{nR} - 1$ andere Codewörter, also

$$P(\text{Fehler 2}) \lesssim 2^{nR} \cdot 2^{-nI(X;Y)} = 2^{-n(I(X;Y)-R)}.$$

$R < I(X;Y)$	Der Exponent ist negativ; der Ausdruck geht **exponentiell** gegen null
$R > I(X;Y)$	Der Exponent ist positiv; die Schranke ist wertlos

Wählt man die Eingangsverteilung optimal, so ist $I(X;Y) = C$, und damit steht die Aussage: Unterhalb der Kapazität geht die Fehlerwahrscheinlichkeit exponentiell mit der Blocklänge gegen null.

Ein zufälliger Code ist unbrauchbar

Der Beweis zeigt, dass fast alle Codes gut sind. Trotzdem kann man keinen zufälligen benutzen.

Blocklänge	Codewörter	Tabellengröße
$n = 20$, $R = \frac{1}{2}$	1024	2,5 KB
$n = 100$, $R = \frac{1}{2}$	$1{,}1 \cdot 10^{15}$	14 PB
$n = 1000$, $R = \frac{1}{2}$	10^{150}	aussichtslos

Der Code müsste als Tabelle gespeichert werden, denn er hat keine Struktur. Und die Decodierung müsste alle Codewörter durchgehen. Beides wächst exponentiell mit der Blocklänge, die für die Fehlerfreiheit gerade groß sein muss.

Die Aufgabe der folgenden Jahrzehnte lautete deshalb: Codes finden, die genug Struktur haben, um sie zu speichern und zu decodieren, und genug Zufälligkeit, um so gut zu sein wie die zufälligen. Kapitel 12 erzählt, wie das gelang.

Aufgabe 10.1.

(a) Führen Sie die Abzählung für den Beweis vollständig durch.
(b) Untersuchen Sie, wie lang die Blöcke sein müssen.
(c) Beweisen Sie die Umkehrung.

Lösung Schritt für Schritt.

(a) Die Abzählung.
Die Größen. Blocklänge n, Rate R, also $M = 2^{nR}$ Codewörter.
Schritt 1 – Typische Mengen.

Menge	Umfang
Typische x-Folgen	$\approx 2^{nH(X)}$
Typische y-Folgen	$\approx 2^{nH(Y)}$
Gemeinsam typische Paare	$\approx 2^{nH(X,Y)}$

Schritt 2 – Die entscheidende Wahrscheinlichkeit.
Zieht man ein x und ein y unabhängig aus ihren typischen Mengen, so ist die Wahrscheinlichkeit, dass das Paar gemeinsam typisch ist:

$$\frac{2^{nH(X,Y)}}{2^{nH(X)} \cdot 2^{nH(Y)}} = 2^{-n(H(X)+H(Y)-H(X,Y))} = 2^{-nI(X;Y)}.$$

Hier taucht die Transinformation ganz von selbst auf, als Abzählverhältnis. Das ist der schönste Moment des Beweises: Eine Größe, die als Erwartungswert definiert wurde, erweist sich als Exponent einer Anzahl.

Schritt 3 – Die Vereinigungsschranke.

$$P(\text{Fehler}) \leq \varepsilon_1 + \sum_{m' \neq m} 2^{-nI(X;Y)} \leq \varepsilon_1 + (M-1)\, 2^{-nI(X;Y)}.$$

$$\leq \varepsilon_1 + 2^{nR} \cdot 2^{-nI(X;Y)} = \varepsilon_1 + 2^{-n(I(X;Y)-R)}.$$

Schritt 4 – Zahlen einsetzen.
Sei $C = 0{,}5$ und $R = 0{,}4$, also $C - R = 0{,}1$.

n	$2^{-0{,}1n}$	
10	0,500	unbrauchbar
50	$3{,}1 \cdot 10^{-2}$	
100	$9{,}8 \cdot 10^{-4}$	
500	$8{,}9 \cdot 10^{-16}$	
1000	$7{,}9 \cdot 10^{-31}$	sehr gut

Schritt 5 – Vom Mittel zum Einzelfall.
Der berechnete Wert ist der Mittelwert über alle zufälligen Codes. Also existiert mindestens ein Code, dessen Fehlerwahrscheinlichkeit höchstens so groß ist.

Ein letzter Feinschliff: Der beste Code hat einen kleinen mittleren *Fehler über alle Nachrichten. Wirft man die schlechtere Hälfte der Codewörter weg, so ist der Fehler für jede verbleibende Nachricht klein, und die Rate sinkt nur um $\frac{1}{n}$.*

(b) Wie lang müssen die Blöcke sein?
Die Faustregel aus Schritt 4.

$$n \approx \frac{-\log_2 \varepsilon}{C - R}.$$

$C - R$	$\varepsilon = 10^{-3}$	$\varepsilon = 10^{-6}$	$\varepsilon = 10^{-12}$
0,30	33	66	133
0,10	100	199	399
0,03	332	664	1329
0,01	997	1993	3986
0,001	9966	19 932	**39 864**

Die letzte Zeile zeigt den Preis der Nähe zur Grenze. Wer bis auf ein Promille an die Kapazität heran will, braucht Blöcke von vierzigtausend Zeichen, und damit auch eine entsprechende Verzögerung.

Was das praktisch bedeutet.

Anwendung	**Blocklänge**	**Grund**
Sprachtelefonie	≈ 100	**Verzögerung zählt**
Mobilfunkdaten	1000–8000	
Satellitenrundfunk	64 800	Verzögerung gleichgültig
Datenträger	≈ 4000	

Eine Blocklänge von 64 800 bei einer Datenrate von 30 Mbit/s bedeutet etwa zwei Millisekunden Verzögerung allein für das Sammeln. Beim Fernsehen ist das gleichgültig, bei einem Telefongespräch nicht.

(c) Die Umkehrung.
Die Behauptung. Bei $R > C$ bleibt die Fehlerwahrscheinlichkeit von null weg.
Schritt 1 – Fanos Ungleichung.

Wird W gesendet und $\hat{W}$ geschätzt, mit M möglichen Nachrichten und Fehlerwahrscheinlichkeit P_e, so gilt

$$H(W|\hat{W}) \leq 1 + P_e \log_2 M.$$

In Worten: Wenn man selten irrt, bleibt wenig Unsicherheit. Das eine bit im Voraus steht für die Frage „war es überhaupt falsch“, der zweite Term für „welche der Alternativen war es dann“.

Schritt 2 – Die Kette aufstellen.
Mit $H(W) = \log_2 M = nR$ bei gleichverteilten Nachrichten:

$$\begin{aligned} nR &= H(W) \\ &= I(W;\hat{W}) + H(W|\hat{W}) \\ &\leq I(X^n;Y^n) + 1 + P_e\, nR \qquad \text{(Datenverarbeitung, Fano)} \\ &\leq nC + 1 + P_e\, nR. \end{aligned}$$

Die letzte Zeile benutzt, dass ein gedächtnisloser Kanal über n Benutzungen höchstens nC überträgt.

Schritt 3 – Auflösen.

$$nR \leq nC + 1 + P_e nR \quad \Longrightarrow \quad P_e \geq 1 - \frac{C}{R} - \frac{1}{nR}.$$

Schritt 4 – Auswerten.
Für $n \to \infty$ verschwindet der letzte Term:

$$P_e \geq 1 - \frac{C}{R}.$$

C	R	P_e **mindestens**	
0,5	0,5	0	Grenzfall
0,5	0,6	0,167	
0,5	0,8	0,375	
0,5	1,0	**0,500**	

Die Schranke hängt nicht von der Blocklänge ab. Man kann also nicht durch längere Blöcke ausgleichen, was oberhalb der Kapazität verloren geht. Das ist der Unterschied zwischen einer technischen Schwierigkeit und einer prinzipiellen Grenze.

Damit ist der Satz von beiden Seiten eingeschlossen: unterhalb C beliebig gut, oberhalb C nachweislich nicht. Die Kapazität ist keine Faustregel, sondern eine scharfe Trennlinie.

Shannon[24] führte den Satz und den Beweis mit zufälligen Codes ein; Gallager[10] gibt die klassische Lehrbuchfassung; Csiszár und Körner[7] behandeln die schärferen Schranken.

11 Fehler erkennen und beheben

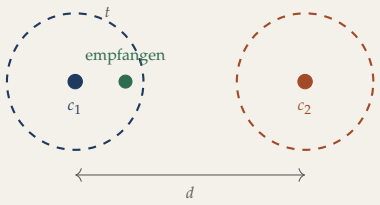
Abstand entscheidet

Ein Code ist eine Auswahl von Wörtern aus allen möglichen. Was er leistet, hängt daran, wie weit sie auseinanderliegen.

Definition 11.1. Der **Hamming-Abstand** zweier gleich langer Wörter ist die Zahl der Stellen, an denen sie sich unterscheiden. Der **Mindestabstand** d eines Codes ist das Minimum über alle Paare von Codewörtern.

Leistung	**Bedingung**	**bei** $d = 3$
Fehler erkennen	$d \geq e + 1$	bis 2 Fehler
Fehler beheben	$d \geq 2t + 1$	**1 Fehler**
Löschungen füllen	$d \geq s + 1$	bis 2 Löschungen

Die Begründung ist geometrisch. Legt man um jedes Codewort eine Kugel vom Radius t, so überschneiden sich die Kugeln nicht, sobald $d \geq 2t + 1$. Jedes empfangene Wort liegt dann in höchstens einer Kugel, und deren Mittelpunkt ist die Antwort.



Der Preis.
Ein größerer Abstand kostet Rate. Man muss mehr Prüfstellen anhängen.

Code	n	k	Rate
Parität	8	7	0,875
Hamming(7,4)	7	4	0,571
Hamming(255,247)	255	247	0,969
Wiederholung $\times 3$	3	1	0,333

Die dritte Zeile zeigt, warum lange Codes günstiger sind. Beide Hamming-Codes beheben genau einen Fehler; der lange kostet dafür drei Prozent Rate, der kurze dreiundvierzig.

Der Hamming-Code

RICHARD HAMMING arbeitete Ende der 1940er Jahre bei Bell Labs an einem Relaisrechner, der am Wochenende unbeaufsichtigt rechnete. Fand er einen Fehler, brach er ab, und montags war nichts fertig.

Seine Bemerkung dazu ist überliefert: Wenn die Maschine erkennen kann, dass ein Fehler vorliegt, warum kann sie dann nicht auch erkennen, wo er liegt, und ihn beheben?

Der Aufbau von Hamming(7,4).
Vier Datenbit $d_1 \dots d_4$, drei Prüfbit p_1, p_2, p_3, angeordnet so, dass die Prüfbit auf den Zweierpotenzstellen stehen.

Stelle	1	2	3	4	5	6	7
Inhalt	p_1	p_2	d_1	p_3	d_2	d_3	d_4

Prüfbit	prüft die Stellen	Gleichung
p_1	$1, 3, 5, 7$	$p_1 = d_1 \oplus d_2 \oplus d_4$
p_2	$2, 3, 6, 7$	$p_2 = d_1 \oplus d_3 \oplus d_4$
p_3	$4, 5, 6, 7$	$p_3 = d_2 \oplus d_3 \oplus d_4$

Die Auswahl der Stellen ist nicht willkürlich: p_1 prüft alle Stellen, deren Nummer im Binärsystem eine Eins an der letzten Stelle hat, p_2 an der vorletzten, p_3 an der drittletzten. Genau daraus ergibt sich die schöne Eigenschaft beim Decodieren.

Die Decodierung.
Man prüft alle drei Gleichungen und schreibt die Ergebnisse als Binärzahl.

$s_3s_2s_1$	**Dezimal**	**Bedeutung**
000	0	kein Fehler
001	1	Stelle 1 kippt
011	3	Stelle 3 kippt
111	7	Stelle 7 kippt

Das Prüfergebnis ist *die Nummer der fehlerhaften Stelle. Das ist keine Rechnung, sondern folgt unmittelbar aus der Zuordnung der Prüfbit zu den Binärstellen.*

Die Hamming-Schranke

Wie viele Prüfbit braucht man mindestens, um einen Fehler zu beheben? Jede Kugel vom Radius 1 um ein Codewort enthält $1 + n$ Wörter. Bei 2^k Codewörtern und 2^n möglichen Wörtern muss gelten

$$2^k(1+n) \leq 2^n \quad \Longleftrightarrow \quad 1+n \leq 2^{n-k}.$$

n	k	2^{n-k}	$1+n$
7	4	8	8 genau
15	11	16	16 genau
31	26	32	32 genau
255	247	256	256 genau

*Die Hamming-Codes erreichen die Schranke mit Gleichheit. Solche Codes heißen **perfekt**: Die Kugeln füllen den ganzen Raum lückenlos aus, jedes empfangene Wort liegt in genau einer.*

Perfekte Codes sind selten. Neben den Hamming-Codes und den trivialen Wiederholungscodes ungerader Länge gibt es nur noch zwei, die von MARCEL GOLAY 1949 gefunden wurden.

Der binäre Golay-Code (23,12) behebt drei Fehler und wurde bei den Voyager-Sonden für die Bildübertragung eingesetzt. Er hat außerdem eine bemerkenswerte Verbindung zu einer der sporadischen einfachen Gruppen der Algebra.

Reed-Solomon-Codes

Hamming-Codes beheben einzelne Bitfehler. In der Praxis treten Fehler oft in Bündeln auf: ein Kratzer, ein Störimpuls, ein defekter Speicherblock.

	Hamming	**Reed-Solomon**
Arbeitet auf	Bits	**Symbolen**
Behebt	Einzelfehler	**Bündelfehler**
Grundlage	lineare Algebra über $\mathbb{F}_2$	**Polynome über** $\mathbb{F}_q$
Typisch	RS$(255, 223)$	16 Symbolfehler

Der Grundgedanke.
Ein Polynom vom Grad $k-1$ ist durch k Stützstellen eindeutig bestimmt. Man wertet es an $n > k$ Stellen aus und überträgt alle Werte.

1. Die k Datensymbole sind die Koeffizienten eines Polynoms f.
2. Gesendet werden $f(x_1), \ldots, f(x_n)$ an n festen Stellen.
3. Der Empfänger sucht das Polynom vom Grad $< k$, das zu den meisten Werten passt.
4. Bis zu $\lfloor (n-k)/2 \rfloor$ falsche Werte ändern daran nichts.

Das ist eine sehr anschauliche Redundanz: Zwei Punkte bestimmen eine Gerade, und wer fünf Punkte einer Geraden sendet, kann zwei davon verlieren und die Gerade trotzdem finden.

Anwendung	Code	Bemerkung
Audio-CD	zwei RS, verschachtelt	Kratzer bis 2,5 mm
QR-Code	RS über $\mathbb{F}_{256}$	bis 30 % zerstört
DSL	RS + Verschachtelung	Störimpulse
Voyager	RS(255,223)	mit Faltungscode
RAID 6	RS-artig	zwei Platten dürfen ausfallen

Die zweite Zeile ist der Grund, warum ein QR-Code noch lesbar ist, wenn ein Firmenlogo in der Mitte klebt. Die Redundanz ist absichtlich so hoch gewählt, dass man einen Teil opfern kann.

Verschachteln macht Bündel zu Einzelfehlern

Ein Code, der t Fehler je Block behebt, versagt bei einem Bündel von $t+1$ aufeinanderfolgenden Fehlern. Die Abhilfe ist einfach und wirkungsvoll.

1. Man schreibt m Codeworte zeilenweise in eine Tabelle.
2. Man sendet spaltenweise.
3. Ein Bündel von m Fehlern trifft jedes Codewort genau einmal.

	ohne	mit ($m = 8$)
Bündel von 8	nicht behebbar	behebbar
Zusatzkosten	—	Speicher und Verzögerung

Die Verschachtelung ändert die Rate nicht. Sie kostet nur Speicher und Verzögerung, weil Sender und Empfänger je eine Tabelle füllen müssen, bevor sie loslegen können. Das ist einer der billigsten Gewinne der Codierungstechnik.

Aufgabe 11.1.

(a) Codieren und decodieren Sie mit Hamming(7,4) vollständig.
(b) Untersuchen Sie Prüfsummen und ihre Grenzen.
(c) Rechnen Sie die Fehlerrate eines Speichers mit und ohne Korrektur aus.

Lösung Schritt für Schritt.

(a) Hamming(7,4) vollständig.
Die Daten. $d_1d_2d_3d_4 = 1011$.
Schritt 1 – Prüfbit berechnen.

$$p_1 = d_1 \oplus d_2 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0,$$
$$p_2 = d_1 \oplus d_3 \oplus d_4 = 1 \oplus 1 \oplus 1 = 1,$$
$$p_3 = d_2 \oplus d_3 \oplus d_4 = 0 \oplus 1 \oplus 1 = 0.$$

Das Codewort.

Stelle	1	2	3	4	5	6	7
Inhalt	p_1	p_2	d_1	p_3	d_2	d_3	d_4
Wert	0	1	1	0	0	1	1

Schritt 2 – Ein Fehler an Stelle 5.
Empfangen wird 0110**1**11.
Schritt 3 – Syndrom berechnen.

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = \mathbf{1},$$
$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 1 \oplus 1 \oplus 1 \oplus 1 = \mathbf{0},$$
$$s_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = \mathbf{1}.$$

Schritt 4 – Auswerten.

$$s_3s_2s_1 = 101_2 = 5.$$

Stelle 5 ist fehlerhaft. Umdrehen liefert 0110011*, das ursprüngliche Codewort. Die Daten sind* $d_1d_2d_3d_4 = 1011$. ✓

Schritt 5 – Zwei Fehler, und was dann geschieht.
Kippen wir Stelle 5 und Stelle 6: empfangen 0110**10**1.

$$s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1,$$
$$s_2 = 1 \oplus 1 \oplus 0 \oplus 1 = 1,$$
$$s_3 = 0 \oplus 1 \oplus 0 \oplus 1 = 0.$$

$$s_3s_2s_1 = 011_2 = 3.$$

Der Decoder korrigiert die falsche Stelle

Er meldet keinen Fehler, sondern behauptet, Stelle 3 sei falsch, und dreht sie um. Das Ergebnis ist ein gültiges Codewort mit **drei** falschen Datenbit.

Fehler	Ergebnis	
0	richtig	gut
1	behoben	gut
2	falsch behoben	**unbemerkt falsch**

Das ist die unangenehmste Eigenschaft jedes Korrekturcodes: Jenseits seiner Leistungsgrenze liefert er nicht „weiß nicht“, sondern eine falsche Antwort mit vollem Selbstvertrauen.

Deshalb hängt man in der Praxis ein zusätzliches Paritätsbit über alles an, den erweiterten Hamming-Code (8,4). Er behebt einen Fehler und *erkennt* zwei, ohne sie zu verschlimmern.

Syndrom	Gesamtparität	Schluss
0	richtig	kein Fehler
$\neq 0$	falsch	ein Fehler, behebbar
$\neq 0$	**richtig**	**zwei Fehler, nicht behebbar**

Dieses Verfahren heißt SECDED und steckt in jedem Serverspeicher mit Fehlerkorrektur. Ein zusätzliches Bit macht aus einem gefährlichen Code einen ehrlichen.

(b) Prüfsummen.

Verfahren	Länge	erkennt
Parität	1 bit	ungerade Fehlerzahl
Quersumme	8–32 bit	vieles, nicht Vertauschungen
CRC-32	32 bit	alle Bündel bis 32 bit
Prüfziffer (IBAN)	2 Stellen	Zifferndreher

Warum die einfache Summe schlecht ist.
Vertauscht man zwei Bytes, ändert sich die Summe nicht. Bei Zahlenketten, die Menschen abtippen, ist das der häufigste Fehler.

Fehlerart	Anteil	Summe erkennt?
Eine Ziffer falsch	$\approx 79\,\%$	ja
Zwei Ziffern vertauscht	$\approx 10\,\%$	nein
Sonstiges	$\approx 11\,\%$	teilweise

Deshalb gewichtet jedes ernsthafte Prüfziffernverfahren die Stellen unterschiedlich. Beim IBAN-Verfahren wird die ganze Zahl modulo 97 gerechnet, und das erkennt Zifferndreher zuverlässig.

Die zyklische Prüfung.
CRC fasst die Nachricht als Polynom über $\mathbb{F}_2$ auf und nimmt den Rest bei Division durch ein festes Generatorpolynom.

Alle Einzelfehler	sicher
Alle Doppelfehler	bei geeignetem Generator sicher
Alle Bündel ≤ 32	sicher
Längere Bündel	mit Wahrscheinlichkeit 2^{-32} übersehen

Die letzte Zeile bedeutet: Etwa eines von vier Milliarden fehlerhaften Paketen kommt unbemerkt durch. Bei zehn Millionen Paketen je Sekunde geschieht das etwa alle sieben Minuten, weshalb wichtige Anwendungen eine zweite Prüfung darüberlegen.

(c) Speicher mit und ohne Korrektur.
Die Angaben. Ein Server mit 128 GB, Fehlerrate etwa 10^{-14} je Bit und Stunde.
Ohne Korrektur.

$$128\ \text{GB} = 1{,}1 \cdot 10^{12}\ \text{bit}.$$

$$\text{Fehler je Stunde} = 1{,}1 \cdot 10^{12} \cdot 10^{-14} = 0{,}011.$$

$$\text{Fehler je Jahr} = 0{,}011 \cdot 8760 = \mathbf{96}.$$

Etwa alle vier Tage ein unbemerkt gekipptes Bit. Es kann in einer ungenutzten Speicherstelle liegen und nichts tun, oder in einem Zeiger, und dann stürzt das Programm ab, ohne dass jemand versteht warum.

Mit SECDED je 64 bit.
Ein Wort ist fehlerhaft, wenn zwei oder mehr Bits kippen. Bei $q = 10^{-14}$ je Bit und Stunde:

$$P(\geq 2 \text{ in } 72\ \text{bit}) \approx \binom{72}{2} q^2 = 2556 \cdot 10^{-28} = 2{,}6 \cdot 10^{-25}.$$

$$\text{Worte} = \frac{1{,}1 \cdot 10^{12}}{64} = 1{,}7 \cdot 10^{10}.$$

$$\text{Doppelfehler je Jahr} = 1{,}7 \cdot 10^{10} \cdot 2{,}6 \cdot 10^{-25} \cdot 8760 = \mathbf{3{,}9 \cdot 10^{-11}}.$$

	ohne	**mit SECDED**
Unbemerkte Fehler/Jahr	96	≈ 0
Erkannte, nicht behebbare	—	$3{,}9 \cdot 10^{-11}$
Mehrkosten Speicher	—	12,5 %

Für zwölfeinhalb Prozent mehr Speicher verschwinden sechsundneunzig unbemerkte Fehler im Jahr. Bei einem Server ist das eine der besten Anlagen überhaupt, und bei einem Arbeitsplatzrechner spart man es meist ein.

Die Rechnung unterstellt unabhängige Fehler. Das ist optimistisch: In der Praxis fallen ganze Zeilen oder Bausteine aus, und dagegen hilft SECDED nicht. Deshalb gibt es Verfahren, die einen kompletten Baustein verkraften, indem sie die Bits eines Wortes über mehrere Bausteine verteilen.

Hammings Aufsatz[11] begründete das Gebiet; Reed und Solomon[21] führten die Polynomcodes ein; Lin und Costello[17] sind die umfassende Darstellung.

12 Codes, die die Grenze erreichen

Fünfundvierzig Jahre Abstand

Shannon zeigte 1948, was möglich ist. Wie weit die Praxis davon entfernt blieb, lässt sich beziffern.

Jahr	Verfahren	Abstand zur Grenze
1950	Hamming	≈ 8 dB
1960	Reed-Solomon	≈ 5 dB
1970	Faltung + Viterbi	≈ 3 dB
1990	RS + Faltung	≈ 2 dB
1993	**Turbo**	**0,5 dB**
1996	LDPC	0,3 dB
2001	LDPC verfeinert	**0,0045 dB**
2009	Polare Codes	beweisbar erreichend

Der Sprung von 1990 auf 1993 ist der bemerkenswerteste. Vier Jahrzehnte mühsamer Verbesserung wurden von einer einzigen Arbeit überholt, die zunächst kaum jemand glaubte.

Was Dezibel hier bedeuten.
Ein Abstand von 3 dB heißt: Man braucht die doppelte Sendeleistung, um dasselbe zu erreichen wie ein idealer Code.

Abstand	Leistungsfaktor	Bedeutung
3 dB	2,0	doppelte Leistung nötig
2 dB	1,6	
0,5 dB	1,1	
0,0045 dB	1,001	praktisch ideal

Bei einer Raumsonde bedeutet 3 dB eine doppelt so große Antenne oder den halben Abstand. Deshalb wurde in der Raumfahrt jedes Zehntel-Dezibel mit erheblichem Aufwand erkauft.

Der gemeinsame Einfall

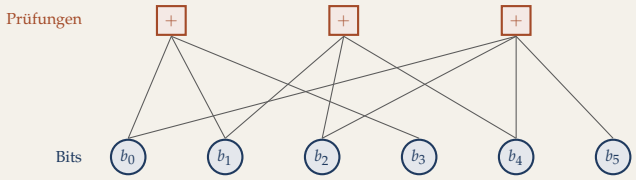
Turbo-Codes und LDPC-Codes wurden unabhängig entwickelt und beruhen auf demselben Gedanken.

Lange Codes	Shannon verlangt große Blocklängen; also nimmt man welche
Zufällige Struktur	Der Code sieht zufällig aus, hat aber genug Struktur zum Rechnen
Weiche Werte	Der Decoder rechnet nicht mit 0 und 1, sondern mit Wahrscheinlichkeiten
Iteration	Teildecoder tauschen ihre Vermutungen aus, bis sie sich einig sind

Die dritte Zeile ist der praktische Schlüssel. Ein Empfänger, der nur „Null“ oder „Eins“ weitergibt, wirft weg, wie sicher er sich ist. Diese Sicherheit ist genau die Information, die der Decoder braucht.

	harte Entscheidung	**weiche Entscheidung**
Weitergabe	0 oder 1	$-3{,}7$ oder $+0{,}2$
Bedeutung	Wert	**Wert und Sicherheit**
Gewinn	—	≈ 2 dB

Der Nachrichtenaustausch beim Decodieren.
Ein LDPC-Code wird durch einen Graphen beschrieben: Knoten für die Bits, Knoten für die Prüfgleichungen, Kanten dazwischen.



1. Jeder Bitknoten schickt seine Anfangsvermutung an alle Prüfknoten.
2. Jeder Prüfknoten rechnet aus, was seine Gleichung über jedes einzelne Bit sagt, und schickt es zurück.
3. Jeder Bitknoten fasst alle eingehenden Meinungen zusammen und aktualisiert seine Vermutung.
4. Wiederholen, meist zwanzig bis fünfzig Mal.

Die Wendung „dünn besetzt“, die im Namen steckt, ist wesentlich. Jeder Bitknoten hängt nur an wenigen Prüfungen, deshalb bleibt der Aufwand je Runde linear, und der Graph enthält kaum kurze Kreise, was die Iteration erst zum Erfolg führt.

Zwei Arbeiten, die niemand glauben wollte

1962: Gallager. Robert Gallager beschrieb in seiner Doktorarbeit am MIT die LDPC-Codes samt iterativem Decoder. Sie erreichten in seinen Rechnungen fast die Kapazität.

> *Die Rechner von 1962 konnten das nicht ausnutzen. Ein Code der Länge 10 000 mit fünfzig Iterationen war jenseits des Möglichen, und die algebraischen Codes wirkten eleganter. Die Arbeit verschwand aus dem Blickfeld.*

1993: Berrou, Glavieux und Thitimajshima. Auf einer Fachtagung stellten drei Ingenieure aus Brest die Turbo-Codes vor und behaupteten, bis auf ein halbes Dezibel an die Kapazität heranzukommen.

Die Reaktion	Verbreitete Skepsis; man vermutete einen Rechenfehler
Der Grund	Zwanzig Jahre lang hatte niemand mehr als kleine Fortschritte erzielt
Die Auflösung	Andere Gruppen bauten es nach, und es stimmte

> *Berrou war kein Codierungstheoretiker im engeren Sinn. Er hat später gesagt, das sei möglicherweise von Vorteil gewesen: Wer die Fachliteratur nicht kennt, weiß auch nicht, was als aussichtslos gilt.*

1996: Die Wiederentdeckung. David MacKay und Radford Neal stießen bei der Suche nach Erklärungen für den Turbo-Erfolg auf Gallagers Arbeit von 1962 und stellten fest, dass dort dasselbe Prinzip stand.

1962	Gallager beschreibt LDPC
1962–1995	fast vollständig vergessen
1993	Turbo-Codes
1996	LDPC wiederentdeckt und als überlegen erkannt
2003	LDPC im Satellitenrundfunk
2016	LDPC und polare Codes im Mobilfunk

Dreiunddreißig Jahre lag die Lösung in einer Bibliothek. Es ist eines der klarsten Beispiele dafür, dass eine Idee nicht nur richtig sein muss, sondern auch zur verfügbaren Technik passen.

Polare Codes

2009 zeigte ERDAL ARIKAN etwas, das die Turbo- und LDPC-Codes nicht leisten: einen Code, von dem sich *beweisen* lässt, dass er die Kapazität erreicht.

Der Gedanke der Polarisierung.

Man kombiniert zwei Kanalbenutzungen zu zwei neuen, virtuellen Kanälen.

	vorher	**nachher**
Kanal 1	$C = 0{,}5$	$C \approx 0{,}2$ schlechter
Kanal 2	$C = 0{,}5$	$C \approx 0{,}8$ besser
Summe	1,0	1,0 *erhalten*

Die Gesamtkapazität bleibt gleich, sie wird nur ungleich verteilt. Wiederholt man das rekursiv, so trennen sich die virtuellen Kanäle immer weiter, bis fast alle entweder nahezu perfekt oder nahezu nutzlos sind.

Stufen	**Kanäle**	**fast perfekt**	**fast nutzlos**
1	2	0	0
4	16	5	6
10	1024	460	480
20	10^6	$\approx 49\,\%$	$\approx 49\,\%$
$\to \infty$	—	$\to C$	$\to 1 - C$

Und dann ist der Code trivial.

Gute Kanäle	Daten hineinschicken; sie kommen fehlerfrei an
Schlechte Kanäle	Feste, dem Empfänger bekannte Werte; sie tragen nichts

Der Anteil der guten Kanäle geht gegen C. Damit erreicht die Rate die Kapazität, und zwar beweisbar. Das ist der erste Code, für den das gilt, einundsechzig Jahre nach Shannons Satz.

	Turbo	LDPC	Polar
Beweis	nein	nein	**ja**
Kurze Blöcke	mittel	schwach	gut
Lange Blöcke	gut	sehr gut	gut
Decodieraufwand	mittel	hoch, parallel	$\mathcal{O}(n \log n)$
Im 5G-Mobilfunk	—	Nutzdaten	**Steuerdaten**

Die letzte Zeile zeigt die Arbeitsteilung. Steuerdaten sind kurz und müssen sehr zuverlässig sein, dafür sind polare Codes gut. Nutzdaten sind lang und brauchen hohen Durchsatz, dafür LDPC.

Aufgabe 12.1.

(a) Führen Sie eine LDPC-Decodierung von Hand durch.
(b) Rechnen Sie die Polarisierung für zwei Kanäle aus.
(c) Vergleichen Sie den Aufwand der Verfahren.

Lösung Schritt für Schritt.

(a) LDPC-Decodierung mit harten Entscheidungen.
Der Code. Sechs Bits, drei Prüfgleichungen:

$$b_0 \oplus b_1 \oplus b_3 = 0, \quad b_1 \oplus b_2 \oplus b_4 = 0, \quad b_0 \oplus b_2 \oplus b_5 = 0.$$

Gesendet. $b = 001011$. Probe:

$$0 \oplus 0 \oplus 0 = 0, \quad 0 \oplus 1 \oplus 1 = 0, \quad 0 \oplus 1 \oplus 1 = 0. \qquad \checkmark$$

Empfangen. $r = 0\mathbf{1}1011$, also Bit 1 gekippt.
Runde 1 – Prüfungen auswerten.

Prüfung	Bits	Summe	erfüllt?
P_1	b_0, b_1, b_3	$0 \oplus 1 \oplus 0 = 1$	nein
P_2	b_1, b_2, b_4	$1 \oplus 1 \oplus 1 = 1$	nein
P_3	b_0, b_2, b_5	$0 \oplus 1 \oplus 1 = 0$	ja

Runde 1 – Stimmen zählen.
Jedes Bit sieht, an wie vielen verletzten Prüfungen es beteiligt ist.

Bit	in Prüfungen	davon verletzt	Anteil
b_0	P_1, P_3	1	$1/2$
$\mathbf{b_1}$	P_1, P_2	**2**	$\mathbf{2/2}$
b_2	P_2, P_3	1	$1/2$
b_3	P_1	1	$1/1$
b_4	P_2	1	$1/1$
b_5	P_3	0	$0/1$

Bit 1 ist als einziges an zwei verletzten Prüfungen beteiligt. Es ist der wahrscheinlichste Schuldige. Die Bits 3 und 4 haben zwar auch den Anteil eins, aber nur eine Stimme statt zweier.

Runde 1 – Umkippen.
b_1 wird von 1 auf 0 gesetzt: $r = 001011$.
Runde 2 – Nachprüfen.

P_1	$0 \oplus 0 \oplus 0$	0	ja
P_2	$0 \oplus 1 \oplus 1$	0	ja
P_3	$0 \oplus 1 \oplus 1$	0	ja

Alle Prüfungen erfüllt, der Decoder hält an. Das Ergebnis ist das gesendete Wort. ✓

Warum weiche Werte besser sind.
Bei zwei gleich verdächtigen Bits kann der harte Decoder nur raten. Der weiche kennt die Empfangsspannungen und weiß, welches unsicherer war.

Empfangswert	harte Deutung	weiche Deutung
$+2{,}8$	1	1, sehr sicher
$+0{,}1$	1	1, **fast geraten**
$-1{,}9$	0	0, ziemlich sicher

Der weiche Decoder wird zuerst am zweiten Bit zweifeln. Dieser Unterschied ist etwa zwei Dezibel wert, also die Hälfte der Sendeleistung.

(b) Polarisierung für zwei Kanäle.

Der Aufbau. Zwei Bits u_1, u_2 werden zu $x_1 = u_1 \oplus u_2$ und $x_2 = u_2$ verknüpft und über zwei binäre Löschkanäle mit Löschwahrscheinlichkeit p gesendet.
Der schlechte virtuelle Kanal W^-.
Er überträgt u_1, wobei u_2 unbekannt ist. Man kann u_1 nur bestimmen, wenn *beide* Ausgaben ankommen.

$$C(W^-) = (1-p)^2.$$

Der gute virtuelle Kanal W^+.
Er überträgt u_2, wobei u_1 bereits bekannt ist. Dann genügt *eine* der beiden Ausgaben.

$$C(W^+) = 1 - p^2.$$

Die Probe auf Erhaltung.

$$\begin{aligned} C(W^-) + C(W^+) &= (1-p)^2 + 1 - p^2 \\ &= 1 - 2p + p^2 + 1 - p^2 \\ &= 2(1-p) = 2\,C(W). \qquad \checkmark \end{aligned}$$

Zahlen für $p = 0{,}5$.

Stufe	schlechtester	bester	Summe/Anz.
0	0,500	0,500	0,500
1	0,250	0,750	0,500
2	0,063	0,938	0,500
3	0,004	0,996	0,500
4	$1{,}5 \cdot 10^{-5}$	0,99998	0,500

Die Kanäle streben auseinander, während der Mittelwert konstant bleibt. Nach vier Stufen ist der beste praktisch fehlerfrei und der schlechteste praktisch nutzlos. Genau das ist die Polarisierung.

Arıkans Satz.
Für $n \to \infty$ geht der Anteil der Kanäle mit Kapazität nahe eins gegen C, und der Anteil derer mit Kapazität nahe null gegen $1 - C$. Der Anteil dazwischen geht gegen null.

Daraus folgt unmittelbar, dass die erreichbare Rate gegen C geht. Der Beweis ist nicht kurz, aber er benutzt nur Martingalkonvergenz und keine Zufallscodes.

(c) Der Aufwand im Vergleich.

Code	Decodieraufwand	bei $n = 10^4$	parallel?
Hamming	$\mathcal{O}(n)$	10^4	ja
Reed-Solomon	$\mathcal{O}(n \log^2 n)$	$1{,}8 \cdot 10^6$	teilweise
Faltung, Viterbi	$\mathcal{O}(n\, 2^K)$	$6{,}6 \cdot 10^5$	nein
Turbo	$\mathcal{O}(n \cdot I)$	$2 \cdot 10^5$	begrenzt
LDPC	$\mathcal{O}(n\, d\, I)$	$3 \cdot 10^6$	**sehr gut**
Polar	$\mathcal{O}(n \log n)$	$1{,}3 \cdot 10^5$	mittel

Warum LDPC trotz hohem Aufwand gewinnt.
Die letzte Spalte entscheidet. Alle Bitknoten lassen sich gleichzeitig aktualisieren, und ebenso alle Prüfknoten.

	Rechenschritte	Zeit bei voller Parallelität
LDPC, $n = 64\,800$	$\approx 10^7$	≈ 50 Runden
Viterbi, $K = 7$	$\approx 10^6$	$\approx 10^6$ Schritte

Der Viterbi-Decoder muss die Zustände der Reihe nach durchgehen, denn jeder hängt vom vorigen ab. Der LDPC-Decoder rechnet zehnmal so viel und braucht dafür zwanzigtausendmal weniger Zeitschritte, wenn genug Recheneinheiten da sind.

Das ist derselbe Gedanke wie in Band 4, Kapitel 11: Nicht die Zahl der Operationen entscheidet, sondern die Länge der Abhängigkeitskette. Ein Verfahren, das sich parallelisieren lässt, schlägt ein sparsameres, das es nicht tut.

Der Fehlerboden

Iterative Decoder haben eine unangenehme Eigenschaft: Unterhalb einer gewissen Fehlerrate sinkt die Kurve nicht mehr steil, sondern flacht ab.

Bereich	Verhalten	Ursache
Wasserfall	steiler Abfall	Iteration greift
Boden	flach	**kurze Kreise im Graphen**

Der Grund: Der Nachrichtenaustausch setzt voraus, dass die eingehenden Meinungen unabhängig sind. Bei einem kurzen Kreis im Graphen bekommt ein Knoten seine eigene Vermutung zurück und bestätigt sich selbst.

Man entwirft die Graphen deshalb so, dass kurze Kreise vermieden werden, und legt für kritische Anwendungen einen zweiten, algebraischen Code darüber, der den Boden wegräumt. Bei Datenträgern ist das Standard.

Gallagers Doktorarbeit[9] enthält LDPC dreißig Jahre zu früh; Berrou und Mitautoren[4] lösten die Wende aus; Arıkan[1] bewies die Polarisierung; MacKay[18] stellt die iterative Decodierung besonders klar dar.

Part IV

Weiterungen

13 Die Information einer einzelnen Folge

Die offene Frage aus Kapitel 2

Die Entropie ist für eine Quelle definiert, nicht für eine feste Zeichenfolge. Trotzdem drängt sich die Frage auf, ob nicht auch eine einzelne Folge einen Informationsgehalt hat.

Folge	Eindruck
`0101010101010101`	regelmäßig
`0110101100101110`	zufällig

Beide sind sechzehn Zeichen lang, und beide haben bei einem fairen Münzwurf dieselbe Wahrscheinlichkeit 2^{-16}. Die Wahrscheinlichkeitsrechnung kann den Unterschied also nicht fassen. Trotzdem ist er offenkundig.

Der Ausweg: nach dem kürzesten Programm fragen.

Definition 13.1. Die **Kolmogorow-Komplexität** $K(x)$ einer Zeichenfolge x ist die Länge des kürzesten Programms, das auf einer festen universellen Maschine x ausgibt und anhält.

Folge	$K(x)$	Programm
n-mal `0`	$\approx \log n$	„schreibe n Nullen“
`0101`…(n Stellen)	$\approx \log n$	„$n/2$-mal 01“
Erste n Stellen von π	$\approx \log n$	**eine Formel**
Zufällige n Stellen	$\approx n$	die Folge selbst

Die dritte Zeile ist bemerkenswert. Die Nachkommastellen von π bestehen jede statistische Zufallsprüfung und haben trotzdem sehr geringe Komplexität, denn ein kurzes Programm erzeugt beliebig viele davon.

Definition 13.2. Eine Folge heißt **zufällig im Sinne von Kolmogorow**, wenn $K(x) \geq |x| - c$ für eine kleine Konstante c gilt: wenn es also kein Programm gibt, das wesentlich kürzer ist als die Folge selbst.

Das ist eine Definition von Zufälligkeit ohne jeden Bezug auf Wahrscheinlichkeiten. Zufällig heißt hier: unkomprimierbar.

Die Maschine spielt fast keine Rolle

Ein Einwand liegt nahe: K hängt doch von der gewählten Maschine ab. Eine Sprache mit dem eingebauten Befehl „gib π aus“ macht π sehr billig.

Invarianzsatz	Für je zwei universelle Maschinen U und V gibt es eine Konstante c mit $\lvert K_U(x) - K_V(x)\rvert \leq c$ für **alle** x

Der Beweis in zwei Zeilen.
U ist universell, kann also V nachahmen. Sei s die Länge des Programms, das U zu einem V-Simulator macht. Dann gilt für jedes x:

$$K_U(x) \leq K_V(x) + s.$$

Ebenso in der anderen Richtung. Mit $c = \max\{s, s'\}$ folgt die Behauptung.

Die Konstante hängt nur von den beiden Maschinen ab, nicht von x. Für lange Folgen wird sie also bedeutungslos. Das rechtfertigt es, von „der“ Komplexität zu sprechen, sofern man nicht auf einzelne bit besteht.

Der Zusammenhang zur Entropie.
Für eine Quelle mit Entropie H gilt

$$\frac{\mathbb{E}[K(X_1 \ldots X_n)]}{n} \longrightarrow H \quad \text{für } n \to \infty.$$

	Entropie	**Kolmogorow**
Definiert für	eine Quelle	**eine Folge**
Braucht	Wahrscheinlichkeiten	**nichts**
Berechenbar	ja	**nein**
Im Mittel	stimmen beide überein	

Die dritte Zeile ist der Preis für die zweite. Man bekommt eine Größe, die ohne Modell auskommt, und verliert die Berechenbarkeit. Das ist kein Zufall, sondern hängt unmittelbar zusammen.

K ist nicht berechenbar

Definition 13.3. Es gibt kein Programm, das zu jeder Eingabe x den Wert $K(x)$ ausgibt.

Der Beweis über das Berry-Paradox.
Angenommen, ein Programm $\mathcal{K}$ berechnet K. Dann schreiben wir folgendes Programm B:

```
Eingabe:  eine Zahl m
Durchlaufe alle Zeichenfolgen x der
Reihe nach:
  wenn K(x) > m:  gib x aus und halte
an
```

Das Programm findet die erste Folge, deren Komplexität größer als m ist. Solche Folgen gibt es, denn es gibt nur $2^m - 1$ Programme kürzer als m, aber unendlich viele Folgen.

Der Widerspruch.
Die Länge von B zusammen mit m beträgt

$$|B| + \log_2 m + O(1).$$

B gibt eine Folge x_0 mit $K(x_0) > m$ aus. Zugleich ist B ein Programm, das x_0 ausgibt, also

$$K(x_0) \leq |B| + \log_2 m + O(1).$$

Zusammen:

$$m < K(x_0) \leq |B| + \log_2 m + O(1).$$

Für hinreichend großes m ist das falsch, denn die linke Seite wächst linear und die rechte nur logarithmisch. Die Annahme, $\mathcal{K}$ existiere, führt also auf einen Widerspruch.

Umgangssprachlich	„Die kleinste Zahl, die sich nicht mit weniger als dreizehn Wörtern beschreiben lässt“ — dieser Satz hat zwölf Wörter
Formal	Genau dasselbe, nur mit Programmlängen statt Wörtern

Es ist dieselbe Struktur wie beim Halteproblem aus Band 2 und bei Gödels Unvollständigkeitssatz: Ein Verfahren, das über sich selbst spricht, erzeugt einen Widerspruch. Der Leitgedanke dieser Reihe taucht hier zum vierten Mal auf.

Was daraus für die Verdichtung folgt

Es gibt kein bestes Verdichtungsprogramm, und es kann keines geben.

Kein Verfahren	findet zu jeder Datei die kürzeste Darstellung
Kein Verfahren	kann beweisen, dass eine Datei nicht kürzer geht
Kein Verfahren	verkleinert alle Dateien

Die dritte Zeile lässt sich elementar zeigen.
Ein verlustfreies Verfahren ist eine umkehrbare Abbildung. Es gibt 2^n Folgen der Länge n und nur $2^n - 1$ Folgen kürzerer Länge insgesamt.

> *Wenn also eine Datei kürzer wird, muss eine andere länger werden. Jedes Verdichtungsprogramm vergrößert die meisten möglichen Eingaben; es funktioniert nur deshalb, weil die Dateien, die tatsächlich vorkommen, ein winziger und sehr geordneter Teil aller möglichen sind.*

Wer eine Datei zweimal packt, sieht das unmittelbar: Der zweite Durchgang macht sie meist ein paar Byte größer.

Aufgabe 13.1.

(a) Zeigen Sie, dass fast alle Folgen unkomprimierbar sind.
(b) Untersuchen Sie die Komplexität konkreter Folgen.
(c) Behandeln Sie das Prinzip der kürzesten Beschreibung.

Lösung Schritt für Schritt.

(a) Fast alle Folgen sind unkomprimierbar.

Die Behauptung. Höchstens ein Anteil von 2^{-c} aller Folgen der Länge n hat $K(x) < n - c$.

Der Beweis durch Abzählen.

Größe	**Anzahl**	
Folgen der Länge n	2^n	
Programme kürzer als $n - c$	$\leq 2^{n-c} - 1$	alle Längen zusammen
Also komprimierbare Folgen	$< 2^{n-c}$	
Anteil	$< 2^{-c}$	

c	**Anteil**	**um c bit kürzbar**
1	50 %	höchstens
10	0,098 %	
20	10^{-4} %	
100	$8 \cdot 10^{-29}$ %	praktisch nie

Weniger als ein Milliardstel aller Dateien lässt sich um dreißig bit verkürzen. Dass Verdichtung trotzdem funktioniert, liegt allein daran, dass Texte, Bilder und Programme nicht zufällig aus allen möglichen Byte-Folgen gezogen werden.

Die Umkehrung als Prüfstein.

Datei	Faktor mit `xz`	Deutung
Deutscher Text	3,5	sehr geordnet
Programmquelltext	4,2	noch geordneter
Ausführbare Datei	2,5	
JPEG-Bild	1,01	**schon verdichtet**
Verschlüsselte Daten	1,00	sieht zufällig aus

Die letzte Zeile ist zugleich ein Prüfkriterium: Wenn sich verschlüsselte Daten verdichten lassen, stimmt mit der Verschlüsselung etwas nicht. Band 8 kommt darauf zurück.

(b) Komplexität konkreter Folgen.
Fall 1 – Eine Million Nullen.

```
for i in range(1000000):  print(0, end="")
```

$K \approx 45$ Zeichen ≈ 360 bit, statt 10^6 bit.

Fall 2 – Die ersten 10^6 Stellen von π.

```
Bailey-Borwein-Plouffe-Formel, etwa 30 Zeilen
```

$K \approx 1000$ bit, statt $3{,}3 \cdot 10^6$ bit.

Kein Verdichtungsprogramm der Welt schafft das. Sie suchen nach Wiederholungen und statistischen Ungleichgewichten, und die Stellen von π haben beides nicht. Die Regelmäßigkeit ist algorithmisch, nicht statistisch.

Folge	`gzip`	K **ungefähr**
10^6 Nullen	$\approx$ 1 KB	45 Byte
π, 10^6 Stellen	$\approx$ 420 KB	$\approx$ 1 **KB**
Echter Zufall	$\approx$ 420 KB	$\approx$ 420 KB

In der mittleren Zeile klafft die Lücke zwischen dem, was ein Verfahren kann, und dem, was möglich wäre, um mehr als Faktor vierhundert. Und man kann nicht einmal entscheiden, ob eine gegebene Datei ein solcher Fall ist.

Fall 3 – Eine Folge mit unbekanntem Status.
Was ist die Komplexität der ersten 10^6 bit der Chaitinschen Konstante Ω, also der Wahrscheinlichkeit, dass ein zufällig gewähltes Programm anhält?

Wohldefiniert	Ja, für eine feste Maschine ist Ω eine bestimmte reelle Zahl
Berechenbar	Nein
Komplexität	$K(\Omega_{1..n}) \geq n - O(1)$: maximal zufällig

Ω ist vollkommen bestimmt und vollkommen unkomprimierbar. Chaitin hat gezeigt, dass jedes formale System nur endlich viele Stellen von Ω beweisen kann. Das ist eine besonders greifbare Form der Unvollständigkeit.

(c) Die kürzeste Beschreibung als Modellwahl.
Das Problem. Man hat Messpunkte und will ein Modell wählen. Ein komplizierteres Modell passt immer besser.

Polynomgrad	**Parameter**	**Restfehler**	
1	2	12,4	
3	4	3,1	
9	10	0,4	
19	20	**0,0**	bei 20 Punkten

Ein Polynom vom Grad 19 geht bei zwanzig Messpunkten exakt durch alle. Es hat nichts gelernt, sondern die Daten auswendig behalten, und sagt zwischen den Punkten Unsinn voraus.

Die Antwort von Rissanen.
Man wählt das Modell, das die *Gesamtbeschreibung* am kürzesten macht.

$$L_{\text{gesamt}} = \underbrace{L(\text{Modell})}_{\text{Parameter}} + \underbrace{L(\text{Daten} \mid \text{Modell})}_{\text{Abweichungen}}.$$

Grad	L(Modell)	L(Daten)	**Summe**
1	16 bit	148 bit	164
3	32 bit	96 bit	128
5	48 bit	62 bit	**110**
9	80 bit	54 bit	134
19	160 bit	0 bit	160

Das Minimum liegt bei Grad fünf. Links davon beschreibt das Modell zu wenig, rechts davon kostet es mehr, als es an Abweichungen einspart. Die Überanpassung wird sichtbar, ohne dass man sie eigens verbieten müsste.

Der Gedanke	Ein gutes Modell ist eines, mit dem sich die Daten kurz beschreiben lassen
Die Verbindung	Verstehen und Verdichten sind zwei Namen für dasselbe
Die Grenze	Die ideale Fassung mit K ist nicht berechenbar; man nimmt praktikable Ersatzmaße

Das ist zugleich eine formale Fassung von Ockhams Rasiermesser. Die einfachere Erklärung ist vorzuziehen, und „einfacher“ heißt hier: kürzer aufzuschreiben, einschließlich der Ausnahmen.

Kolmogorows Aufsatz[14] und Chaitins[5] entstanden unabhängig; Solomonoff[27] kam von der Induktion her; Li und Vitányi[16] sind die vollständige Darstellung; Rissanen[22] begründete die kürzeste Beschreibung als Modellwahl.

14 Dieselbe Größe in anderen Fächern

Warum die Entropie so heißt

Der Zusammenhang zur Thermodynamik ist kein Wortspiel. Die Formeln haben denselben Bau.

	Physik	**Information**
Formel	$S = -k_B \sum p_i \ln p_i$	$H = -\sum p_i \log_2 p_i$
Einheit	Joule je Kelvin	bit
Misst	Zahl der Mikrozustände	Unsicherheit
Maximal bei	Gleichverteilung	Gleichverteilung

Der Unterschied liegt allein im Vorfaktor und in der Basis des Logarithmus. Boltzmanns Konstante ist ein Umrechnungsfaktor zwischen Energie und Temperatur, sonst nichts.

$$1 \text{ bit} \mathrel{\widehat{=}} k_B \ln 2 = 9{,}57 \cdot 10^{-24} \text{ J/K}.$$

Das Prinzip von Landauer.
ROLF LANDAUER zeigte 1961: Das *Löschen* eines bit setzt notwendig Wärme frei.

$$E_{\min} = k_B T \ln 2 \approx 2{,}9 \cdot 10^{-21} \text{ J bei } 300 \text{ K}.$$

Rechnen	kann im Grundsatz umkehrbar und damit beliebig energiearm sein
Löschen	**nicht**; wer zwei Zustände auf einen abbildet, verliert Information

Die Begründung ist eine Buchhaltung: Der Phasenraum des Systems schrumpft beim Löschen um den Faktor zwei. Da die Gesamtentropie nicht abnehmen darf, muss die Differenz an die Umgebung abgegeben werden.

Vorgang	Energie je bit	Verhältnis
Landauer-Grenze	$2{,}9 \cdot 10^{-21}$ J	1
Heutiger Transistor	$\approx 10^{-17}$ J	≈ 3400
Transistor 1990	$\approx 10^{-13}$ J	$\approx 3 \cdot 10^{7}$

Die Technik liegt noch mehr als drei Größenordnungen über der physikalischen Grenze. Sie ist also kein unmittelbares Hindernis, wohl aber eine Aussage darüber, wohin die Reise höchstens gehen kann.

Maxwells Dämon und seine Auflösung

JAMES CLERK MAXWELL beschrieb 1867 ein Gedankenexperiment: Ein Wesen an einer Klappe zwischen zwei Gasbehältern lässt schnelle Moleküle nach links und langsame nach rechts. Es entsteht ein Temperaturunterschied ohne Arbeit, und damit ein Widerspruch zum zweiten Hauptsatz.

Jahr	Wer	Vorschlag
1929	Szilárd	Das Messen kostet Entropie
1961	Landauer	**Das Löschen kostet**
1982	Bennett	Messen kann umkehrbar sein

> *Die heutige Auflösung stammt von* CHARLES BENNETT*: Der Dämon muss sich merken, was er gemessen hat. Sein Gedächtnis füllt sich. Um weiterzumachen, muss er löschen, und genau dabei zahlt er die Rechnung.*

Das ist bemerkenswert: Ein hundert Jahre altes physikalisches Rätsel wurde durch eine Überlegung über Information gelöst, nicht durch eine über Wärme.

Experimentell bestätigt wurde die Landauer-Grenze 2012 an einem einzelnen Teilchen in einer optischen Falle. Der gemessene Wärmebetrag stimmte im Rahmen der Genauigkeit.

Information im Nervensystem

HORACE BARLOW schlug 1961 vor, die Verarbeitung sinnlicher Reize als Verdichtungsproblem zu lesen.

Die Beobachtung	Natürliche Reize sind hochgradig redundant; benachbarte Bildpunkte sind ähnlich
Die Vermutung	Frühe Verarbeitungsstufen entfernen diese Redundanz
Die Folgerung	Die Antworten sollten möglichst unabhängig und sparsam sein

Der Gedanke ist ökonomisch: Nervenzellen und Aktionspotentiale kosten Energie. Wer dieselbe Information mit weniger Aktivität überträgt, spart. Das ist genau die Frage aus Teil II, nur mit Stoffwechsel statt Bandbreite.

Zentrum-Umfeld	Ganglienzellen der Netzhaut antworten auf Kontraste, nicht auf gleichmäßige Helligkeit
Anpassung	Die Kennlinie verschiebt sich mit der mittleren Helligkeit
Sparsame Codierung	Wenige Zellen sind gleichzeitig aktiv

Die erste Zeile ist eine Differenzbildung und damit eine Vorhersagecodierung: Übertragen wird nicht der Wert, sondern die Abweichung vom Erwarteten. Kapitel 7 kennt dasselbe Verfahren aus der Bildverdichtung.

Wie viel überträgt eine Nervenzelle?

Präparat	bit je Sekunde	bit je Aktionspotential
Fliege, Bewegungssehen	≈ 100	≈ 1–2
Wirbeltiernetzhaut	≈ 10–50	≈ 1–3
Theoretische Obergrenze	—	≈ 5

Die Werte stammen aus Messungen, bei denen man die Transinformation zwischen Reiz und Antwort schätzt. Das Verfahren ist heikel, weil solche Schätzungen bei wenigen Beobachtungen systematisch zu hoch ausfallen; man arbeitet deshalb mit Korrekturen und Extrapolation.

Transinformation ist keine Ursache

Die Symmetrie aus Kapitel 3 hat hier eine unbequeme Folge: $I(\text{Reiz}; \text{Antwort})$ sagt nichts darüber, ob die Antwort für das Verhalten benutzt wird.

Sagt	Die Antwort hängt statistisch mit dem Reiz zusammen
Sagt nicht	Der Organismus wertet diesen Zusammenhang aus

Eine Zelle kann eine Reizeigenschaft zuverlässig abbilden, die kein nachgeschalteter Teil je liest. Die Informationsmenge ist eine obere Schranke für das, was nutzbar ist, und keine Aussage über den tatsächlichen Gebrauch.

Band 14 dieser Reihe geht der Frage nach, wie sich das eingrenzen lässt: durch Eingriffe statt bloßer Beobachtung, und durch Modelle, die den Decodierer mitbeschreiben.

Information und Geheimhaltung

Shannon veröffentlichte 1949 eine zweite grundlegende Arbeit, in der er die Verschlüsselung informationstheoretisch fasste.

Definition 14.1. Ein Verfahren heißt **vollkommen sicher**, wenn der Geheimtext keinerlei Information über den Klartext trägt:

$$I(M;C) = 0, \quad \text{also} \quad H(M|C) = H(M).$$

Shannons Satz. Vollkommene Sicherheit erfordert

$$H(K) \geq H(M).$$

Der Schlüssel muss also mindestens so viel Entropie haben wie die Nachricht. Das ist eine harte Grenze und der Grund, warum das Einmalblatt zwar beweisbar sicher und zugleich praktisch kaum brauchbar ist.

	Einmalblatt	Übliche Verfahren
Schlüssellänge	**wie die Nachricht**	128–256 bit
Sicherheit	beweisbar	**aufwandsbasiert**
Bricht bei	Wiederverwendung	genug Rechenzeit

Alles, was heute benutzt wird, ist im Sinne dieser Definition unsicher: Der Geheimtext bestimmt den Klartext eindeutig. Er tut es nur nicht auf eine Weise, die sich in vertretbarer Zeit ausrechnen lässt. Band 8 handelt von dieser Verschiebung.

Aufgabe 14.1.

(a) Rechnen Sie die Landauer-Grenze für einen realen Rechner aus.
(b) Schätzen Sie die Informationsrate des Sehsystems ab.
(c) Zeigen Sie Shannons Satz über vollkommene Sicherheit.

Lösung Schritt für Schritt.

(a) Landauer für einen Rechner.
Die Angaben. Ein Prozessor mit 10^{10} Transistoren, 4 GHz, davon schalten je Takt etwa ein Prozent.
Schritt 1 – Löschvorgänge je Sekunde.

$$10^{10} \cdot 0{,}01 \cdot 4 \cdot 10^{9} = 4 \cdot 10^{17} \text{ je Sekunde.}$$

Schritt 2 – Die untere Schranke.

$$P_{\min} = 4 \cdot 10^{17} \cdot 2{,}9 \cdot 10^{-21}\ \mathrm{J} = 1{,}16 \cdot 10^{-3}\ \mathrm{W} \approx 1{,}2\ \mathrm{mW}.$$

Schritt 3 – Der Vergleich.

Größe	**Wert**
Landauer-Grenze	1,2 mW
Tatsächlich	≈ 125 W
Verhältnis	$\approx 10^{5}$

Fünf Größenordnungen Luft. Der Verbrauch stammt fast vollständig aus dem Umladen von Kapazitäten und aus Leckströmen, nicht aus dem thermodynamisch unvermeidbaren Anteil.

Wo die Grenze näher rückt.

System	Abstand	
Heutiger Prozessor	10^5	
Biologische Synapse	$\approx 10^4$	
Einzelmolekülmotor	≈ 10	nahe dran

Die Natur arbeitet in einigen Fällen deutlich näher an der Grenze als die Technik. Das liegt daran, dass sie mit einzelnen Molekülen und thermischer Bewegung arbeitet statt mit Spannungen, die weit über dem Rauschen liegen müssen.

(b) Die Informationsrate des Sehsystems.
Stufe 1 – Die Rohdaten.

Größe	Wert	
Fotorezeptoren	$\approx 1{,}2 \cdot 10^8$	je Auge
Zeitliche Auflösung	≈ 50 Hz	
Amplitudenstufen	≈ 100	$\approx 6{,}6$ bit
Roh	$\approx 4 \cdot 10^{10}$ bit/s	

Stufe 2 – Der Sehnerv.

Ganglienzellen	$\approx 10^6$	je Auge
Rate je Zelle	≈ 10 bit/s	geschätzt
Übertragen	$\approx 10^7$ bit/s	

$$\text{Verhältnis} = \frac{4 \cdot 10^{10}}{10^7} = \mathbf{4000}.$$

Zwischen Netzhaut und Sehnerv liegt eine Verdichtung um etwa Faktor viertausend. Ein Teil davon ist verlustfrei, weil benachbarte Rezeptoren ohnehin fast dasselbe messen. Ein großer Teil ist verlustbehaftet, und genau deshalb sehen wir nicht alles.

Stufe 3 – Was bewusst ankommt.

Stufe	Rate	Faktor
Rezeptoren	$4 \cdot 10^{10}$ bit/s	1
Sehnerv	10^7 bit/s	$4 \cdot 10^3$
Bewusste Verarbeitung	≈ 50 bit/s	$\mathbf{8 \cdot 10^8}$

Solche Zahlen sind mit Vorsicht zu lesen

Die letzte Zeile	beruht auf Aufgaben, bei denen Versuchspersonen Entscheidungen treffen; sie misst einen Engpass, nicht „das Bewusstsein“
Schätzverzerrung	Transinformation wird bei wenigen Beobachtungen systematisch überschätzt
Modellabhängigkeit	jede Zahl setzt ein Modell dessen voraus, was als Reiz und als Antwort gilt

Die Größenordnungen sind gut belegt, die einzelnen Zahlen nicht. Wer sie als Messwerte behandelt statt als Abschätzungen, überdehnt das Verfahren. Als Argument dafür, dass das Sehsystem überwiegend wegwirft und nicht überträgt, tragen sie trotzdem.

(c) Shannons Satz über vollkommene Sicherheit.
Die Behauptung. Aus $I(M;C) = 0$ folgt $H(K) \geq H(M)$.
Schritt 1 – Der Empfänger kann entschlüsseln.
Aus C und K ergibt sich M eindeutig, also

$$H(M \mid C, K) = 0.$$

Schritt 2 – Die Kettenregel anwenden.

$$\begin{aligned} H(M \mid C) &\leq H(M, K \mid C) \\ &= H(K \mid C) + \underbrace{H(M \mid C, K)}_{=0} \\ &= H(K \mid C) \\ &\leq H(K). \end{aligned}$$

Schritt 3 – Die Voraussetzung einsetzen.
Vollkommene Sicherheit bedeutet $H(M|C) = H(M)$, also

$$H(M) \leq H(K).$$

Die Probe am Einmalblatt.

Größe	Wert	
Nachricht	n bit	$H(M) \leq n$
Schlüssel	n bit, zufällig	$H(K) = n$
Bedingung	$n \leq n$	erfüllt

Warum Wiederverwendung alles zerstört.
Werden zwei Nachrichten mit demselben Schlüssel verschlüsselt, so gilt

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2.$$

> *Der Schlüssel fällt heraus. Übrig bleibt die Verknüpfung zweier Klartexte, und bei natürlichsprachlichen Nachrichten lassen sich beide daraus rekonstruieren. Genau daran scheiterte das sowjetische Verfahren im Venona-Projekt: Schlüsselblätter waren versehentlich doppelt ausgegeben worden.*

Der Name „Einmalblatt“ ist also keine Empfehlung, sondern eine Bedingung. Ein zweimal benutzter Einmalschlüssel ist kein schwächeres Verfahren, sondern gar keines.

Landauers Aufsatz[15] ist kurz und folgenreich; Barlow[2] begründete die sparsame Codierung; Shannons Arbeit über Geheimhaltung[25] enthält den Satz; Cover und Thomas[6] behandeln die Verbindungen zur Statistik und Physik.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Quellencodierung	gelöst	seit 1948 bewiesen, seit 1977 praktisch
Kanalcodierung	gelöst	seit 2009 auch beweisbar erreicht
Entropiecodierung	gelöst	arithmetisch und ANS sind optimal
Fehlerkorrektur	gelöst	für den einzelnen Kanal
Modellierung	in Arbeit	**das eigentliche Restproblem**
Verlustmaße	in Arbeit	Wahrnehmung ist schwer zu fassen
Kurze Blöcke	in Arbeit	Shannon gilt asymptotisch
Netze statt Kanäle	in Arbeit	mehrere Sender und Empfänger
Kolmogorow-Komplexität	offen	**grundsätzlich unberechenbar**
Kapazität allgemeiner Netze	offen	seit Jahrzehnten
Semantische Information	offen	Bedeutung bleibt außen vor

Die fünfte Zeile ist die wichtigste. Die Theorie sagt, dass die Entropie die Grenze ist. Sie sagt nicht, welches Modell die Entropie klein macht, und genau daran arbeitet die Praxis seit siebzig Jahren.

Die zweitletzte Zeile verdient eine Erläuterung.

Für einen einzelnen Kanal ist die Kapazität seit 1948 bekannt. Für ein Netz mit mehreren Sendern, Empfängern und Zwischenstationen ist sie es im Allgemeinen nicht.

Anordnung	Kapazität	seit
Ein Sender, ein Empfänger	bekannt	1948
Mehrfachzugriff	bekannt	1971
Rundsendung, abgestuft	bekannt	1972
Zwischenstation	unbekannt	**seit 1971 offen**
Störkanal	unbekannt	

Der einfachste offene Fall besteht aus drei Beteiligten: einem Sender, einem Empfänger und einer Zwischenstation, die helfen darf. Seit über fünfzig Jahren kennt man nur Schranken, keine exakte Antwort.

Fünf Entwicklungen

Erstens: Verdichten und Vorhersagen sind dasselbe geworden.
Der Zusammenhang war seit der arithmetischen Codierung bekannt: Wer die Wahrscheinlichkeit des nächsten Zeichens kennt, kann optimal codieren. Neu ist, dass die Vorhersager sehr viel besser geworden sind.

Vorhersager	bit je Zeichen	Jahr
Buchstabenhäufigkeit	4,10	—
`gzip`	≈ 2,6	1991
Kontextmischung	≈ 1,2	2005
Sprachmodell	≈ 0,7	2020er
Shannons Rateversuch	0,6–1,3	1951

Ein Sprachmodell ist ein Vorhersager, und ein Vorhersager lässt sich unmittelbar als Verdichter benutzen. Man erreicht damit Verhältnisse, die kein herkömmliches Verfahren schafft, und zahlt mit einem Modell von einigen Gigabyte und erheblicher Rechenzeit.

Verfahren	Faktor	Geschw.	Modell
`gzip`	3,1	100 MB/s	32 KB
`zstd`	3,6	500 MB/s	8 MB
`xz`	4,3	3 MB/s	64 MB
`cmix`	6,5	0,0002 MB/s	30 GB
Sprachmodell	≈ 8	sehr langsam	Gigabyte

Die Spanne beim Tempo beträgt sechs Größenordnungen für einen Gewinn von Faktor zweieinhalb. Deshalb bleibt `zstd` für den Alltag die vernünftige Wahl, und die extremen Verfahren sind Forschungsgegenstände.

Zweitens: Die Frage kurzer Blöcke.
Shannons Sätze gelten asymptotisch. Für Anwendungen mit strengen Verzögerungsgrenzen ist das unbefriedigend.

Neue Theorie	Seit etwa 2010 gibt es genaue Näherungen für endliche Blocklängen
Die Größe	Ein Zusatzterm, der von der „Kanaldispersion“ abhängt
Der Nutzen	Man weiß jetzt, wie viel man bei Länge 100 oder 500 erwarten darf

Blocklänge	C	erreichbar	Anteil
100	0,500	0,36	72 %
1000	0,500	0,45	90 %
10 000	0,500	0,484	97 %
∞	0,500	0,500	100 %

Bei Blocklänge hundert sind nur etwa siebzig Prozent der Kapazität zu haben, gleichgültig wie gut der Code ist. Das ist keine Schwäche der Verfahren, sondern eine Eigenschaft des Problems, und für Anwendungen mit sehr kurzen Verzögerungen entscheidend.

Drittens: Fehlerkorrektur für Quantenrechner.
Ein Quantenbit lässt sich nicht kopieren und nicht ohne Störung auslesen. Beides sind Grundlagen der klassischen Codierung.

	klassisch	Quanten
Kopieren	selbstverständlich	unmöglich
Messen	harmlos	zerstört den Zustand
Fehlerarten	Kippen	Kippen **und** Phasendrehung
Aufwand	+12 %	100–1000 physikalische je logisches

Der Ausweg besteht darin, nur die Fehlersyndrome zu messen und nie den Zustand selbst. Man erfährt, dass *etwas gekippt ist, ohne zu erfahren,* was *da war. Die letzte Zeile zeigt, warum brauchbare Quantenrechner so schwer zu bauen sind.*

Viertens: Verlustmaße nach Wahrnehmung statt nach Quadrat.
Der quadratische Fehler ist rechnerisch bequem und sagt wenig darüber, wie etwas aussieht.

Klassisch	Fehlerquadrat; zwei sehr verschiedene Bilder können denselben Wert haben
Strukturmaße	Vergleichen lokale Muster statt einzelner Punkte
Gelernte Maße	Ein Modell schätzt, was ein Mensch bemerken würde

Damit verschiebt sich die Rate-Distortion-Theorie: Die Kurve hängt davon ab, welches Verzerrungsmaß man ansetzt, und ein besseres Maß erlaubt kleinere Dateien bei gleichem Eindruck.

Fünftens: Codierung im Netz statt auf der Strecke.
Die klassische Sicht behandelt jede Verbindung für sich. Bei Netzen lässt sich mehr erreichen, wenn Zwischenstationen nicht nur weiterleiten, sondern rechnen.

Netzcodierung	Ein Knoten verknüpft eingehende Pakete, statt sie einzeln weiterzugeben
Verteilte Speicherung	Daten über viele Server so verteilen, dass Ausfälle billig zu reparieren sind
Springbrunnen-codes	Beliebig viele Pakete erzeugen; der Empfänger braucht irgendwelche k davon

Die dritte Zeile löst ein altes Ärgernis: Bei einer Rundsendung an viele Empfänger, die verschiedene Pakete verlieren, muss nicht mehr jedem einzeln nachgeliefert werden. Jeder sammelt, bis er genug hat, gleichgültig welche.

Drei Einsichten

Erstens: Information misst Unwahrscheinlichkeit, nicht Bedeutung. Das ist die Voraussetzung dafür, dass das Fach überhaupt funktioniert, und zugleich seine schärfste Beschränkung.

	Was die Theorie kann	Was sie nicht kann
Menge	exakt	—
Grenzen	beweisbar	—
Bedeutung	—	gar nichts
Nutzen	—	gar nichts

Shannon selbst hat 1956 in einem kurzen Text davor gewarnt, sein Maß auf Gebiete zu übertragen, in denen es nichts zu suchen hat. Der Aufsatz heißt „The Bandwagon“ und ist eine halbe Seite lang.

Zweitens: Verdichten heißt vorhersagen.

Ein Modell	liefert Wahrscheinlichkeiten für das nächste Zeichen
Ein Codierer	wandelt Wahrscheinlichkeiten optimal in bit um
Der zweite Teil	ist seit 1976 gelöst; der erste ist die ganze Arbeit

Daraus folgt eine Umkehrung, die dieser Band mehrfach berührt hat: Wer gut verdichtet, hat verstanden. Die kürzeste Beschreibung eines Datensatzes ist das beste Modell dafür, und das ist die Aussage von Rissanens Prinzip aus Kapitel 13.

Drittens: Grenzen kennt man früher als Verfahren.

	Grenze	**erreicht**	**Abstand**
Quellencodierung	1948	1977	29 Jahre
Kanalcodierung	1948	1993	**45 Jahre**
Beweisbar erreicht	1948	2009	61 Jahre

Ein Existenzbeweis ist kein Verfahren. Shannons Beweis zeigte, dass fast alle Codes gut sind, und keiner davon war benutzbar. Es brauchte fünfundvierzig Jahre und einen Umweg über eine Arbeit, die dreiunddreißig Jahre lang in einer Bibliothek lag.

Band 8 wendet sich der Kryptographie zu und damit einer Verschiebung, die in Kapitel 14 schon angeklungen ist: Von der Sicherheit, die sich beweisen lässt, zur Sicherheit, die auf Aufwand beruht. Es ist derselbe Übergang wie von Band 2 zu Band 3, von der Berechenbarkeit zur Komplexität.

Information ist die Auflösung von Ungewissheit.

— eine oft CLAUDE SHANNON zugeschriebene Zuspitzung.
In dieser Kürze steht sie nicht in seinen Arbeiten;
sie trifft trotzdem genau, was das Maß misst, und ebenso genau,
was es nicht misst. Ungewissheit über was,
bleibt offen und muss offen bleiben.

Part V

Anhang

A Formeln und Zahlen

Die Grundformeln

Informationsgehalt	$I(x) = -\log_2 p(x)$
Entropie	$H(X) = -\sum_i p_i \log_2 p_i$
Binäre Entropie	$H(p) = -p \log p - (1-p)\log(1-p)$
Verbundentropie	$H(X,Y) = H(X) + H(Y\|X)$
Transinformation	$I(X;Y) = H(X) + H(Y) - H(X,Y)$
Kapazität	$C = \max_{p(x)} I(X;Y)$
Rate-Distortion	$R(D) = \min I(X;\hat{X})$ bei $\mathbb{E}[d] \leq D$
Shannon-Hartley	$C = B \log_2(1 + S/N)$

Kapazitäten und Schranken

Binärer symmetrischer Kanal	$C = 1 - H(p)$
Binärer Löschkanal	$C = 1 - p$
Kraft-Ungleichung	$\sum_i 2^{-\ell_i} \leq 1$
Quellencodierung	$H \leq \bar{\ell} < H + 1$
Huffman-Optimalität	$\bar{\ell}_{\text{Huffman}} \leq \bar{\ell}$ für alle Präfixcodes
Hamming-Schranke	$2^k \sum_{i=0}^{t} \binom{n}{i} \leq 2^n$
Fehlerkorrektur	$d \geq 2t + 1$ behebt t Fehler
Landauer	$E_{\min} = k_B T \ln 2$
Vollkommene Sicherheit	$H(K) \geq H(M)$

Zahlenwerte

Größe	Wert	Bemerkung
$\log_2 3$	1,585	
$\log_2 6$	2,585	Würfel
$\log_2 10$	3,322	eine Dezimalstelle
$\log_2 26$	4,700	ein Buchstabe
1 nat	1,443 bit	
$H(0{,}1)$	0,469	
$H(0{,}11)$	0,500	
$H(0{,}5)$	1,000	Höchstwert
Deutscher Text	$\approx 1{,}3$ bit/Zeichen	mit gutem Modell
Landauer bei 300 K	$2{,}9 \cdot 10^{-21}$ J	je gelöschtem bit
Sehnerv	$\approx 10^7$ bit/s	je Auge

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Was Information ist	Band 1 (Sortierschranke)
2	Entropie	Band 0 (Logarithmen)
3	Transinformation	Band 14 (Neurowissenschaft)
4	Codes	Band 1 (Bäume)
5	Quellencodierungssatz	Band 3 (asymptotische Aussagen)
6	Huffman, arithmetisch	Band 1 (Gierige Verfahren)
7	Wörterbücher, Modelle	Band 13 (Lernende Systeme)
8	Verlustbehaftet	Band 4 (Signalverarbeitung)
9	Kanäle	Band 9 (Netze)
10	Kanalcodierungssatz	Band 3 (Existenzbeweise)
11	Fehlerkorrektur	Band 5 (Speicher, Dauerhaftigkeit)
12	Moderne Codes	Band 11 (Graphen)
13	Kolmogorow	**Band 2 (Halteproblem)**
14	Andere Fächer	Band 8, Band 14, Band 15
15	Stand 2026	Band 8 (Kryptographie)

Sechs Verbindungen zur Mathematikreihe

Entropie	Konvexität, Jensensche Ungleichung
Typische Folgen	Gesetz der großen Zahlen
Kanalcodierungssatz	Konzentration von Maßen
Reed-Solomon	Polynome über endlichen Körpern
LDPC-Decodierung	Graphen, Nachrichtenaustausch
Polarisierung	Martingale und ihre Konvergenz

Die vorletzte Zeile ist zugleich die Verbindung zu Band 11 dieser Reihe. Ein LDPC-Decoder ist ein Verfahren auf einem dünn besetzten Graphen, und seine Güte hängt an dessen Kreisstruktur.

C Glossar

Begriffe

ANS	Asymmetrisches Zahlensystem; schnelle optimale Codierung
Arithmetische Codierung	Verfahren, das ganze Nachrichten als Intervall darstellt
bit	Einheit der Information; auch: Speicherstelle
Entropie	Mittlerer Informationsgehalt einer Quelle
Fehlerboden	Abflachen der Fehlerkurve bei iterativen Decodern
Hamming-Abstand	Zahl der abweichenden Stellen
Kapazität	Größte fehlerfrei übertragbare Rate
Kolmogorow-Komplexität	Länge des kürzesten erzeugenden Programms
Kraft-Ungleichung	Bedingung für die Existenz eines Präfixcodes
LDPC	Code mit dünn besetzter Prüfmatrix
Löschkanal	Kanal, bei dem Fehler erkennbar sind
Polarisierung	Aufspaltung in fast perfekte und fast nutzlose Kanäle
Präfixcode	Kein Codewort ist Anfang eines anderen
Rate	Nutzbit je gesendetem Zeichen
Rate-Distortion	Kleinste Rate bei erlaubter Verzerrung
Redundanz	Differenz zwischen benutzter Rate und Entropie
Syndrom	Prüfergebnis, das den Fehlerort angibt
Transinformation	Was zwei Größen übereinander verraten
Typische Folge	Folge mit ungefähr erwarteter Zusammensetzung
Verschachtelung	Umsortieren, das Bündelfehler verteilt
Weiche Entscheidung	Weitergabe von Wert und Sicherheit

D Zeittafel

Von 1924 bis 2026

1924	Nyquist: Grenze der Signalisierungsrate
1928	Hartley: Information als Logarithmus
1948	**Shannon: A Mathematical Theory of Communication**
1949	Shannon: Theorie der Geheimhaltung
1949	Golay: perfekte Codes
1950	**Hamming: fehlerkorrigierende Codes**
1951	Shannon: Entropie gedruckter Sprache
1952	**Huffman: optimaler Präfixcode**
1955	Elias: Faltungscodes
1956	Shannon: „The Bandwagon“
1960	Reed, Solomon: Polynomcodes
1961	Landauer: Löschen kostet Energie
1961	Barlow: sparsame Codierung im Nervensystem
1962	**Gallager: LDPC — und dann vergessen**
1964	Solomonoff: induktive Inferenz
1965	**Kolmogorow: algorithmische Information**
1966	Chaitin: unabhängig dasselbe
1967	Viterbi: Decodierung von Faltungscodes
1968	Berlekamp, Massey: RS-Decodierung
1971	Cover: Kanal mit Zwischenstation — bis heute offen
1976	Rissanen, Pasco: arithmetische Codierung

1977	**Ziv, Lempel: LZ77**
1978	Rissanen: kürzeste Beschreibung als Modellwahl
1978	Ziv, Lempel: LZ78
1982	Bennett: Auflösung von Maxwells Dämon
1984	Welch: LZW
1987	Cleary, Witten: PPM
1991	Deutsch: `deflate`
1993	**Berrou u. a.: Turbo-Codes**
1995	Shor, Steane: Quantenfehlerkorrektur
1996	MacKay, Neal: LDPC wiederentdeckt
1996	Burrows, Wheeler: Blocksortierung
2001	Richardson, Urbanke: LDPC bis 0,0045 dB
2003	DVB-S2: LDPC im Rundfunk
2009	**Arıkan: polare Codes**
2009	Duda: asymmetrische Zahlensysteme
2010	Polyanskiy u. a.: Theorie endlicher Blocklängen
2012	Landauer-Grenze experimentell bestätigt
2015	Zstandard
2016	5G: LDPC und polare Codes
2020	Sprachmodelle als Verdichter
2026	Modellierung bleibt das Restproblem

E Kommentierte Quellen

[1] Arıkan, E.: *Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels*. IEEE Trans. Inf. Theory 55(7), 2009.

Der erste Code, für den bewiesen ist, dass er die Kapazität erreicht — einundsechzig Jahre nach Shannons Satz. Der Beweis benutzt Martingalkonvergenz und ist bemerkenswert klar aufgebaut.

[2] Barlow, H.: *Possible Principles Underlying the Transformations of Sensory Messages*. In: Sensory Communication, MIT Press 1961.

Der Aufsatz, der die Informationstheorie in die Sinnesphysiologie brachte. Barlows Vermutung, dass frühe Verarbeitungsstufen Redundanz entfernen, hat die Forschung über Jahrzehnte geprägt und ist bis heute produktiv.

[3] Berger, T.: *Rate Distortion Theory: A Mathematical Basis for Data Compression*. Prentice Hall 1971.

Die maßgebliche Darstellung der verlustbehafteten Verdichtung. Mathematisch anspruchsvoll und ohne Alternative, wenn man die Theorie dahinter genau wissen will.

[4] Berrou, C., Glavieux, A., Thitimajshima, P.: *Near Shannon Limit Error-Correcting Coding and Decoding: Turbo-Codes*. ICC 1993.

Die Arbeit, die vierzig Jahre mühsamen Fortschritt an einem Nachmittag überholte. Wurde zunächst für fehlerhaft gehalten. Ein Lehrstück darüber, was Fachfremdheit gelegentlich bewirkt.

[5] Chaitin, G.: *On the Length of Programs for Computing Finite Binary Sequences*. Journal of the ACM 13(4), 1966.

Unabhängig von Kolmogorow und mit stärkerem Gewicht auf den Folgen für die Unvollständigkeit. Chaitins spätere Arbeiten zur Konstanten Ω sind die greifbarste Form von Gödels Satz, die es gibt.

[6] Cover, T., Thomas, J.: *Elements of Information Theory*. 2. Aufl., Wiley 2006.

Das Standardlehrbuch. *Vollständig, sorgfältig und mit vielen durchgerechneten Beispielen. Deckt alles ab, was in diesem Band steht, und noch einiges mehr. Wenn Sie ein Buch zur Vertiefung wählen, dann dieses.*

[7] Csiszár, I., Körner, J.: *Information Theory: Coding Theorems for Discrete Memoryless Systems*. 2. Aufl., Cambridge University Press 2011.

Die strengste Darstellung der Codierungssätze, mit der Methode der Typen statt der typischen Mengen. Deutlich anspruchsvoller als Cover und Thomas und in den Schranken schärfer.

[8] Duda, J.: *Asymmetric Numeral Systems*. arXiv, ab 2009.

Ein Codierverfahren, das die Optimalität der arithmetischen Codierung mit der Geschwindigkeit von Huffman verbindet. Steckt seit 2015 in Zstandard und in modernen Bildformaten. Ein bemerkenswert später Fortschritt in einem Gebiet, das als abgeschlossen galt.

[9] Gallager, R.: *Low-Density Parity-Check Codes*. Doktorarbeit, MIT 1962.

Dreiunddreißig Jahre zu früh. Enthält die Codes und den iterativen Decoder vollständig, in einer Zeit, in der niemand sie ausführen konnte. Wer wissen will, wie eine Idee auf ihre Technik warten muss, lese das.

[10] Gallager, R.: *Information Theory and Reliable Communication*. Wiley 1968.

Das klassische Lehrbuch vor Cover und Thomas. Bei den Kanalmodellen und den Fehlerexponenten bis heute unübertroffen, im Stil deutlich sperriger.

[11] Hamming, R.: *Error Detecting and Error Correcting Codes*. Bell System Technical Journal 29(2), 1950.

Zwölf Seiten, die aus einem Ärgernis am Wochenende ein Fachgebiet machten. Enthält den Abstandsbegriff, die Konstruktion und die Schranke. Klar geschrieben und ohne Voraussetzungen lesbar.

[12] Hartley, R.: *Transmission of Information*. Bell System Technical Journal 7(3), 1928.

Der Vorläufer, zwanzig Jahre vor Shannon. Hartley maß die Zahl der Möglichkeiten und ließ die Wahrscheinlichkeiten außen vor. Lesenswert, weil man sieht, wie nah man an einer Theorie sein kann, ohne sie zu haben.

[13] Huffman, D.: *A Method for the Construction of Minimum-Redundancy Codes*. Proceedings of the IRE 40(9), 1952.

Drei Seiten. Huffman fand das Verfahren als Student, statt eine Prüfungsarbeit zu schreiben, und bewies dabei nebenbei die Optimalität. Eines der schönsten kurzen Ergebnisse der Informatik.

[14] Kolmogorow, A.: *Three Approaches to the Quantitative Definition of Information*. Problems of Information Transmission 1(1), 1965.

Die Begründung der algorithmischen Informationstheorie. Kolmogorow stellt den kombinatorischen, den wahrscheinlichkeitstheoretischen und den algorithmischen Zugang nebeneinander und zeigt, wie sie zusammenhängen.

[15] Landauer, R.: *Irreversibility and Heat Generation in the Computing Process*. IBM Journal of Research and Development 5(3), 1961.

Sechs Seiten mit einer Aussage, die Physik und Informatik verbindet: Löschen kostet Energie, Rechnen im Grundsatz nicht. Wurde 2012 experimentell bestätigt.

[16] Li, M., Vitányi, P.: *An Introduction to Kolmogorov Complexity and Its Applications*. 4. Aufl., Springer 2019.

Die vollständige Darstellung des Gebiets, von den Grundlagen bis zu Anwendungen in Lerntheorie und Physik. Umfangreich, sorgfältig und ohne ernsthafte Konkurrenz.

[17] Lin, S., Costello, D.: *Error Control Coding*. 2. Aufl., Pearson 2004.

Das umfassende Werk zur Codierungstheorie: algebraische Codes, Faltungscodes, Turbo und LDPC. Ingenieurwissenschaftlich orientiert, mit vielen konkreten Konstruktionen.

[18] MacKay, D.: *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press 2003.

Frei im Netz verfügbar und ein Vergnügen. *Verbindet Informationstheorie mit Statistik und maschinellem Lernen, was sonst niemand in dieser Klarheit tut. Die Kapitel zu LDPC stammen von einem der Wiederentdecker.*

[19] Nyquist, H.: *Certain Topics in Telegraph Transmission Theory*. Transactions of the AIEE 47(2), 1928.

Bestimmte die höchste Signalisierungsrate bei gegebener Bandbreite. Zusammen mit Hartley der technische Vorlauf, aus dem Shannon zwanzig Jahre später eine Theorie machte.

[20] Pierce, J. R.: *An Introduction to Information Theory: Symbols, Signals and Noise*. 2. Aufl., Dover 1980.

Der beste Einstieg ohne Mathematik. *Pierce war Kollege Shannons bei Bell Labs und erklärt das Fach mit einer Anschaulichkeit, die Lehrbücher selten erreichen. Sehr preiswert und in einem Wochenende zu lesen.*

[21] Reed, I., Solomon, G.: *Polynomial Codes over Certain Finite Fields*. Journal of the SIAM 8(2), 1960.

Fünf Seiten für Codes, die heute in jeder CD, jedem QR-Code und jeder Raumsonde stecken. Der Decodieralgorithmus fehlte damals noch; er kam acht Jahre später von Berlekamp und Massey.

[22] Rissanen, J.: *Modeling by Shortest Data Description*. Automatica 14(5), 1978.

Begründet das Prinzip der kürzesten Beschreibung als Verfahren der Modellwahl. Eine formale Fassung von Ockhams Rasiermesser, die ohne Bayessche Vorannahmen auskommt.

[23] Sayood, K.: *Introduction to Data Compression*. 5. Aufl., Morgan Kaufmann 2017.

Das praktische Gegenstück zu Cover und Thomas. Behandelt alle gängigen Verfahren mit vollständig durchgerechneten Beispielen, verlustfrei wie verlustbehaftet. Wer selbst etwas bauen will, greift hierzu.

[24] Shannon, C.: *A Mathematical Theory of Communication*. Bell System Technical Journal 27, 1948.

Die Gründungsarbeit. *Siebenundfünfzig Seiten, in denen ein vollständiges Fach auf einmal dasteht. Frei verfügbar und über weite Strecken erstaunlich gut lesbar. Man sollte sie einmal im Original gesehen haben.*

[25] Shannon, C.: *Communication Theory of Secrecy Systems*. Bell System Technical Journal 28(4), 1949.

Die informationstheoretische Grundlage der Kryptographie, im Krieg entstanden und danach freigegeben. Enthält den Satz über vollkommene Sicherheit und den Begriff der Einheitsdistanz.

[26] Shannon, C.: *Prediction and Entropy of Printed English*. Bell System Technical Journal 30(1), 1951.

Der Rateversuch mit Versuchspersonen. Methodisch elegant, mit einfachsten Mitteln durchgeführt, und die Schätzung hält siebzig Jahre später noch stand. Ein Musterbeispiel dafür, wie man eine schwer zugängliche Größe misst.

[27] Solomonoff, R.: *A Formal Theory of Inductive Inference*. Information and Control 7, 1964.

Kam ein Jahr vor Kolmogorow und von der anderen Seite: Solomonoff suchte eine formale Fassung des induktiven Schließens und stieß dabei auf dieselbe Größe. Der Zusammenhang zwischen Verdichten und Vorhersagen steht hier zum ersten Mal.

[28] Ziv, J., Lempel, A.: *A Universal Algorithm for Sequential Data Compression*. IEEE Trans. Inf. Theory 23(3), 1977.

Ein Verfahren, das ohne jede Kenntnis der Quelle auskommt und trotzdem asymptotisch die Entropie erreicht. Fast jedes heute laufende Verdichtungsprogramm geht darauf zurück.

Wenn Sie nur drei Bücher kaufen

[20]	**Pierce**	Zum Einstieg. Ohne Mathematik, sehr anschaulich, und in einem Wochenende gelesen
[18]	**MacKay**	Zum Lernen. Frei verfügbar, verbindet das Fach mit Statistik und Lernverfahren
[6]	**Cover/Thomas**	Zum Nachschlagen. Das vollständige Standardwerk mit allen Beweisen

Wer danach noch etwas möchte: [24] Shannon im Original, weil man die Gründungsarbeit eines Fachs einmal gesehen haben sollte, und [23] Sayood, wenn es ans Selbermachen geht.